python documentation

as you start your journey as a developer, you must learn how to read documentation. like all popular programming languages, python comes with a rich set of standard libraries you can use. however, you must study the python documentation to familiarize yourself with the language and its different classes, modules, functions, and methods.

some developers also refer to the documentation for a class, module, function, or method as its api. api is a somewhat overloaded acronym for application programming interface that can refer to the documentation or how applications talk to each other. if someone asks whether you looked at the list api, they want to know if you read the python documentation for the list class. however, if they want to know how to use google's search api, they want you to tell them how those search services can be used programmatically.

in this section, we'll look briefly at the python documentation. since the documentation sometimes changes structure and appearance, this quick look doesn't use images or many details of the documentation's organization.

the python docs website, as of august 2023, has a button near the top labeled "python docs." this button takes you to the most recent version of the documentation, which may not be the same as your python version. if the version number matters, view the documentation releases by version.

the main page for each documentation version should show a high-level table of contents. however, finding the information you need may not be painless even with that table of contents. for the purposes of this book, you can focus on these parts:

the language reference explores the language itself. oddly, the most useful sections for beginners are near the bottom of the page (as of december 2023):
expressions
simple statements
compound statements
the library reference covers all built-in classes and modules. the most valuable sections (as of december 2023) are:
built-in types
built-in functions
built-in constants
turn to the glossary if you need unfamiliar terms explained.
you can use the complete table of contents if you know the right words. your browser's in-window search will help you quickly find the appropriate entry.
we'll refer you to the documentation to highlight special items of interest when the information will benefit you most. for now, bookmark the documentation and try to familiarize yourself with its structure.

it's worth noting that the documentation is not always helpful, even when you find the right page. you can search the web for a better explanation if necessary. be careful, though: searches often turn up outdated or incorrect content, so focus on recent items and try to look at several pages for conflicting information. ai tools like chatgpt may also be helpful, but be extra careful: ai tools may lie to you. convincingly.

using python

in this chapter, we'll show you how to run python code. we'll also describe some basic python coding conventions and briefly discuss python packages. finally, we'll look at a basic debugging technique. none of this information is exhaustive, but it should help you get through the rest of the book.

there are two main ways to run python code with the python command:

you can use a special line-by-line execution mode of python to execute code one line at a time. this mode is most helpful in determining what built-in functions and methods will do under different circumstances.

you can use the python command to run the code from a file on your disk. this is the most common way to run python code.
note that we will be showing you some python code in this chapter without thoroughly explaining what the code does. what's important in this chapter is learning how to use python, whether it's using a repl or running a python program from a file. we'll also briefly talk about indentation, continuations, and using the print function.

indentation
before we begin doing anything with python -- even running some trivial code -- we need to talk about indentation. python is unusual among programming languages; it uses indentation to understand your code.

most programming languages have little care about indentation. while most programmers indent their code carefully, this choice primarily concerns readability. simple carelessness, though, often leads to code that is inconsistently or improperly indented. however, the language interpreter or compiler doesn't care about indentation. the code still works, no matter how it's indented. for instance, in the javascript language, you'll encounter code like this:

copy code
```
function doublearray(array) {
  // some code here
  for (let index = 0; index < array.length; index += 1) {
    // some more code here
    array[index] = 2 * array[index];
  }
}
```
everything is neatly indented to show the structure of the code. the main program is flush against the left margin, the function body is indented by two spaces, and the body of the for loop is indented by another two spaces. however, javascript doesn't care. the following code is equally valid but less readable:

copy code
```
function doublearray(array) {
// some code here
for (let index = 0; index < array.length; index += 1) {
// some more code here
array[index] = 2 * array[index];
}
}
```
this code is also perfectly acceptable to javascript:

copy code
```
          function doublearray(array) {
// some code here
   for (let index = 0; index < array.length; index += 1) {
                               // some more code here
                     array[index] = 2 * array[index];
            }
   }
```
python is different. the code must be indented and indented consistently in any given "block" of code. for instance, the python equivalent of the above code looks like this:

copy code
```
def double_list(input_list):
    # some code here
    for index in range(len(input_list)):
        # some more code here
        input_list[index] = 2 * input_list[index]
```
the function definition (def) is flush left. the function's body is indented by 4 spaces, and the body of the for loop is indented another 4 spaces. if you try

to rewrite that code without the indentation, python will raise an error:

```
copy code
def double_list(input_list):
# some code here
for index in range(len(input_list)):
    # some more code here
    input_list[index] = 2 * input_list[index]
# indentationerror: expected an indented block after
# function definition on line 1
```

another thing to note: most languages delineate the beginning and end of the code inside multi-line statements like function definitions and loops. javascript and many other languages use the { and } characters to mark a block's beginning and end. other languages use keywords like do and end to do the same thing. python doesn't do this. okay, it does use a : to mark the beginning of a block of code, but the block ends only when the indentation is reduced:

```
copy code
def double_list(input_list):
    for index in range(len(input_list)):
        input_list[index] = 2 * input_list[index]

    return input_list
```

in this code, the return statement is not part of the for loop since its indentation is less than that of the loop's body. the indentation marks the beginning and end of the body of a multi-line statement.

if you've programmed in other languages, using indentation in python may initially seem quirky. however, once you get used to it, you'll wonder why other languages do it differently.

using the repl

let's first discuss how to execute python code line-by-line. we'll use python's built-in repl (read-eval-print loop). the repl is an interactive environment where you type python statements and expressions. you will see the results immediately. python has a built-in repl that's ideal for beginners exploring the language.

repls are programs that let you write and execute small snippets of code interactively. you can use python's repl to quickly try code to see what happens.

python has several repls, but the most basic is the built-in repl. the python command invokes the built-in repl when run without arguments. for instance:

```
$ python
python 3.11.2
type "help", "copyright", "credits" or "license" for more
information.
>>>
```

the last line (>>>) is the repl's prompt. you can enter a python statement or expression after the prompt and see its output or return value, if any, below the prompt. it also distinguishes between what you type and the resulting output. you can enter as much python code as you want, one at a time. each time you enter a complete statement or expression, python executes it and displays any output or return values produced.

as with shell commands that produce output, we disable the copy code functionality when depicting repl sessions.

the repl's prompt may vary depending on your system configuration, so it may look different on your computer. that's okay. just ignore the leading >>> when it appears in a code block. do not type it. for example, in the following, you will type print('hello world!') at the >>> prompt, then press the return or

enter key. the repl will then display the output produced by print(...) (hello world!) followed by a new >>> prompt.

```
>>> print('hello world!')
hello world!
>>>
```

you don't have to use print explicitly if it's the last line entered. the repl outputs the evaluated value of the last line of code to execute. in fact, this is a great way to perform quick mathematical calculations:

```
>>> 4 * 9 + 6
42
>>>
```

multi-line statements, such as if statements, can be tricky since you must remember to indent code properly and use colons (:) where required. you also need to press the return or enter key once more to show that you are done typing. it looks like this:

```
>>> a = 42    # this is assignment; we'll discuss it later
>>> if a > 42:
...     print('no')
... else:
...     print('yes')
...
yes
>>>
```

remember: indentation is important in python.

notice that the continuation lines in multi-line statements use ... as a prompt. as with >>>, the actual prompt may vary. you do not type the prompt, no matter what the prompt looks like.

you can also define and call functions in the repl:

```
>>> def add_three(value):
...     return value + 3
...
>>> add_three(39)
42
```

the help function
the help function lets you request help in the python repl:

display a short summary of how to use help:

copy code
help()
type q and press {return} to exit help.

display a short summary of how to use the len function:

copy code
help(len)
display a summary of the str class:

copy code
help(str)
display a summary of the bool class by passing help a boolean value:

copy code
help(3 == 4)
help is very flexible and will try to give you helpful information, no matter what value you pass it. you can't always predict how it might interpret some input, but it's reasonably good at guessing your intent.

additional repl tips
to exit the repl, type exit() or quit() followed by the return or enter key. press the control-d keyboard combination for an even easier way to close the repl.

as of python 3.13.0, you only have to type exit or quit; you may omit the parentheses.

use the up and down arrow keys to navigate through your repl history. you can use the left and right arrow keys to position the cursor and begin editing some prior input. (this may not work if your python was built without the proper "readline" support.)

the repl is a great way to experiment and learn with python. you can quickly test out ideas, understand how functions behave, or even debug code snippets without the overhead of creating a full-fledged script.

copying and pasting in the python repl
copying and pasting code into the python repl isn't reliable. in particular, if all 3 of the following statements are true, pasting into the repl probably won't work:

at least one line of code is indented.
one of the indented lines is followed by one or more empty lines.
the line after the empty line(s) is indented less than the original indented line or not indented at all.
you will probably get an indentation error on the line after the empty lines.

unfortunately, there is no way to fix the copy and paste issue in python prior to 3.13.0. however, there are at least two workarounds:

use coderpad. you can paste code into the repl side of the coderpad workspace (the right side) and it will work properly. you can also paste into the left side of the workspace, but you'll have to click the run button. coderpad is a good choice since we use it for assessments. however, there are some limitations with coderpad, such as not being able to install packages.
use jupyter notebook or jupyterlab. just paste your code into a notebook, then run it. (we'll discuss these two products a bit later.)
python 3.13.0 introduced a new feature in the repl that makes pasting code work more reliably. to paste code cleanly in the repl, first press the {f3} key. this should change the prompt to (paste). you can now paste your code. when you are done pasting, press the {f3} key again. you may need to press {return} to get back to the >>> prompt.

running python code from a file
suppose you have a file named example.py that contains some python code you want to execute. for instance, assume the file contains the following code:

example.pycopy code
print('i wish to complain about this parrot.')
print("it's probably pining for the fjords!")
once you've saved this file, you can run it as follows:

copy code
$ python example.py
i wish to complain about this parrot.
it's probably pining for the fjords!
note that files with python code typically have a .py extension.

when you run a python file from the command line, the python compiler reads and converts the code into a form the computer understands. the compiled code is called byte code. once the byte code is available, it gets passed to the interpreter, which executes the code. you don't need to know much about the

compiler and interpreter except that you should recognize the terms.

one last tip: some python programs may run for a long time, maybe infinitely. that may be intentional or due to a logic error. regardless, you can use the control-c keyboard combination to terminate such programs. control-c sends an interrupt signal to the running program, which usually ends the program. for instance, try running the program in this endless_loop.py file:

copy code
```
import time

while true:
    time.sleep(1)
    print(time.asctime())
```
press the control-c keyboard combination when you tire of watching the endless output.

the python community has developed stylistic conventions that help make python code easier to read and write. there are actually multiple conventions, and the different guidelines sometimes disagree. however, there's plenty of overlap in the recommendations.

adhering to the style conventions of a programming language is helpful and meaningful even when you disagree with some recommendations. you probably won't be the sole person developing and maintaining a software project; sticking to a particular style convention helps your teammates and future maintainers understand your code. it's hard enough to understand code written by someone else; don't make it more challenging with unusual or non-standard stylistic choices.

the conventions discussed in this section are specific to the python community. other programming languages -- and even some python sub-communities -- may have different preferences about each guideline.

here's a short list of guidelines that we recommend. if you already work with a formal set of guidelines, please feel free to use them. if you don't, our suggestions should help you write readable code.

set your text editor to use four space characters -- not tabs -- for indentation.

set your text editor to expand tab characters to spaces.

limit code lines and comments to 79 characters. this guideline isn't universal, but it helps readability. not all developers have massive screens or good eyesight, so be kind and stick to this guideline.

use spaces around operators, except for **, which should not be surrounded by spaces:

correct spacingcopy code
```
print(3 + 4)    # + is an operator
print(5 * 7)    # * is an operator
print(6 - 2)    # - is an operator
print(7 / 3)    # / is an operator
print(8**3)     # ** operator is not surrounded by spaces
my_num = 3      # = is an operator
```
incorrect spacingcopy code
```
print(3+4)
print(5* 7)
print(6 -2)
print(7/3)
print(8 ** 3)   # don't use spaces with **
my_num=3
```

python uses the # character as the beginning of a comment. comments run through the end of the line.

copy code
```
# this line only has a comment
ultimate_answer = 42        # initializing variable
a_round_number = 3.141592   # another initialization
```
programmers use comments to leave notes for other programmers or themselves at a later point in time. however, don't overdo your comments. let your code do the talking instead.

some python language features use code blocks -- one or more lines of code that are treated as a unit. the statement introducing a block of code ends with a :. in contrast, the block is indented with 4 spaces. the block ends when you outdent back to the original level.

copy code
```
if is_ok():
    print('first thing')
    print('another thing')
    print('yet another thing')
print('this is not part of the above block')

if is_something():
    print('do something')
    print('do another thing')
else:
    print('do something else')

while try_again():
    print('first thing')
    print('another thing')
    print('yet another thing')
print('the loop has ended.')
```
you can also have multiple levels of indentation, though you should avoid using more than two levels:

copy code
```
if value <= 10:
    print('value <= 10')
    if value >= 5:
        print('value >= 5')
    else:
        print('value < 5')
else:
    print('value > 10')
```
use spaces between operators and operands to make your code less cluttered and easier to read:

copy code
```
sum=x+5             # bad
sum = x + 5         # good
```

peps

the previous section covers the essential style conventions you need to get started. if you want more information about python styling, we recommend the style guide for python code, otherwise known as pep 8. check it out, but don't try to memorize all of the rules right away.

pep stands for python enhancement proposal. peps are numbered design documents that provide information and recommendations to the python community, information about new or proposed language features, and standards and conventions.

pep 8 is perhaps the best-known and most widely used pep. it is the basis for most python style conventions and should be reviewed occasionally by python developers.

continuation lines
if you checked out pep8, you may have noticed that one of the stylistic conventions in python is that code lines should be no more than 79 characters in length. that means that you need to learn how to continue code over multiple lines. this can get a little tricky, but there are several ways to continue code over multiple lines:

string literals can be continued over several lines, provided you enclose all the string content in a set of parentheses:

copy code
```
return ("this is a long string. "
        "it's actually very long. "
        "it spans multiple lines. "
        "are you getting tired of this? "
        "so am i.")
```
this also works with f-strings and r-strings, which we'll encounter later.

sometimes, a triple quoted string is easier to deal with:

copy code
```
return """
    this is a long string.
    it's actually very long.
    it spans multiple lines.
    are you getting tired of this?
    so am i.
"""
```
it's worth noting that triple-quoted strings preserve leading whitespace and any newline characters. you also don't need parentheses.

multi-value literals, such as lists and tuples, can be written over several lines by just using line-breaks between the brackets, braces, or parentheses used by the literal:

list continuationcopy code
```
my_list = [
    "antonina",
    "brandi",
    "trevor",
]
```
tuple continuationcopy code
```
my_tuple = (
    3.141592,
    2.718282,
    1.414213,
)
```
set continuationcopy code
```
my_set = {
    "dog",
    "cat",
    "pet",
}
```
when writing collection literals over multiple lines like this, the convention is to end every item in the list with a comma, including the last element. this convention makes it easy to add new elements to the end of the collection, and also permits easy sorting and rearranging.

you can use parentheses to delimit code that will be split over multiple lines:

```
copy code
return (obj.bar1
      + obj.bar2
      + obj.bar3)
```
you can use a backslash to end any line you want to continue:

```
copy code
result = value1 + \
         value2 + \
         value3
```
a good rule of thumb is that you can use a backslash wherever you can put a space.

that said, you should use backslash continuations sparingly, and only where they improve readability. you should not use backslashes when they aren't syntactically required for continuation. it's usually better to use one of the other continuation techniques.

the trickiest part of using continuations is often how to indent the continuation lines, especially when using if and while statements and comprehensions. the only rule we can offer is to aim for readability and consistency. the continued code should clearly be connected to the first line, and shouldn't blend into any following code blocks. we'll show some readable examples in this book and the core curriculum.

using jupyterlab/jupyter notebook

jupyterlab and jupyter notebook are alternatives to the standard python repl. both can also operate as a complete python program editor and documentation generator. jupyter offers many advantages over using the built-in repl. the most significant benefits may be that you can write and try it out, copy and paste code from launch school materials and elsewhere, and much more. the copy-and-paste functionality is especially handy as the built-in python repl badly mishandles blank lines in indented code when pasting. the built-in repl is effectively unusable when you want to copy and paste code that has blank lines in indented code.

the easiest way to use jupyter is to use one of the experimental browser-based versions of jupyterlab and jupyter notebook at the try jupyter page. you don't have to install anything -- not even python. all you need is your browser. however, you won't have any control over what version of python you are using. to see what version of python is currently being used by one of these experimental programs, run the following code in a notebook:

```
copy code
import platform
platform.python_version()
```
if you would rather install your own version of jupyter, you can do so on macs, windows (wsl2 preferred), and most linux systems. note that we don't recommend installing jupyter on aws cloud9; the process is currently too complicated. it doesn't play well with cloud9's "preview" browser.

recent versions of linux operating systems (including the wsl2 version of ubuntu) are starting to implement a security measure that prevents installing packages on top of a system-controlled version of python. that means that installing jupyter will fail on these systems. later, in py101, we'll show you how to set up a virtual environment that will let you install jupyter on these systems. you can skip the rest of this section unless you are using a macintosh with homebrew.

on mac systems, you can easily install jupyterlab with homebrew:

```
copy code
brew install jupyterlab
```
once installed, you can start jupyter with one of these commands:

copy code
jupyter lab
jupyter notebook
the program will open in your browser, after which you can create a new notebook and write some code. when you want to execute the code, press {shift}{enter}.
debugging python code with print
python has a nifty built-in module called pdb that lets you step through programs to see what happens on each line of code. this is a fantastic way to debug programs. however, before using pdb to debug a program, you must learn how to use it. now isn't the time to do that. we'll learn how to use pdb later in the core curriculum.

for now, the easiest way to debug python programs is to add print statements at strategic points in your program. these print statements can help quickly identify where and why your program isn't working.

suppose we are writing a program that displays the maximum numeric value from a list of numeric strings:

compute_max.pycopy code
```
max = '0'
for number in ['10', '2', '34', '6', '25']:
    if number > max:
        max = number

print('max value is', max)
```
some of you may already see the problem, but not everyone will. let's walk through using print to debug this code.

first, let's run the code:

```
$ python compute_max.py
max value is 6
```
this answer isn't correct - the maximum numeric value should be 34, not 6. let's use print to try to diagnose the problem. there are two crucial values we need to see what is happening, max and number, so we'll print both values during each iteration of the loop:

compute_max.pycopy code
```
max = '0'
for number in ['10', '2', '34', '6', '25']:
    print('max =', max, 'number =', number)
    if number > max:
        max = number

print('max value is', max)
```
```
$ python compute_max.py
max = 0 number = 10
max = 10 number = 2
max = 2 number = 34
max = 34 number = 6
max = 6 number = 25
max value is 6
```
this output shows that max was 0 and number was 10 on the first iteration. (remember, though, that these values are strings.) on the second iteration, max was 10 and number was 2. this seems reasonable: 10 > 0, so max gets set to 10. however, on the 3rd iteration, we see that max is 2. that can't be right: 2 < 10, so max should still be equal to 10.

let's display the value of number > max to get a better idea of what's happening:

compute_max.pycopy code

```
max = '0'
for number in ['10', '2', '34', '6', '25']:
    print('max =', max, 'number =', number,
          'number > max =', number > max)
    if number > max:
        max = number

print('max value is', max)
$ python compute_max.py
max = 0 number = 10 number > max = true
max = 10 number = 2 number > max = true
max = 2 number = 34 number > max = true
max = 34 number = 6 number > max = true
max = 6 number = 25 number > max = false
max value is 6
```
hmm. it looks like the comparison is always true except when we check 6 against 25. this is a huge clue. it tells us we're comparing number and max as strings, not numbers. strings are compared lexicographically, e.g., character by character, so '25' < '6' while '6' > '34'. that's because '2' < '6' and '6' > '3'. to fix this program, we must compare the values numerically. we can do this by converting the strings to integers when comparing them.

```
compute_max.pycopy code
max = '0'
for number in ['10', '2', '34', '6', '25']:
    print('max =', max, 'number =', number,
          'number > max =', number > max)
    if int(number) > int(max):
        max = number

print('max value is', max)
$ python compute_max.py
max = 0 number = 10 number > max = true
max = 10 number = 2 number > max = true
max = 10 number = 34 number > max = true
max = 34 number = 6 number > max = true
max = 34 number = 25 number > max = false
max value is 34
```
now we're getting the correct answer: 34 is the largest numeric value in the list.

before we leave this problem, let's go ahead and delete the debugging code (the print statement) and run the program one last time:

```
compute_max.pycopy code
max = '0'
for number in ['10', '2', '34', '6', '25']:
    # print statement deleted
    if int(number) > int(max):
        max = number

print('max value is', max)
$ python compute_max.py
max value is 34
```
it's worth your time to learn to use this debugging technique. in particular, you may need it in coderpad, which we use for interview assessments. coderpad does not support debugging tools, so print is the only way to go.

data types

a core job of most programming languages is dealing with data. everything you do in a computer program involves data in some way. data in any programming language has different data types. data types help programmers and their programs determine what they can and cannot do with a given piece of data. this chapter takes a quick tour of python's basic data types. everything you do in

python involves data and data types.

everything with a value in python is said to be an object. furthermore, each object has an associated data type or, more concisely, type, which has an associated class. for instance, the objects 42 and 123 are instances of the integer type, i.e., the int class. similarly, the values true and false are instances of the boolean type, i.e., the bool class.

in python, the terms object and value can be used interchangeably, as can the terms class, data type, and type. we will use them that way, too.

data types
python has a large number of built-in data types compared to many other languages:

data type	class	category	kind	mutable
integers	int	numerics	primitive	no
floats	float	numerics	primitive	no
boolean	bool	booleans	primitive	no
strings	str	text sequences	primitive	no
ranges	range	sequences	non-primitive	no
tuples	tuple	sequences	non-primitive	no
lists	list	sequences	non-primitive	yes
dictionaries	dict	mappings	non-primitive	yes
sets	set	sets	non-primitive	yes
frozen sets	frozenset	sets	non-primitive	no
functions	function	functions	non-primitive	yes
nonetype	nonetype	nulls	--?--	no

this table shows just a fraction of the available data types. in this book, you will learn about all these types.

the first 4 data types are python's so-called primitive types. primitive types are the most fundamental data types in a language. almost all data in a language is built up from these primitive types. for instance, the sequence and mapping types may include primitives and complex types built from those primitives.

some programmers think of none as a primitive type, but that's debatable. we're agnostic on the matter.

the sequences, mappings, and sets are all non-primitive types. in particular, they are also called collection types since each collects multiple items in a single object. we'll refer to these types as collections.

though functions are non-primitives, they are not collections.

the final column of the table shows which types are mutable and which are not. mutable types are types whose objects can be altered after they are created. so, for example, we can add, remove, and replace elements in a list. immutable types can't be altered after creation. instead, you must create a new object with a different value. we'll explore this distinction and its subtleties in greater detail in the variables as pointers chapter.

in this chapter, we'll focus on the primitive types. however, we'll also make a nodding acquaintance with the non-primitive types. first, though, let's talk about literals, and then we'll meet the different primitive types.

literals
you can use literals to represent most data type values. a literal is any syntactic notation that lets you directly represent an object in source code. for instance, all of the following are literals in python:

copy code
```
'hello, world!'   # str literal
3.141592          # float literal
```

```
true              # bool literal
{'a': 1, 'b': 2}  # dict literal
[1, 2, 3]         # list literal
(4, 5, 6)         # tuple literal
{7, 8, 9}         # set literal
```
not all objects have literal forms. in those cases, we use the type constructor to create objects of the type. for instance:

copy code
```
range(10)         # range of numbers: 0-9
range(1, 11)      # range of numbers: 1-10
set()             # empty set
frozenset([1, 2]) # frozen set of values: 1 and 2
```
numeric values
numeric values represent numbers. numbers can be added, subtracted, multiplied, and divided and can be used in a wide variety of mathematical operations. python also supports other numeric types, such as complex, decimal, and fractional numbers.

some programming languages treat integers and floating point numbers as a single numeric data type. python uses different data types to represent numbers. let's talk about integers first.

integers
the integer data type (int) represents integers, i.e., the whole numbers (..., -3, -2, -1, 0, 1, 2, 3, ...). integers do not have a fractional (or decimal) part.

examples of integer literalscopy code
```
1
2
-3
234891234
131_587_465_014_410_491
```
note that you can't use commas or periods for grouping: neither 123,456,789 nor 123.456.789 is valid. instead, you can write the number without separators (123456789) or break up the number with underscores (123_456_789).

there is no pre-defined limit to the size of an integer. you can have integers with as many digits as will fit in memory, though you may not be able to do much with them then.

floats
the floating point data type (float) represents floating point numbers, aka, the real numbers. floating point numbers include integers and numbers with digits after the decimal point.

examples of float literalscopy code
```
1.0
1.4142
-3.14159
42348.912346
131_587_465.014_410_491
```
as with integers, you can't use commas or periods for grouping: neither 42,348,912.346 nor 42.348.912,346 is valid. instead, you can write the number with only a single decimal point (42348.912346) or break up the number with underscores (42_348.912_346).

working with big and small floats
this section is optional. you probably won't need this information unless you're working on a math- or science-intensive application.

in theory, floats can represent any number, with or without a decimal point. in practice, there are limits that can be looked up in the sys.float_info module as

follows:

copy code
```
import sys

# the maximum number of digits that can be accurately
# presented
print(sys.float_info.dig)          # typically 15

# the largest possible positive float value
print(sys.float_info.max)          # about 1.8 * (10**308)

# the smallest non-zero positive float value
print(sys.float_info.min)          # about 2.2 * (10**-308)
```
don't worry about memorizing those values.

note that 10**n, where n is positive, represents a 1 followed by n zeros. thus, 10**308 is a 1 followed by 308 zeros; a truly enormous number. on the other hand, 10**-n is equivalent to the reciprocal of 10**n, e.g., 1.0 / 10**n. thus, 10**-308 is a truly minuscule number.

python prints extremely large and small floats using scientific notation:

copy code
```
print(3.14 * (10**20))        # 3.14e+20
print(3.14 * (10**-20))       # 3.14e-20
```
you can read the e+20 as meaning 10 to the 20th power, and e-20 as the reciprocal of 10 to the 20th power.

you can also use scientific notation when writing float literals:

copy code
```
print(3.14e+20 / 2.72e-15)    # 1.1544117647058823e+35
```
it's worth pointing out that these floating point issues are not present when working with integers. integers can be as small or large as you want, provided they fit in memory. furthermore, integers don't get printed with scientific notation.

copy code
```
print(3 * (10**20))           # 300000000000000000000
```
if you want to use scientific notation to write a large integer, you must use the int function to convert the number to an integer:

copy code
```
print(int(3e+20))             # 300000000000000000000
```

variables and assignment
most programming languages use something called variables. variables are merely names used to identify values (that is, data). in fact, they are also called identifiers, a more general term that applies to variables, constants, function names, class names, and more.

variables in python are usually created by initializing them to a value. for instance, consider the following code:

copy code
```
pi = 3.141592653589793
life_the_universe_and_everything = 42
```
here we've created two variables, pi and life_the_universe_and_everything, then initialized them to the values 3.141592653589793 and 42 respectively.

the syntax used when creating variables, e.g., foo = 'abc', is called assignment. we can say that the variable foo is assigned to the value 'abc' or that the value 'abc' is assigned to the variable foo. both phrases mean the same

thing, but the former is usually preferred. however, the latter phrasing is perfectly acceptable. we use both phrasings at launch school, depending on what reads better in any particular context.

if a particular variable is being assigned a value for the very first time, we may call the assignment an initialization. if the variable has already been initialized, we should refer to the assignment as a reassignment. for example:

copy code
```
foo = 'hello, world.'         # assignment or initialization
foo = "that's all, folks!"    # reassignment
```
we'll discuss variables and assignment in much more detail later on.

boolean values
boolean values represent binary states such as true or false, on or off, yes or no, one or zero, and so on. the only boolean values are true and false.

in python, booleans sometimes act like numeric values: you can use them in mathematical and comparison expressions. you shouldn't do that, really. however, you can always play with it in the repl:

```
>>> 3 + true
4

>>> true == 1
true
```
later, we will discuss a related but different concept: truthiness. while boolean values are also truthiness values, truthiness values often aren't boolean.

suppose you want to represent the state of a light switch in your application. you can use boolean values: true for on and false for off.

copy code
```
toggle_on = true
session_active = false
```
boolean values have a starring role when working with comparison operators. we'll talk about comparisons in the next chapter. for now, it's enough to know that comparisons check whether a value is equal to, less than, or greater than another value. they return a boolean result.

```
>>> 5 == 5
true

>>> 100 < 99
false
```
text sequences
text sequences are strings of characters, such as words, sentences, dates, foreign text, and so on. they also include things like byte strings, which are sequences of byte-sized values. we don't mention byte strings again in this book.

example text sequence valuescopy code
```
'hello!'
"he's pining for the fjords!"
'1969-07-20'
```
a string is a text sequence of unicode characters. developers often work with text data like names, messages, and descriptions. python uses strings to represent such data.

text sequences can often be treated as ordinary sequences. for instance:

copy code
```
greeting = 'hello'
```

```
for letter in greeting:
    print(letter)
# h
# e
# l
# l
# o
```
there is one significant difference between a text sequence and an ordinary sequence. ordinary sequences contain zero or more objects, but a text sequence does not contain any objects: it simply contains the characters (or bytes) that make up the text sequence. those characters or bytes are not objects; they are simply part of the value.

string literals and dealing with quotes
you can write string literals with single, double, or triple quotes.

copy code
```
'hi there'                      # single quotes
"monty python's flying circus"  # double quotes

# triple single quotes
'''king arthur: "what is your name?"
black knight: "none shall pass!"
king arthur: "what is your quest?"
black knight: "i have no quarrel with you,
               but i must cross this bridge."
'''

# triple double quotes
"""man: "is this the right room for an argument?"
other man: "i've told you once."
man: "no you haven't!"
"""
```
all of these forms create strings in the same way. most python programmers use single quotes as the primary way to write string literals but turn to double quotes when the string's value contains single quotes. however, triple quotes are often used for multi-line strings and a special kind of comment that we won't get into.

you may sometimes need a string that contains both single and double quotes. for instance, if you want to print my nickname is "wolfy". what's yours?, you must consider the fact that the string has both single and double quotes. you can use triple quotes with such a string, or you can escape one of the quote characters like so:

copy code
```
print("""my nickname is "wolfy". what's yours?""")
print('my nickname is "wolfy". what\'s yours?')
print("my nickname is \"wolfy\". what's yours?")
```
the backslash, or escape character (\), tells the computer that the next character isn't syntactic but is part of the string. escaping a quote prevents python from seeing it as the string's end. pay attention to the type of slash you use: a forward slash (/) has no significance between quotes. if you want a literal backslash in your string, you must double up the backslashes or use a so-called raw-string literal (we'll discuss raw strings shortly):

copy code
```
print("c:\\users\\xyzzy")  # each \\ produces a literal \
```
indexing strings
you can access the individual characters in a string with the [] indexing syntax. the value between the brackets must be an integer between 0 and the length of the string minus 1:

```
>>> my_str = 'abc'
```

```
>>> my_str[0]
a

>>> my_str[1]
b

>>> my_str[2]
c

>>> my_str[4]
indexerror: string index out of range
```
you can also use negative integers to access characters based on the distance from the end of the string. for instance, my_str[-1] returns the last character in the string, while my_str[-2] returns the next to last character. the index of the first character is given by -len(my_str):

```
>>> my_str = 'abc'
>>> my_str[-1]
c

>>> my_str[-2]
b

>>> my_str[-3]
a

>>> my_str[-4]
indexerror: string index out of range
```

raw strings and f-strings
string literals with an r prefix are raw string literals. raw string literals don't recognize escapes, so you can use literal \ characters freely.

copy code
```
# both of these print c:\users\xyzzy
print("c:\\users\\xyzzy")  # each \\ produces a literal \
print(r"c:\users\xyzzy")  # raw string literal
```
raw strings are most often used with windows-style file names and regular expressions. we cover regular expressions in our regular expressions book.

another prefix you can use with string literals is f. it's a recent python addition for defining formatted string literals or, more succinctly, f-strings. f-strings enable an operation called string interpolation. you create an f-string by preceding the opening quote with the letter f:

```
>>> f'5 plus 5 equals {5 + 5}.'
'5 plus 5 equals 10.'

>>> my_name = 'karl'
>>> f'my name is {my_name}.'
'my name is karl.'

>>> my_name = 'clare'
>>> greeting = 'ey up?'
>>> f'{greeting} my name is {my_name}.'
ey up? my name is clare.
```
string interpolation is a handy way to merge python expressions with strings. the basic syntax is:

copy code
```
f'blah {expression} blah.'
```
python replaces the {expression} substring with the value of the expression inside the braces: it interpolates the expression's value. you'll learn to love this feature; it saves time and effort. however, remember that string

interpolation is a feature of f-strings only. it doesn't work if you forget the f or omit the braces.

you may include as many {expression} substrings as needed in an f-string. if you need literal { or } characters in an f-string, you can escape them by doubling the { and } characters you want to use as literal characters:

copy code
```
foo = 'hello'
print(f'use {{foo}} to embed a string: {foo}')
# use {foo} to embed a string: hello
```
in the above example, we have an f-string that embeds both {{foo}} and {foo}. the former gets interpreted as the literal string {foo}, while the latter gets replaced by the value of foo.

we have two more f-string variants. earlier in this chapter, we discussed the use of underscores in numbers to make them more readable, e.g., 12_345_678. however, if you print a number, no matter its size, it won't be printed with underscores (or even commas) unless you use an appropriate f-string:

copy code
```
print(f'{123456789:_}')       # 123_456_789
print(f'{123456789:,}')       # 123,456,789
```
if you use this special form of f-string on a floating point number, only the part before the decimal point will be formatted as expected:

copy code
```
print(f'{123456.7890123:_}')  # 123_456.7890123
print(f'{123456.7890123:,}')  # 123,456.7890123
```
functions

functions are chunks of standalone, reusable python code designed to perform an action. they provide a way to break up your program into modular bits that enable a high degree of code reuse.

one thing that may surprise us is that python functions have a data type. many, but not all, languages share this characteristic. we won't discuss this characteristic in this book. however, we will do so in the core curriculum.

none: when you want nothing

in programming, we need a way to express the absence of a value. in python, we do this with none. it can also indicate missing, unset, or unavailable data and may sometimes be an error indication.

none is a literal whose value is the lone representative of the nonetype class.

sequences

sequences represent an ordered collection of objects. a sequence's objects can be accessed using a numeric index. in many other languages, the array is the best-known sequence type, but python uses lists to fill the same role. however, tuples and ranges are also important types.

lists and tuples

lists and tuples are so similar that we'll cover them together.

python organizes information into ordered lists using lists and tuples. lists and tuples may contain any objects.

list literals use square brackets [] surrounding a comma-delimited list of values, while tuples use parentheses (). the comma-delimited list values are known as elements. let's make a list and a tuple:

```
>>> my_list = [1, 'xyz', true, [2, 3, 4]]
>>> my_list
[1, 'xyz', true, [2, 3, 4]]
```

```
>>> tup = ('xyz', [2, 3, 4], 1, true)
>>> tup
('xyz', [2, 3, 4], 1, true)
```
notice that both objects contain a mix of element types, including another list ([2, 3, 4]).

list and tuple literals may also use a multi-line format, which is especially useful for more extensive or nested collections:

copy code
```
[ # begin multi-line list literal
    "monty python's flying circus",
    ( # begin multi-line tuple literal
      'eric idle',
      'john cleese',
      'terry gilliam',
      'graham chapman',
      'michael palin',
      'terry jones',
    ), # end multi-line tuple literal
] # end multi-line list literal
```
the trailing comma on the last items in the list and tuple are optional and common practice when writing multi-line collection literals, such as lists, tuples, and dictionaries. try to be consistent if you use it. it should only be used in multi-line literals.

you can access objects in a list or tuple with the [] indexing syntax. the value between the brackets must be an integer between 0 and the length of the sequence minus 1:

```
>>> my_list = [1, 'xyz', true, [2, 3, 4]]
>>> my_list[0]
1

>>> my_list[1]
'xyz'

>>> my_list[2]
true

>>> my_list[3]
[2, 3, 4]

>>> my_list[4]
indexerror: list index out of range
```
a crucial distinction between lists and tuples is that lists are mutable. tuples, however, are immutable.

one way to mutate a list is to use the indexing syntax to the left of the = in an assignment. this is called indexed reassignment or element reassignment. (we usually prefer element reassignment, but you will often hear and read about indexed reassignment.) for instance:

```
>>> my_list[3] = 'new value'
>>> my_list
[1, 'xyz', true, 'new value']

>>> tup[3] = 'new value'
typeerror: 'tuple' object does not support item
assignment
```
the immutability of tuples limits their usefulness slightly, but python can perform more optimizations on tuples. the optimizations can reduce storage requirements and improve performance.

here's a bit of python weirdness: if you want to use a literal to create a tuple with one element, you can't simply write:

copy code

```
my_tuple = (1)
print(type(my_tuple))        # <class 'int'>
```

in this case, the parentheses cause python to treat the expression as a parenthesized expression, not as a tuple literal. to define a one-element tuple, you must place a comma after the element value:

copy code

```
my_tuple = (1,)
print(type(my_tuple))        # <class 'tuple'>
```

the most important facts to remember about lists and tuples are:

the order of the elements is significant.
lists are mutable; tuples are immutable.
use indexing syntax to retrieve specific elements.
use indexing syntax to reassign specific list elements.
index numbers are non-negative integers starting from 0 and ending at the sequence's length minus 1. (later, we will learn that negative integers are also allowed. for now, though, we'll only use non-negative integers.)

ranges
a range is a sequence of integers between two endpoints. ranges are most commonly used to iterate over an increasing or decreasing range of integers. let's see how we can create some ranges:

```
>>> tuple(range(5))
(0, 1, 2, 3, 4)

>>> tuple(range(5, 10))
(5, 6, 7, 8, 9)

>>> list(range(1, 10, 2))
[1, 3, 5, 7, 9]

>>> list(range(0, -5, -1))
[0, -1, -2, -3, -4]
```

with one argument, the range starts from 0 and ends just before the argument. thus, range(5) goes from 0 through 4.

with two arguments, the first argument is the starting point, while the second argument is one greater than the last number in the range. thus, range(5, 10) goes from 5 through 9.

with three arguments, the 3rd argument is a step value. thus, range(1, 10, 2) goes from 1 through 9 with a step of 2 between the numbers. on the other hand, range(0, -5, -1) goes from 0 through -4 with a step of -1. this results in a range that goes from the highest value to the lowest.

python optimizes ranges to save memory; it doesn't need the integers before a program asks for them. one way to get those numbers is to convert the range to a list or tuple. this is why we used the list and tuple functions to expand each range; either will suffice.

like strings, lists, and tuples, you can use indexing syntax to access specific numbers in a range object:

```
>>> my_range = range(5, 10)
>>> my_range[3]               # 8
```

mappings
mappings represent an unordered collection of objects stored as key-value pairs.

each key (usually a string) provides a unique identifier for a specific object in the mapping. the value is the object associated with that key.

the dictionary (class dict) is the most common mapping in python. it's the only kind of mapping we'll see in this book.

a dictionary (dict) is a collection of key-value pairs. a dict is similar to a list but uses keys instead of indexes to access specific elements. other languages use different names for similar structures: hashes, mapping, objects, and associative arrays are the most common terms. essentially, a python dictionary is a collection of key-value pairs.

you can create dictionaries using dict literals. they have zero or more key-value pairs separated by commas, all embedded within curly braces ({}). a key-value pair associates a key with a specific value. key-value pairs in dict literals use the key followed by a colon (:) and then the value.

that sounds more complicated than it is. let's put those words into action. type along!

our example creates a dict with three key-value pairs:

```
>>> my_dict = {
...     'dog': 'barks',
...     'cat': 'meows',
...     'pig': 'oinks',
... }
{'dog': 'barks', 'cat': 'meows', 'pig': 'oinks'}
```

this dict contains 3 keys: the strings 'dog', 'cat', and 'pig'. the 'dog' key is mapped to the string 'barks' while 'cat' is mapped to 'meows'

dictionary literals may also use a multi-line format, which is especially useful for larger or nested dictionaries:

copy code

```
my_dict = {
    'title': "monty python's flying circus",
    'cast': [
        'eric idle',
        'john cleese',
        'terry gilliam',
        'graham chapman',
        'michael palin',
        'terry jones',
    ],
    'first_season': 1969,
    'last_season': 1974,
    'reboot_season': none,
}
```

you can access objects in a dict with the [] key access syntax. the value between the brackets must be one of the keys in the dict:

```
>>> my_dict = {
...     'dog': 'barks',
...     'cat': 'meows',
...     'pig': 'oinks'
... }
...
>>> my_dict['cat']
'meows'

>>> my_dict['bird']
keyerror: 'bird'
```

you can use the same key access syntax on the left side of an assignment to

replace the associated key's value with a different object:

```
>>> my_dict['cat'] = 'purrs'
>>> my_dict
{'dog': 'barks', 'cat': 'purrs', 'pig': 'oinks'}
```

as of python 3.7, dictionary key/value pairs are stored in the same order they are inserted into a dictionary; before 3.7, the key/value pair order was unpredictable. this can be interpreted as meaning that dictionaries are ordered. however, even with this ordering, the key/value pair order can change. for instance:

```
copy code
dic = {}
dic['a'] = 1
dic['b'] = 2
print(dic)          # {'a': 1, 'b': 2}
del dic['a']
dic['a'] = 1
print(dic)          # {'b': 2, 'a': 1}
```

for this reason, we prefer to categorize dictionaries (and other mappings) as unordered collections. dictionaries, in particular, are unordered collections in which insertion order is preserved.

one last note: you can use almost any immutable object as a key in a dict; it doesn't have to be a string. the only significant requirement for keys is that they are hashable. we discuss hashing in our py110 course. for now, though, the important concept is that immutable types are almost always hashable, while mutable types are almost always non-hashable. thus, since lists are mutable and, therefore, unhashable, you can't use a list as a dictionary key. however, you can use a tuple as a dictionary key provided all of its elements are immutable and hashable.

sets

sets represent an unordered collection of unique objects; the objects are sometimes called the members of the set. sets are similar to mappings, except instead of using keys and values, a set is simply a collection of immutable (and hashable, as mentioned above) objects.

as the name suggests, sets support several operations corresponding to mathematical sets: unions, intersections, etc. we won't discuss those in this book.

the literal syntax for sets is a comma-separated list of object values between curly braces ({}). as a special case, empty sets must be created with the set constructor since {} by itself is an empty dictionary. let's see how it's done:

```
>>> d1 = {}         # empty dict
>>> print(type(d1))
<class 'dict'>

>>> s1 = set()      # empty set
>>> print(s1)
set()

# create a set from a literal
>>> s2 = {2, 3, 5, 7, 11, 13}
>>> print(s2)
{2, 3, 5, 7, 11, 13}

# create a set from a string
>>> s3 = set("hello there!")
{'t', 'o', 'e', 'l', ' ', 'h', '!', 'r'}
```

in the last example, notice that the set only contains one occurrence of each character, even though the string has repeated instances of some characters. set

members are always unique, even if you try to add duplicates. this example also shows that the order of the characters in the set has nothing to do with the order of the characters in the string.

set literals may also use a multi-line format, which is especially useful for larger sets:

copy code
```
monty_python_cast = {
    'eric idle',
    'john cleese',
    'terry gilliam',
    'graham chapman',
    'michael palin',
    'terry jones',
}
```
there are two main set types: ordinary sets (class set) and frozen sets (class frozenset). frozen sets are merely immutable sets. they also lack a literal syntax: you must use the frozenset function to create one.

```
>>> s5 = frozenset([1, 2, 3])
>>> print(s5)
frozenset({1, 2, 3})

>>> s6 = frozenset((1, 2, 3))
>>> print(s6)
frozenset({1, 2, 3})

>>> s7 = frozenset({1, 2, 3})
>>> print(s7)
frozenset({1, 2, 3})

>>> s8 = frozenset(range(1, 4))
>>> print(s8)
frozenset({1, 2, 3})
```
note that you can initialize a set or frozen set with any iterable type. in the most recent examples, we initialize a frozen set using a list, a tuple, a set, and a range. however, when printing frozen sets, they are always represented as using a set to initialize the frozen set.

one last note: you can use almost any immutable value as a set member. the only significant requirement is that the objects must be hashable.

basic operations
almost all python data types provide a variety of operations that you can perform on objects of that type. in the previous section, we learned about python's basic data types. however, we haven't done anything that shows how they are used. we'll do something about that in this chapter.

a remarkable feature of python is how much functionality the different types share. for instance:

you can compare almost any pair of objects with the same type for equality and, in many cases, for less than or greater than operations.
you can determine the length of any collection object or text sequence by calling the len() function on that object.
you can convert nearly any object into a human-readable string form.
you can iterate over different collection types without changing your syntax.
this feature brings considerable flexibility and ease of programming to the language.

in this chapter, we'll begin to distinguish between three main kinds of types:

the built-in types are part of python. they are available in every program if

you want them; just start using them. most types we'll encounter in this book are built-in types.
the standard types aren't automatically loaded when running python. instead, they are available from modules you can import into your programs. you don't have to download the standard types, but you do need to import them. we'll see some examples of importing modules in this chapter and learn more about importing modules later in the book.
the non-standard types come from either you and your colleagues or as downloadable modules available on the internet.
arithmetic operations
first up, let's turn our attention to the elementary arithmetic operations. all built-in and standard numeric data types work with all or most of these operations:

operator	operation
+	addition
-	subtraction
*	multiplication
/	division
//	integer division
%	modulo
**	exponentiation

these operators work with the built-in integer and float types and other types like the complex, decimal, and fractional number types. we'll focus on integers and floats here, but working with other data types is nearly identical.

let's look at some examples of the basic operations in action:

addition

copy code
```
print(38 + 4)      # 42
print(38.4 + 41.9) # 80.3

# mixing integers & floats
print(38 + 41.5)   # 79.5
```
subtraction

copy code
```
print(38 - 4)      # 34
print(38.4 - 41.9) # -3.5

# mixing integers & floats
print(38 - 41.5)   # -3.5
```
multiplication

copy code
```
print(38 * 4)      # 152
print(38.4 * 41.1) # 1578.24

# mixing integers & floats
print(38 * 41.5)   # 1577.0
```
division with /

copy code
```
print(16 / 4)      # 4.0
print(16 / 2.5)    # 6.4
```
when you divide two integers, two floats, or one of each with /, the result is always a float, even if both operands are integers.

integer division with //

copy code
```
print(16 // 3)     # 5
```

```
print(16 // -3)    # -6
print(16 // 2.3)   # 6.0
print(-16 // 2.3)  # -7.0
```
the // operator returns the largest whole number less than or equal to the floating point result. that is, it rounds the result down to the nearest whole number. thus, 16 // 3 returns 5, not 5.333333333333333. likewise, 16 // -3 returns -6. if either operand is a float, the return value is also a float, but it's still rounded down to a whole number.

note: the // operator doesn't work with the built-in complex numbers.

exponentiation (powers)

the expression a**b tells python to raise a to the power of b. thus, something like 3**4 means 3 to the 4th power, or 3 * 3 * 3 * 3 or 81.

copy code
```
print(16**3)     # 4096
```
modulo

the % operator is called the modulo operator. it is sometimes called the modulus operator or the remainder operator, both of which are technically incorrect. (there are some subtle differences between modulo and remainder operations). this confusion is understandable since the % operator is usually used to calculate the remainder of dividing two integers:

copy code
```
print(15 % 3)   # 0
print(16 % 3)   # 1
print(17 % 3)   # 2
print(18 % 3)   # 0
```
we won't get into the differences between modulo, modulus, and remainder in this book. what's important right now is that % is easiest to understand when the operands are both positive integers.

when both a and b are positive integers, a % b returns the non-negative remainder of dividing a by b. of special interest is that a % b returns 0 if, and only if, a is exactly divisible by b. if either a or b is negative, things get weird, as can be seen in the following table:

a	b	a % b	remainder
14	7	0	0
17	7	3	3
17	-7	-4	3
-17	7	4	-3
-17	-7	-3	-3

the final column in the above table shows what a remainder operator would return if python had one. you can compute the remainder in python using the math module's remainder function:

copy code
```
from math import remainder

print(int(remainder(14, 7)))    # 0
print(int(remainder(17, 7)))    # 3
print(int(remainder(17, -7)))   # 3
print(int(remainder(-17, 7)))   # -3
print(int(remainder(-17, -7)))  # -3
```
in general, you don't need to know the difference between modulo and remainder; just be aware that they are different. you also don't need to worry about how they work with negative numbers if you can avoid using negative numbers. since most use cases of % only care about divisibility, that will happen naturally. if a % b == 0, then a is evenly divisible by b, no matter the signs of a and b.

note: the % operator doesn't work with the built-in complex numbers.

floating point imprecision
floating point numbers have some precision issues you must be aware of. for instance, if you add 0.1 and 0.2 and compare the result for equality with 0.3, you'll learn that they are not equal:

copy code
```
print(0.1 + 0.2 == 0.3)       # false
```
this can be a serious problem in some applications, such as finance. it's an artifact of how real numbers are stored on most computers; you'll encounter this problem in almost every language. one way around the problem in python is to use the math.isclose function:

copy code
```
import math
math.isclose(0.1 + 0.2, 0.3)  # true
```
you can also use the decimal.decimal type to make precise computations:

copy code
```
from decimal import decimal
decimal('0.1') + decimal('0.2') == decimal('0.3')
# true
```
note that you should always use strings with decimal.decimal. you can use float values. however, you will lose the benefit of precise computation if you do.

equality comparison
you sometimes want to determine whether two values are identical or different. the == and != operators can help with that. == compares two operands for equality and returns true or false as appropriate. in contrast, != returns true if they are not equal, false otherwise.

let's try some comparisons in python:

equality (==)copy code
```
print(42 == 42)       # true
print(42 == 43)       # false
print('foo' == 'foo') # true (works with strings)
print('foo' == 'foo') # false (case matters)
```
inequality (!=)copy code
```
print(42 != 42)       # false
print(42 != 43)       # true
print('foo' != 'foo') # false (works with strings)
print('foo' != 'foo') # true (case matters)
```
the operators == and != work with almost all data types. assuming that the values represented by a and b below are instances of the built-in data types, then:

if a and b have different data types, a == b usually returns false while a != b returns true. however, numbers are an exception: all built-in and standard number types can be compared for equality without regard to their specific types. thus, 1 == 1.0 is true.

if a and b have the same data type, a == b almost always returns true if the two objects have the same value, while a != b returns false.

when working with standard and non-standard types, all bets are off. while most non-builtin types obey the same rules, not all do. the only way to be sure is to study the documentation, look at the source code, or try to determine the behavior from tests.

ordered comparisons
as you might expect, python supports several other comparison operations: you can check for less than (<), less than or equal to (<=), greater than (>), and

greater than or equal to (>=):

less than (<) and less than or equal to (<=)copy code

```
print(42 < 41)          # false
print(42 < 42)          # false
print(42 <= 42)         # true
print(42 < 43)          # true

print('abcdf' < 'abcef') # true
print('abc' < 'abcdef')  # true
print('abcdef' < 'abc')  # false
print('abc' < 'abc')     # false
print('abc' <= 'abc')    # true
print('abd' < 'abcdef')  # false
print('a' < 'a')         # true
print('z' < 'a')         # true

print('3' < '24')        # false
print('24' < '3')        # true
```

note the comparisons involving strings. strings are compared lexicographically, which means they are compared character-by-character from left-to-right.

for instance, on line 6, we first compare the first character of each string ('a'). since they are the same, we move on to the next pair of characters ('b'), then the pair after that ('c'). everything is equal up to this point.

however, when we compare 'd' against 'e', we finally see a difference between the two strings. since 'd' < 'e', the final result is true. python doesn't bother looking at the remaining character in these strings ('f').

this lexicographic comparison also explains why '3' < '24' on line 15 is false and '24' < '3' on line 16 is true. only the first character of each string gets compared.

it's also worth looking at 'abc' < 'abcdef' on line 7. while the first 3 characters of both strings are the same, the first string is shorter than the second. when python compares two strings that are equal up to the length of the shorter string, then the shorter string is deemed to be less than the longer string.

when comparing strings, python stops as soon as it makes a decision. for instance, in 'abd' < 'abcdef', python only needs to check the first 3 characters in both strings. when it reaches the 3rd character, it can see that 'abd' is not less than 'abcdef'.

finally, lines 12 and 13 help demonstrate that uppercase alphabetic letters are considered to be less than lowercase alphabetic letters. every uppercase letter is less than every lowercase letter. this behavior often confuses new programmers, but almost all languages work this way.

greater than (>) and greater than or equal to (>=)copy code

```
print(42 > 41)          # true
print(42 > 42)          # false
print(42 >= 42)         # true
print(42 > 43)          # false

print('abcdf' > 'abcef') # false
print('abc' > 'abcdef')  # false
print('abcdef' > 'abc')  # true
print('abc' > 'abc')     # false
print('abc' >= 'abc')    # true
print('abcdef' > 'abd')  # false
print('a' > 'a')         # false
print('z' > 'a')         # false
```

```
print('3' > '24')        # true
print('24' > '3')        # false
```
let's look more closely at 'abcdef' > 'abc'. in this example, the strings have unequal sizes. furthermore, the longer string is identical up to the shorter string's length. python returns true here; when it can no longer take characters from the shorter string, it concludes that the longer string has the greater value. similar behaviors occur with the other ordered comparison operators.

it's also worth noting that even numeric strings are compared character by character. thus, '3' > '24' returns true since the character 3 is greater than the character 2.

in general, numeric characters in a string are less than alphabetic characters, and uppercase letter characters are less than lowercase letters. other characters appear at different points. for instance, '#' < '5' < ';' < 'a' < '^' < 'a'. (you don't have to memorize this.)

if the precise ordering of character values becomes sufficiently significant to matter, look them up in a standard ascii table.

as with == and !=, many other types besides numbers and strings work with the ordered comparison operators. for instance, you can compare sets with these operators to determine if set a is a subset or superset of set b. you can also compare lists and tuples: like string comparisons, list and tuple comparison goes element by element to determine which object is less than or greater than the other:

setscopy code
```
print({3, 1, 2} < {2, 4, 3, 1})         # true
print({3, 1, 2} > {2, 4, 3, 1})         # false
print({2, 4, 3, 1} > {3, 1, 2})         # true
```
lists (tuples work identically)copy code
```
print([1, 2, 3] < [1, 2, 3, 4])         # true
print([1, 4, 3] < [1, 3, 3])            # false
print([1, 3, 3] < [1, 4, 3])            # true
```

string concatenation
string concatenation looks like numeric addition and multiplication, but the result is entirely different. it uses the + and * operators to join strings.

back to python:

```
>>> 'foo' + 'bar'
'foobar'
```
that looks simple enough.

there is at least one surprise you may encounter. what will the following code return? try answering before you try it.

copy code
```
'1' + '2'
```
if you thought it would return 3, that makes sense. however, since '1' and '2' are both strings, python performs concatenation instead. that's why the result is '12'.

you can also use the * operator to perform repetitive concatenation. for instance, if you want the string 'abcabcabc', you can use * to generate it:

copy code
```
print('abc' * 3)             # 'abcabcabc'
print(3 * 'abc')             # 'abcabcabc'
```
coercion

suppose you have two string values in your program, '1' and '2', that you want to add mathematically and get 3. you can't do that directly since + performs concatenation when its operands are both strings. somehow, we need to coerce the strings '1' and '2' to the numbers 1 and 2: we want to perform a type coercion.

strings to numbers
the int and float functions coerce a string to a number. int coerces a string to an integer, while float coerces a string to a float.

copy code
```
print(int('5'))              # 5
print(float('3.141592'))     # 3.141592
```
these functions take a string or another number and attempt to convert it to an integer or a float. you can then perform arithmetic operations on the result.

numbers to strings
you can also coerce numbers into strings. the str function provides this capability. let's see an example:

copy code
```
print(str(5))                # '5'
print(str(3.141592))         # '3.141592'
```
in reality, str can convert most python values to a valid string. we'll see more situations where you can use str later.

using functions like str, int, etc., is required for many conversions and a good idea in others where clarity is needed.

python will also automatically convert an object from one type to another in the context of certain operations. for instance, when you use print() to print an object -- any object -- print will coerce it to a string before printing it. that saves considerable typing:

copy code
```
# (unnecessary) coercion with `str`
print(str(3))           # 3
print(str(false))       # false
print(str([1, 2, 3]))   # [1, 2, 3]
print(str({4, 5, 6}))   # {4, 5, 6}

# implicit coercion
print(3)                # 3
print(false)            # false
print([1, 2, 3])        # [1, 2, 3]
print({4, 5, 6})        # {4, 5, 6}
```
type coercion also occurs when mixing numbers of different types in an expression:

type a	type b	result type
int	float	float
int	decimal	decimal
int	fraction	fraction
float	decimal	--error--
float	fraction	float
decimal	fraction	--error--

these results aren't surprising. mixing different number types is handy, and most people expect it.

gotchas sometimes arise with type coercion. one such gotcha occurs when you use a boolean in an arithmetic expression. python coerces true to the integer value 1 and false to 0. the resulting integer may be coerced further based on the types of the other subexpressions.

copy code

```
print(true + true + true)     # 3
print(true + 1 + 1.0)         # 3.0
print(false * 5000)           # 0
```
if there's any chance that one of your variables contains a boolean value, be careful. while python won't mind, the result may not be what you expect. even if you expect it, will the next person who looks at your code?

one last example of coercion is truthiness coercion. python can use any value, regardless of type, in a conditional expression in an if or while statement. we'll discuss this in the truthiness topic of the flow control chapter.

determining types
as we've seen, python data types give the programmer many choices. however, when things go wrong and you need to debug your code, you may have to determine what classes you're working with. knowing how to view type information in the repl is worthwhile even if you aren't debugging code. you can even make programmatic decisions based on type. however, that's usually a sign of a problem with your design.

with that said, let's see some of the ways we can determine types. first up is the type function, which can be called with any object:

copy code
```
print(type(1))         # <class 'int'>
print(type(3.14))      # <class 'float'>
print(type(true))      # <class 'bool'>
print(type('abc'))     # <class 'str'>
print(type([1, 2, 3])) # <class 'list'>
print(type(none))      # <class 'nonetype'>

foo = 42               # variables work, too
print(type(foo))       # <class 'int'>
```
note that the return value usually includes more information than you need. if you just want the class name, you can access the __name__ property from the result:

copy code
```
print(type('abc').__name__)   # str
print(type(false).__name__)   # bool
print(type([]).__name__)      # list
```
finally, you can use type with the is operator.

copy code
```
print(type('abc') is str)     # true
print(type('abc') is int)     # false
print(type(false) is bool)    # true
print(type([]) is list)       # true
print(type([]) is set)        # false
```
note that all 3 of the above approaches discount the effects of inheritance. since we don't discuss inheritance in this book, that doesn't matter for now. however, you may want to consider using the isinstance function, which determines whether an object is an instance of a particular type. it takes inheritance into account. we'll return to this in our object oriented programming book.

copy code
```
print(isinstance('abc', str))    # true
print(isinstance([], set))       # false

class a:
    pass

class b(a):
    pass
```

```
b = b()

print(type(b).__name__) # b
print(type(b) is b)     # true
print(type(b) is a)     # false (b's type is
                        # not a)
print(isinstance(b, b)) # true
print(isinstance(b, a)) # true (b is instance of a and b)
```

string representations

you can use the str and repr functions with any object. they each return a string representation of an object. the str function returns a string intended to be something that humans can read. it is often used when printing an object. python calls str() under the hood when it needs to print or interpolate a value. the repr function is a bit lower-level. it returns a string that you can, in theory, use to create a new instance of the object.

for most built-in types, str and repr return the same value. however, that is not always the case. strings are one of the more obvious types with different str and repr return values:

copy code

```
my_str = 'abc'
print(my_str)       # abc
print(str(my_str))  # abc (same as print(my_str))
print(repr(my_str)) # 'abc' (note the quotes)
```

collection and string lengths

all built-in collection types (strings, sequences, mappings, and sets) have lengths. the length of a string is the number of characters in the string, while the length of other collections is the number of elements in the collection. you can easily determine the length of a collection using the len function:

copy code

```
print(len('launch school'))       # 13 (string)
print(len(range(5, 15)))          # 10 (range)
print(len(range(5, 15, 3)))       # 4 (range)
print(len(['a', 'b', 'c']))       # 3 (list)
print(len(('d', 'e', 'f', 'g')))  # 4 (tuple)
print(len({'foo': 42, 'bar': 7})) # 2 (dict)
print(len({'foo', 'bar', 'qux'})) # 3 (set)
```

indexing and key access

strings, ranges, lists, and tuples all support indexed access to individual elements in the collection. indices begin at 0 and run through 1 less than the length of the string or collection. any index used must be in this range, or you will get an indexerror:

indices may also be negative in the range -1 to -len(seq). we'll discuss this later.

copy code

```
my_str = "abc"                # string
print(my_str[0])              # 'a'
print(my_str[1])              # 'b'
print(my_str[2])              # 'c'
print(my_str[3])
# indexerror: string index out of range
```

copy code

```
my_range = range(5, 8)         # range
print(my_range[0])             # 5
print(my_range[1])             # 6
print(my_range[2])             # 7
print(my_range[3])
# indexerror: range object index out of range
```

```
copy code
my_list = [4, 5, 6]           # list
print(my_list[0])             # 4
print(my_list[1])             # 5
print(my_list[2])             # 6
print(my_list[3])
# indexerror: list index out of range
copy code
tup = (8, 9, 10)              # tuple
print(tup[0])                 # 8
print(tup[1])                 # 9
print(tup[2])                 # 10
print(tup[3])
# indexerror: tuple index out of range
```

dictionaries use keys that work similarly to indexes. however, it is incorrect to describe dictionary keys as indexes: they are keys. as with indexes, using a key that does not exist in the dictionary produces an error:

```
copy code
my_dict = {'a': 1, 'b': 2, 'c': 3}
print(my_dict['a'])           # 1
print(my_dict['b'])           # 2
print(my_dict['c'])           # 3
print(my_dict['d'])           # keyerror: 'd'
```

using [] to update elements

since they are mutable, lists and dictionaries let you use the [] operator to replace collection elements. as you might expect, lists use indexes to update elements, while dictionaries use keys. you cannot use [] to create new list elements, but you can do so with dictionaries.

note that strings, ranges, tuples, and frozen sets are immutable, so they do not support using [] for updates. sets are mutable, but they don't support indexing.

```
copy code
my_list = [1, 2, 3, 4]
my_list[2] = 6
print(my_list)          # [1, 2, 6, 4]
my_list[4] = 10
# indexerror: list assignment index out of range
```

expressions and statements

the distinction between statements and expressions in python is fundamental and affects how you write and structure your code. let's define them now.

an expression combines values, variables, operators, and calls to functions to produce a new object. expressions must be evaluated to determine the expression's value. examples of expressions include:

literals: 5, 'karl', 3.141592, true, none
variable references: foo or name when these variables have been previously defined.
arithmetic operations: x + y or a * b - 5.
comparison operations: 'x' == 'x' or 'x' < 'y'.
string operations: 'x' + 'y' or 'x' * 32.
function calls: print('hello') or len('python').
any valid combination of the above that evaluates to a single object.
you can think of an expression as something that produces a value that can be assigned to a variable, passed to a function or method, or returned by a function or a method.

a statement, on the other hand, is an instruction that tells python to perform an action of some kind. unlike expressions, statements don't return values. they do something but don't produce a value as expressions do.

examples of statements include:

assignment: like x = 5. this doesn't evaluate as a value; it assigns a value to a variable.
control flow: such as if, else, while, for, and so on. these determine the flow of your program but don't evaluate as a value themselves.
function and class definitions: using def or class.
return statements: like return x, which tells a function to exit and return a value. return itself doesn't return a value; it informs the function what value it should return.
import statements: such as import math.
there are some key differences to keep in mind:

expressions always return a value; statements do not.
expressions are often part of statements. for example, in the statement y = x + 5, x + 5 is an expression.
statements often represent bigger chunks of functionality like loops or conditionals; expressions deal with determining values.
some expressions are stand-alone. they aren't part of an ordinary statement, but their return values are ignored. these stand-alone expressions fall into a gray area; the return values are ignored, making them seem more like statements. in this book and elsewhere at launch school, we consider these stand-alone expressions as both expressions and statements.

some stand-alone expressionscopy code
```
3 + 4            # simple expression
print('hello')   # function call; returns none
my_list.sort()   # method call; returns none
```

expression evaluation
by default, python evaluates most expressions from left to right. this is fine if all of the operators in an expression are the same operator:

```
>>> 1 + 2 + 3 + 4 + 5
15
```
here, python first evaluates 1 + 2, which is 3. it then adds 3 to the first 3, which yields 6. next, we add 4 to get 10 and, finally, we add 5 to get 15.

however, what happens if the operators are mixed?

```
>>> 4 * 5 - 1 + 2 * 3
??
```
should that expression be 63? is the result possibly 72? 25? -8? maybe it's something else altogether? no, this isn't one of those silly math puzzles you see on social media.

python evaluates that expression as 25: the multiplications occur first (4 * 5 is 20 and 2 * 3 is 6), followed by the additions and subtractions from left to right (20 - 1 + 6 is 25). python dictates this order of evaluation through its precedence rules, which we discuss in more detail in the core curriculum.

you shouldn't rely on the precedence rules even if you've memorized them. use parentheses to tell python explicitly how you want to evaluate the expression:

copy code
```
print(((4 * 5) - 1) + (2 * 3))   # 25
print((4 * ((5 - 1) + 2)) * 3)   # 72
print(((((4 * 5) - 1) + 2) * 3)) # 63
print(4 * (5 - (1 + (2 * 3))))   # -8
```
parenthesized sub-expressions are usually evaluated before any non-parenthesized sub-expressions. so, on line 1 above, python evaluates (4 * 5) and (2 * 3) before it performs any addition or subtraction operations.

nested parenthesized sub-expressions are evaluated before the parenthesized

expressions they are contained in. thus, on line 1 above, (4 * 5) and (2 * 3) get evaluated first. next, 1 gets subtracted from the result of (4 * 5), then the result of (2 * 3) gets added to that value.

in general, you should not depend on the precedence rules in python. no matter how well you know them, you will eventually forget one of the trickier cases, and readers of your code may not understand the code. you should always use parentheses to inform python and readers of your code of your intent.

output vs. return values
inexperienced programmers are often confused with the difference between returning or outputting a value. when we invoke the print function, we're telling python to write some output to the display. in python, that is usually your screen. the term log is often used as a synonym for printing or showing something on the display. sometimes, we even use log, the noun, as a synonym for your terminal screen. for example, in the following code, the print function simply prints the string 'abc'. it doesn't return a useful value; it's sole purpose is to print something.

copy code
print('abc')
in many cases, a function or expression doesn't print anything, but simply returns a value that gets assigned to a variable, evaluated as a condition, or passed to another function. for example, the following code first calls the range function, which returns a range object. that value is subsequently passed to the list function which, in turn, returns a list object.

copy code
list(range(3))
in the python repl, the return value can also be displayed on the screen. however, take care not to confuse these return values with actual output. it's simply the repl's way of showing you that an expression returned a value. if you run the code from a file, you won't see these return values.

variables
a fundamental tenet of programming is that programs must store information in memory to use and manipulate it. variables are the means for doing that in almost all computer languages. they provide a way to label data with a descriptive name that helps us and other readers understand a program. consider variables as labels that identify particular objects your program creates and uses.

we'll use a few "functions" in this chapter to ensure the code works as intended. don't worry too much about how functions work. all you need to know for this chapter is:

the def statement creates a function.
you "call" a function by appending () to the function name.
a function can be called multiple times; each call executes the code in the function body.
we'll learn more about functions later. for now, please don't put too much effort into understanding them.

variables and variable names
a variable is a label attached to a specific value stored in a program's memory. typically, variables can be changed; we can connect the variable to a different value somewhere else in memory. for brevity, we often talk of assigning or reassigning an object to a variable. we also speak of variables as containing data, though that's not entirely accurate, as we'll learn later in this book.

consider this code:

copy code
answer = 41

```
print(answer)       # 41

answer = 42
print(answer)       # 42
```

when python sees line 1 of this code, it sets aside a bit of memory and stores the value 41 in that area. it also creates a variable named answer that we can use to access that value.

on line 4, we reassign the value 42 to the variable named answer. that is, python makes answer refer to a new object. in particular, you must understand that we're not changing the object that represents 41 -- we're assigning an entirely new value to the answer variable.

variable naming

there are two hard problems in computer science: cache invalidation, naming things, and off-by-one errors.

-- author unknown

properly naming variables is traditionally viewed as one of the most challenging tasks in computer programming. if you're new to programming, this might seem odd. after all, how tough can it be to choose a name? as it turns out, it is challenging. consider how much effort many new parents require to select a name for their baby.

a variable name must accurately and concisely describe the variable's data. in large programs, it's challenging to remember what each variable contains. if you use non-descriptive names like x, i, and dct, you may forget what data that variable represents. other readers -- programmers -- must suss out the meaning for themselves. therefore, when naming variables, think hard about the names. ensure they are accurate, descriptive, and understandable to other readers. that might be you when you revisit the program a few months or years later.

variable names are often referred to by the broader term identifiers. in python, identifiers refer to several things:

variable and constant names
function and method names
function and method parameter names
class and module names

you'll meet all these things as you move through the launch school core curriculum. we'll often use the term identifier when discussing names in general: variables, constants, functions, classes, etc. we'll use one of the more specific terms when we need to limit the scope of a discussion.

naming conventions

closely related to the stylistic conventions discussed in the using python chapter are the python naming conventions. names that follow these conventions are idiomatic. in contrast, those that do not are non-idiomatic (valid but not idiomatic) or illegal (your code will either raise a syntax error or do something unexpected).

naming conventions for most identifiers (excluding constant and class names):

use snake_case formatting for these names.
names may contain lowercase letters (a-z), digits (0-9), and underscores (_).
names should begin with a letter.
if the name has multiple words, separate the words with a single underscore (_).
names that begin or end with one or two underscores have special meaning under the naming conventions. don't use them until you understand how they are used.
names may only use letters and digits from the standard ascii character set.

idiomatic names

foo
answer_to_ultimate_question

```
eighty_seven
index_2
index2
```
constant names (unchanging named values):

use screaming_snake_case formatting for these names.
names may contain uppercase letters (a-z), digits (0-9), and underscores (_).
names should begin with a letter.
if the name has multiple words, separate the words with a single underscore (_).
names that begin or end with one or two underscores have special meaning under the naming conventions. don't use them until you understand how they are used.
names may only use letters and digits from the standard ascii character set.
idiomatic names
```
foo
answer_to_ultimate_question
eighty_seven
index_2
index2
```
the constant naming convention is advisory only. python has no way to ensure that constants are never changed. whether a constant ever changes is entirely up to the programmer, not python.

class names:

use pascalcase formatting for these names. pascalcase is sometimes called camelcase (with both cs capitalized).
names may contain uppercase and lowercase letters (a-z, a-z) and digits (0-9).
names should begin with an uppercase letter.
if the name has multiple words, capitalize each word.
idiomatic names
```
foo
ultimatequestion
fourleggedpets
pythonversion2rules
```
we'll meet a few class names in this book but will leave a full discussion for our object oriented programming in python book.

note that many non-idiomatic names are still legal (valid) python identifiers. sometimes, it makes sense to break the rules. however, not all identifiers are legal; here are some things you must do:

you can use letters, digits, and underscores in python identifiers. extended ascii and unicode letters and digits are allowed.
you may not use punctuation characters, most special characters, or whitespace.
you may not start identifiers with a digit.
you may not use python's reserved words such as if, def, while, return, and pass as names.
```
non-idiomatic names     explanation
fourwayintersection     camelcase
schÃ¶n      extended ascii
illegal names     explanation
pass  reserved word
3xy   starts with digit
ultimate-question hyphen
one two three     whitespace
is_lowercase?     punctuation
is+lowercase      special character
```
most illegal names will raise an error. however, if the illegal name looks like valid python, you won't get an error:

copy code
```
first,last = ['mary', 'wonder']
```
that example creates two variables, first and last, and assigns them to mary and wonder, respectively.

creating and reassigning variables
we create (initialize) most variables by simply giving them a value. that happens as part of an assignment statement:

copy code
```
forename = 'clare'       # initialization (also called assignment)
```
initializations are also said to provide a definition for the variable.

no special keywords are required to define variables. we can also give new values to variables by simply reassigning them:

copy code
```
forename = 'clare'       # initialization (also called assignment)
# omitted code
forename = 'victor'      # reassignment
```
an assignment like foo = bar can be described in two main ways:

the variable foo is assigned the value of bar.
the value of bar is assigned to the variable foo.
if bar is a literal value, you can skip mentioning "value of".

both phrases mean the same thing, but phrase 1 is usually preferred. however, phrase 2 is perfectly acceptable. think of it like this:

kim has been assigned the task of building safety coordinator.
the task of building safety coordinator has been assigned to kim.
both statements mean the same thing, and so it is with variables and values.

note that the same phrasing can be used when talking about reassignment: a variable can be reassigned to a new value, or a value can be reassigned to a variable.

at launch school, we mainly use phrase 1. however, don't be confused if we sometimes use phrase 2. sometimes, phrase 2 just reads better.

how initialization and reassignment work
when you initialize a variable, python gives it an initial value and sticks that value somewhere in the computer's memory. it also allocates a small amount of memory for the variable itself, then places the value's memory address into the variable's spot in memory.

for instance, consider this code:

copy code
```
foo = 'abcdefghi'
```
when python encounters this code, it will create the string 'abcdefghi' somewhere in memory. let's assume python stores the string at memory address 73420. next, it creates a variable somewhere else in memory. let's suppose the variable is at address 10230. it then associates address 10230 with the name foo. python stores the address of the string (73420) at address 10230. thus, we get a situation that looks like this:

consider what happens when we later run this reassignment:

copy code
```
foo = 'hello'
```
python creates the new string 'hello' somewhere in memory. we can assume that 'hello' is stored at memory address 87160. since we already have a foo variable, python simply replaces the value at address 10230 with 87160. this breaks the connection with the original string and establishes a new one with the new string.

creating constants

constants are created (initialized) in the same way as variables: by giving them a value. that happens as part of an assignment statement:

copy code
```
pining_for = 'fjords'         # initialization
```
as with variable creation, no keywords are required to create constants. however, you should use screaming_snake_case in observation of the constant naming convention.

since constants should be unchanging, you should never reassign a constant.

constants are usually defined in the global scope. the definitions are usually written at the top of the program file, just below any imports. you can use constants anywhere, but they should be defined globally. thus, you can use constants in functions, but you shouldn't create a constant inside a function.

constants aren't constant in python. python does not support true constants. instead, the screaming_snake_case naming convention is solely for programmers. for instance, the name first_base meets the naming conventions for constants, so you should think of first_base as a constant. however, since python places no constraints on such names, you can easily change the value assigned to first_base. changing a constant's value is poor practice and may make you unpopular at work.

expressions and assignment
assignment and reassignment are often more complex than simply assigning a literal value to a variable. assignment and reassignment often use expressions on the right side of the = to determine the desired value. for instance:

copy code
```
left_side = 5
right_side = 32
total = left_side + right_side  # total = 37
print(total)                    # prints 37
```
this code starts with two simple assignments that initialize the left_side and right_side variables to 5 and 32, respectively. next, we add the values of left_side and right_side together and assign the result to total, which we then print.

here's an example that uses a function call to compute the resulting value for the assignment:

copy code
```
def square(number):
    return number * number

forty_two_squared = square(42)
print(forty_two_squared)                # 1764
```
the expression on the right side of the = operator can be any valid expression. it can be as simple or complex as needed, though you should strive to keep them readable and easy to understand.

the variable to the left of the = operator is always the target variable for the resulting value. that is, the expression's value will be assigned to the variable.

it's worth noting that the right side of an assignment is always completely evaluated before assigning the result to the variable. that means you can write code like this:

copy code
```
foo = 42            # foo is 42
foo = foo - 2       # foo is now 40
foo = foo * 3       # foo is now 120
```

```
foo = foo + 5       # foo is now 125
foo = foo // 25     # foo is now 5
foo = foo / 2       # foo is now 2.5
foo = foo**3        # foo is now 15.625
print(foo)          # prints 15.625
```
in lines 2-7, the right side of each assignment is computed first using whatever value foo had most recently. thus, foo was 42 on line 2, 40 on line 3, and so on. on each line, a computation is performed using the current value of foo. python then reassigns the newly computed value to foo.

you can also use expressions to initialize constants. however, this is less common than evaluating expressions for a variable.

augmented assignment
the assignments at the end of the previous section all follow a similar pattern: they take the current value of a variable, perform an arithmetic operation on the variable's value, and then reassign the variable to the newly computed value. this sort of operation is so common that most languages have a shorthand notation called augmented assignment or assignment operators:

copy code
```
foo = 42            # foo is 42
foo -= 2            # foo is now 40
foo *= 3            # foo is now 120
foo += 5            # foo is now 125
foo //= 25          # foo is now 5
foo /= 2            # foo is now 2.5
foo **= 3           # foo is now 15.625
print(foo)          # prints 15.625
```
each statement is noticeably shorter than the original, but the results are identical. while there's little or no performance benefit to augmented assignment, most developers find the syntax more readable, and you're less likely to misspell one of the variable names.

augmented assignment also works with string concatenation and repetition. in fact, it works with any type that supports the various operators.

augmented assignment with stringscopy code
```
bar = 'xyz'          # bar is 'xyz'
bar += 'abc'         # bar is now 'xyzabc'
bar *= 2             # bar is now 'xyzabcxyzabc'
print(bar)           # prints xyzabcxyzabc
```
augmented assignment with listscopy code
```
bar = [1, 2, 3]     # bar is [1, 2, 3]
bar += [4, 5]       # + with lists appends
                    # bar is now [1, 2, 3, 4, 5]
print(bar)          # prints [1, 2, 3, 4, 5]
```
augmented assignment with setscopy code
```
bar = {1, 2, 3}     # bar is {1, 2, 3}
bar |= {2, 3, 4, 5} # | performs set union
                    # bar is now {1, 2, 3, 4, 5}
bar -= {2, 4}       # - performs set difference
                    # bar is now {1, 3, 5}
print(bar)          # prints {1, 3, 5}
```
augmented assignment also works with constants. however, as with constant reassignment, it is poor practice.

note that augmented assignment is a statement, not an expression. thus, you can't use augmented assignment as a function argument or return value:

copy code
```
def foo(bar):
    print(bar)
```

```
a = 3
foo(a *= 2)
#     ^^
# syntaxerror: invalid syntax
copy code
def foo():
    a = 3
    return a *= 2
#           ^^
# syntaxerror: invalid syntax
```
if you study the above examples, you may get the impression that a += b is equivalent to a = a + b. if the left-side variable (e.g., a) is immutable, that's true. however, if a is mutable, the expression may not be equivalent to a = a + b. for instance, if a and b are both lists, then a += b is actually equivalent to a.extend(b). the resulting values are the same, but we'll see later why this behavior is fundamentally different from a = a + b.

reassignment vs. mutation

there are two ways to change things in python and most other programming languages. you can change the binding of the variable by making it reference a new object, or you can change the value of the object assigned (bound) to the variable. the former is known as reassignment while the latter is known as mutation.

a variable is a name that refers to an object at a specific place in memory. reassignment makes that name refer to a different object somewhere else in memory. mutation, on the other hand, does not change which object the variable refers to. instead, it changes the object itself. after mutating an object assigned to a specific variable, the variable continues to refer to the same object (albeit altered) at the same memory location.

this distinction is crucial in any language that permits reassignment and mutation, as it significantly impacts how those languages work. one subtle but important distinction is the difference between reassigning a variable and reassigning an element in a list, dict, or other mutable collection. reassigning an element of a mutable collection doesn't reassign the variable; it mutates the collection.

here are some examples illustrating the differences between reassignment and mutation. pay close attention, as this is a crucial concept.

```
copy code
num = 3               # assignment (initialization)
my_list = [1, 2, 3]   # assignment (initialization)
my_dict = {           # assignment (initialization)
    'a': 1,
    'b': 2,
}

num = 42              # reassignment
my_list[1] = 42       # reassignment of element,
                      # my_list is mutated!
my_dict['b'] = 3      # reassignment of dict pair
                      # my_dict is mutated!

# you can still reassign the variables
my_list = [2, 3, 4]   # reassignment
my_dict = { 'x': 0 }  # reassignment
```

things get a little fuzzy when using augmented assignment. as mentioned earlier, a += b is equivalent to a = a + b if a is immutable. in that case, then augmented assignment is a form of reassignment.

however, if a is mutable, it may be (and frequently is) mutated in augmented

assignments. as we mentioned earlier, a += b is equivalent to a.extend(b) if a and b are lists. thus, a += b is not reassignment; it is a mutating operation. this distinction becomes crucial when we discuss variables as pointers.

as much as possible, we use the terms mutation or mutate to describe any mutating operation. we'll talk more about these concepts throughout this book and in almost every course at launch school. for now, keep the distinction between reassignment and mutation in mind.

input/output
computers and software don't live in isolation. they are tools that solve real-world problems, meaning they must interact with the real world. a computer needs to take input from some source, perform some actions on that input, and then provide output.

computer programs can use many input sources, such as mice, keyboards, disks, the network, and environmental sensors. similarly, a program can write output to multiple places, such as the screen, a log, or a network.

a computer can obtain input from another computer's output. for example, when you enter your login credentials for a website, your computer first encrypts them. it then sends them to the website's authentication server, which accepts (inputs) the data and attempts to verify your identity. in this fashion, two computers can perform different parts of a common task.

in this chapter, we'll discuss the most basic techniques a program can use to interact with the outside world: reading keyboard input and displaying output in a terminal.

terminal output
you've already seen one way to display output in the terminal: the print function. this built-in function takes any python value, regardless of type, and prints it. let's see an example. create a file named greetings.py with the following content:

greetings.pycopy code
```
name = 'jane'
print(f'good morning, {name}!')
```
in this simple example, we initialize a name variable with the string 'jane', then use it in an f-string interpolation to build and display a greeting message. run the program to see it in action:

copy code
```
$ python greetings.py
good morning, jane!
```
the print() function works with all data types, often -- but not always -- formatting the output in some manner that makes it somewhat understandable to humans. for instance:

```
>>> print({ 'a': 1, "b": 42, 'c': "string",
...         'd': [5, 6], 'e': { 8, 9, 10 }})
# pretty printed for clarity
{
    'a': 1,
    'b': 42,
    'c': 'string',
    'd': [5, 6],
    'e': {8, 9, 10}
}

>>> import time
>>> print(time.asctime())
mon aug 21 22:59:29 2023
```
you can print multiple objects by just listing them one after the other as

arguments to print():

```
>>> print(1, 2, 3, 'a', 'b')
1 2 3 a b

>>> print([1, 2, 3], 4, false, { 5, 6, 7, 8})
[1, 2, 3] 4 false {8, 5, 6, 7}
```

by default, multiple items are separated by spaces. you can use a different separator with the sep keyword argument (we discuss keyword arguments in the core curriculum):

copy code

```
print(1, 2, 3, 'a', 'b', sep=',')      # 1,2,3,a,b
print('a', 'b', 'c', 'd', 'e', sep='') # abcde
```

note that using an empty string as the sep value causes all the values to be printed without separation.

the end keyword argument defines what print() prints after it prints the last argument. by default, it prints a newline (\n). the most common reasons for using end are compatibility with windows (which sometimes needs a newline of \r\n) and for suppressing the newline altogether. here are two more examples:

```
>>> print(1, 2, 'a', 'b', sep=',', end=' <-\n')
1,2,a,b <-

>>> print('a', 'b', end='', sep=','); print('c', 'd', sep=',')
a,bc,d
```

note the semicolon (;) on line 4: that's an easy way to enter multiple statements on a single line. mostly, you should only use semicolons like this in the repl.

finally, to start a new line immediately without printing anything else, just run print() with no arguments:

```
>>> print()

>>>
```

terminal input

in the previous example, we assigned a value to name and displayed a fixed greeting message that uses that name. that's probably not the most exciting thing you've seen today. we hard-coded the name, which means the program will always use the same name. we must modify and rerun the program to greet a different person. you probably wouldn't bother using a program that worked like that.

can we ask the user to provide her name and greet her with a message that uses the name she entered? yes, we can. python has a built-in function called input() that lets python programs read input from the terminal.

example: greet the user by name

let's use input() to write a program that displays a personalized greeting message for the user based on the name she provides. create a file named personalized_greeting.py with the following code:

personalized_greeting.pycopy code

```
print("what's your name?")
name = input()

print(f'good morning, {name}!')
```

run the program from the command line. it displays the what's your name? question, then waits for your input. when you enter a name and hit return, the program displays a custom greeting message:

copy code

```
$ python personalized_greeting.py
what's your name?
rachele
good morning, rachele!
```
you can also use the input() function to display the prompt the user sees:

```
personalized_greeting.pycopy code
name = input("what's your name? ")

print(f'good morning, {name}!')
```
note the space after the ?: you'll see why that's needed when you run the program:

```
copy code
$ python personalized_greeting.py
what's your name? rachele
good morning, rachele!
```
see how input() retrieved the input from the same line as the prompt? that's why we needed the extra space. without the space, we'd see:

```
copy code
what's your name?rachele
good morning, rachele!
```
alternatively, we can have input() output a newline instead of a space:

```
personalized_greeting.pycopy code
name = input("what's your name?\n")

print(f'good morning, {name}!')
copy code
$ python personalized_greeting.py
what's your name?
rachele
good morning, rachele!
```
that's more like what we started with when we used print() to display the prompt. in this code, the \n is an escape character that the computer treats as a newline. \n is accepted almost universally in programming languages, though the circumstances needed to produce the desired effect may vary slightly.

add two numbers

let's write a program that asks for two numbers from the user, adds them, and then displays the result. put the following code in a new file called sum_numbers.py and then run it:

```
sum_numbers.pycopy code
number1 = input('enter the first number: ')
number2 = input('enter the second number: ')
sum = number1 + number2

print(f'the numbers {number1} and {number2} '
      f'add to {sum}')
copy code
$ python sum_numbers.py
enter the first number: 2
enter the second number: 3
the numbers 2 and 3 add to 23
```
oops. something isn't right. the program reports that the result is 23, not 5 like we want. where do you think the problem lies? if you said that input() returns strings, so + performs concatenation, you're right! since number1 and number2 both hold string values instead of numbers, the + operator concatenates them instead of adding them. if you want to add two variables arithmetically, both must have a numeric data type.

as we learned earlier, we can convert (coerce) strings to numbers with the int()

or float() function. update line 3 from sum_numbers.py to match this code:

sum_numbers.pycopy code

```
number1 = input('enter the first number: ')
number2 = input('enter the second number: ')
sum = float(number1) + float(number2)

print(f'the numbers {number1} and {number2} add to {sum}')
```

copy code

```
$ python sum_numbers.py
enter the first number: 2
enter the second number: 3
the numbers 2 and 3 add to 5.0
```

input() still returns a string, but we've coerced each string to a float this time. with numbers on both sides of the +, python adds them arithmetically.

be sure to play with these examples a bit. you can, for example, replace addition with subtraction or multiplication in the above example. try to think of some applications of input() on your own.

i/o (input/output) in python is a complex topic requiring knowledge of concepts we haven't yet learned. they're essential concepts, but we don't discuss them in this book; we cover them in the core curriculum. until then, we can use print() and input() to interact with users.

functions and methods

as a programmer, you often need a chunk of code in two or more program locations. no matter how often you want to use that code, you don't want to write that code repeatedly. what can you do?

sometimes, you may realize that you have some code that would be perfect for use in another program. can you somehow reuse that code without copying and pasting it?

you may sometimes find yourself with some complicated code that has become hard to read, understand, and maintain. what can you do to improve matters?

most languages support the concept of procedures, blocks of code that run as separate units. in python, we call these functions or methods.

functions help solve all of these problems. they help reduce repetitive code, enable easy code reuse, and improve code readability and maintainability. methods provide the same benefits as functions, but their behaviors are limited to specific objects.

calling functions

using a function means calling, invoking, executing, or running it. all of those terms mean the same thing. when python encounters a function call, it transfers program flow to the code that comprises the function and executes that code. when the code finishes its work, the function returns a value to the code that invoked it. the calling code is free to use or ignore the return value as the programmer sees fit. execution resumes from where the function was called.

to invoke a function, write the function's name followed by a pair of parentheses (). for example, when we call the hello function in the following code, we write hello():

copy code

```
def hello():
    print('hello')
    return true

hello()         # invoking function; ignore return value
print(hello())  # using return value in a `print` call
```

the first 3 lines of this program contain the function definition or function declaration. we'll come back to function definitions a little later in this chapter. for now, notice that the function returns the value true when it is done running.

lines 5 and 6 show two calls to the hello function. in the first invocation, we ignore the return value. in the second, we capture the return value and pass it to the print function for printing.

what does the print function return? that's easy to check:

```
>>> print(print())
none
```

we'll see why that is later.

functions can also take one or more comma-separated arguments between the parentheses. functions usually use arguments to modify the actions they take. a single argument is passed to the print function on lines 2 and 6 above. in this case, it tells print what to print.

to provide additional arguments, separate them with commas:

copy code

```
print('hello', 'good-bye', 'farewell')
```

if you have a lot of arguments (or some longish ones), you can spread them over several lines:

copy code

```
print(
    'hello',
    'good-bye',
    'farewell',
    'adios',
    'ciao',
    'auf wiedersehen',
)
```

built-in functions

built-in functions are functions that are always available in python. many of these functions are used in almost every significant python program. for instance, you should already be comfortable with the print function.

as of python 3.11.4, there are approximately 70 built-in functions. we've already met a number of these:

float, int
str, list, tuple
set, frozenset
input, print
type
len

you can learn more in the python documentation for built-in functions. let's look at a few of these functions (we'll meet even more in the core curriculum).

min and max

you can use the min and max functions to determine a collection's minimum and maximum values. the collection's objects must have an ordering that recognizes the < and > operators for comparing any pair of those objects.

copy code

```
print(min(-10, 5, 12, 0, -20))      # -20
print(max(-10, 5, 12, 0, -20))      # 12

print(min('over', 'the', 'moon'))   # 'moon'
```

```
print(max('over', 'the', 'moon'))   # 'the'

print(max(-10, '5', '12', '0', -20))
# typeerror: '>' not supported between instances
# of 'str' and 'int'
```
you can also use max and min with a single iterable argument, such as a list, set, or tuple:

copy code
```
my_tuple = ('i', 'tawt', 'i', 'taw', 'a',
            'puddy', 'tat')
print(min(my_tuple)) # 'a'
print(max(my_tuple)) # 'tawt'
```
we'll discuss iterables in a later chapter. for now, you only need to know that strings, sequences, mappings, and sets are iterable.

ord and chr
given a single character, ord returns an integer that represents the unicode code point of that character. for the standard ascii character sets, the code points refer to the values of the characters in the standard ascii character set. for example:

copy code
```
print(ord('a'))               # 97
print(ord('a'))               # 65
print(ord('='))               # 61
print(ord('~'))               # 126
```
chr is the inverse of ord. that is, it converts an integer to the corresponding unicode character:

copy code
```
print(chr(97))                # a
print(chr(65))                # a
print(chr(61))                # =
print(chr(126))               # ~
```

truthy and falsy
in the remainder of this assignment, we will use the terms truthy and falsy to describe values. these are not the same thing as true and false, but are a little more general. the following values are said to be falsy:

false, none
all numeric 0 values (integers, floats, complex)
empty strings: ''
empty collections: [], (), {}, set(), frozenset(), and range(0)
custom data types can also define additional falsy value(s).
all other values are said to be truthy.

we will revisit this concept in more detail in the next chapter. for now, you only need to be aware of the concept. you can think of truthy and falsy as true and false for now.

any and all
two very useful built-in functions are the any and all functions. they both operate on iterable collections, such as lists, tuples, ranges, dictionaries, and sets.

the any function returns true if any element in a collection is truthy, false if all of the elements are falsy. on the other hand, all returns true if all of the elements are truthy, false otherwise.

copy code
```
collection1 = [false, false, false]
collection2 = (false, true, false)
```

```
collection3 = {true, true, true}

print(any(collection1))       # false
print(any(collection2))       # true
print(any(collection3))       # true
print(any([]))                # false

print(all(collection1))       # false
print(all(collection2))       # false
print(all(collection3))       # true
print(all([]))                # true
```
notice that any returns false for an empty collection since none of the elements are truthy. however, all returns true for the same collection since none of the elements are falsy (all of the elements are thus truthy).

this example may not seem very useful. however, if you have a collection of something other than booleans, you can use a comprehension with any or all to test whether any or all of the elements meet a certain condition. (we'll meet comprehensions a little later.)

for instance, assume you want to determine whether a list of numbers has any even values in it. you can use a list comprehension to determine which values are even or odd:

copy code
```
numbers = [2, 5, 8, 10, 13]
print([number % 2 == 0 for number in numbers])
# [true false true true false]
```
you can take that one step further to determine whether any or all of the numbers are even:

copy code
```
numbers = [2, 5, 8, 10, 13]
print(any([number % 2 == 0 for number in numbers])) # true
print(all([number % 2 == 0 for number in numbers])) # false

numbers = [5, 13]
print(any([number % 2 == 0 for number in numbers])) # false
print(all([number % 2 == 0 for number in numbers])) # false

numbers = [2, 8, 10]
print(any([number % 2 == 0 for number in numbers])) # true
print(all([number % 2 == 0 for number in numbers])) # true
```
functions for the repl
some functions are intended for use in a repl. these functions provide quick access to information about your program or python itself.

the id function
the id function returns an integer that serves as an object's identity. every object has a unique identity that does not change during the object's lifetime. (the identity may be reused later in the program if the original object is discarded.)

in most cases, two instances of an object with the same value will always have two distinct identities. this is not always true, though. for instance, in a process called interning, every unique integer object from -5 through 256 has the same identity. interning also applies to some strings:

copy code
```
# paste this code into the python repl
# don't run it as a program

s = 'hello, world!'
t = 'hello, world!'
```

```
print(id(s) == id(t))        # false

s = 'supercalifragilisticexpialidocious'
t = 'supercalifragilisticexpialidocious'
print(id(s) == id(t))        # true

x = 5
y = 5
print(id(x) == id(y))        # true

x = 5
y = 6
print(id(x) == id(y))        # false
```

interning yields memory space savings and performance improvements. we discuss it mainly because new python programmers sometimes discover this behavior and think they've found a bug. in practice, it's not an important concept, but it's worth being aware of.

by the way: the reason we asked you to paste that code into the python repl is that python interns different things when you run a program file. in the above code, the first print will output true if you run it as a program.

later on, we'll see why id is useful.

interning is an implementation detail that varies based on the flavor and version of python you are using. while most flavors and versions intern the small integers shown above, some intern other values such as certain strings.

python "flavors" are different implementations of python. the flavor you are most likely using is cpython, but other flavors include jython, pypy, anacondapython, and more.

the dir function

when used without arguments, the dir function returns a list of all identifiers in the current local scope. when used with an argument, dir() returns a list of the object's attributes (typically, the object's methods and instance variables).

```
>>> dir()
['__builtins__', '__name__', 'struct']

>>> dir(5)
['__abs__', '__add__', '__and__', '__bool__',
    ... a bunch of stuff omitted ...,
 '__xor__', 'as_integer_ratio', 'bit_count',
 'bit_length', 'conjugate', 'denominator',
 'from_bytes', 'imag', 'numerator', 'real',
 'to_bytes']
```

many of the names shown by dir begin and end with two underscores. these are names for the so-called dunder (double-underscore) or magic methods and magic variables. we'll encounter some of these in the object oriented programming book.

helpful hint: use the sorted function with the output of dir:

```
>>> sorted(dir(range(1)))
['__bool__', '__class__', '__contains__',
 '__delattr__', '__dir__', '__doc__', '__eq__',
    ... a bunch of stuff omitted ...,
 'count', 'index', 'start', 'step', 'stop']
```

another helpful hint: use pretty print to print the output in an easier to read format:

```
>>> from pprint import pp
>>> names = sorted(dir(range(1)))
```

```
>>> pp(names)
['__bool__',
 '__class__',
 '__contains__',
 '__delattr__',
 ... a bunch of stuff omitted ...,
 'count',
 'index',
 'start',
 'step',
 'stop']
```
yet another helpful hint: use a comprehension to limit the output to just the names that don't contain __.

```
>>> names = sorted(dir(range(1)))
>>> names = [name for name in names
...          if '__' not in name]
>>> print(names)
['count', 'index', 'start', 'step', 'stop']
```
the help function
when used without arguments, the help function prints some basic help on how to use help, then leaves you in a special help mode that uses a help> prompt.

```
>>> help()

welcome to python 3.11's help utility!

if this is your first time using python, you should definitely check out
the tutorial on the internet at https://docs.python.org/3.11/tutorial/.

enter the name ...
   <omitted text>

help>
```
from the help> prompt, you can request help on module names, python keywords, built-in functions, class names, etc. type the appropriate words at the help> prompt and press {return} or {enter}.

quitting the help system may involve two separate steps. if you are currently reading a long help item (such as the help for str), you may need to press q to terminate the output. after terminating the output, you may need to type another q and then press {return} or {enter} to exit from the help> prompt.

as of python 3.13.0, you no longer need to use write help() to access the help system. you can, instead, just write help. when supplying an argument to help, you still need to use the parentheses.

you can also request help more directly by placing an appropriate identifier between the parentheses:

```
>>> help(ord)
help on built-in function ord in module builtins:

ord(c, /)
    return the unicode code point for a one-character string.

>>> help(bool)
help on class bool in module builtins:

class bool(int)
 |  bool(x) -> bool
 |
 |  returns true when the argument x is true, false otherwise.
 |  the builtins true and false are the only two instances of the class bool.
```

```
 |  the class bool is a subclass of the class int, and cannot be subclassed.
 |
 |  method resolution order:
 |      bool
 |      int
 |      object

   <omitted text>

>>> help('topics')

here is a list of available topics. enter any topic name to get more help.

assertion           deletion            looping             shifting
assignment          dictionaries        mappingmethods      slicings
attributemethods    dictionaryliterals  mappings            specialattributes
attributes          dynamicfeatures     methods             specialidentifiers
augmentedassignment ellipsis            modules             specialmethods
   <omitted text>

>>> help('boolean')
boolean operations
******************

   or_test  ::= and_test | or_test "or" and_test
   and_test ::= not_test | and_test "and" not_test
   not_test ::= comparison | "not" not_test

in the context of boolean operations, and also when expressions are
used by control flow statements...

   <omitted text>
creating functions
unless a function is built-in or provided by an imported module, you must create
it. a typical function definition (a.k.a., function declaration) looks like
this:

copy code
def func_name():
    func_body
here, we're assigning the function to func_name, and func_body is some code that
performs the function's required action(s).

here's a function named say that prints hi!:

copy code
def say():
    print('hi!')
why do we want a function named say? to say something, of course! let's try it.
first, we'll create a file named say.py, then place the following code inside
the file.

say.pycopy code
def say():
    print('output from say')

print('first')
say()
print('last')
save the file and run it from the terminal:

$ python say.py
first
output from say
```

last
on line 5 of say.py, the code say() is a function call to the say function. when python runs this program, it creates a function named say whose body causes python to print the text output from say when the function executes. however, that doesn't happen immediately.

on line 4, we use print to display first on the terminal. on line 5, we call the function say by appending a pair of parentheses -- () -- to the function's name. this causes python to temporarily jump into the function's body, which prints the text output from say. finally, python returns to the code that immediately follows the call to say and prints last.

note that the parentheses on line 5 -- () -- make this code a function call. without the parentheses, say doesn't do anything useful. it's the function's name, not an invocation. forgetting the parentheses is usually a bug that can be tough to find since the code may run long after the line with the error. it just won't work. we'll learn about function objects later in the core curriculum.

python programmers often add a triple-quoted string at the beginning of a function's block. this string is a documentation comment -- a docstring -- that python can access with its help() function and the __doc__ property. it has no effect on your code unless your program is somehow interested in the comments (which can happen):

copy code
```
def say():
    """
    the say function prints "hi!"
    """
    print('hi!')

print('-' * 60)
print(say.__doc__)
print('-' * 60)
help(say)
```
outputcopy code
```
------------------------------------------------------------

    the say function prints "hi!"

------------------------------------------------------------
```
help on function say in module __main__:

```
say()
    the say function prints "hi!"
```
we won't follow this convention at launch school, but you should learn to recognize it. you will likely encounter it elsewhere.

scope
the scope of an identifier determines where you can use it. python determines scope by looking at where you initialize the identifier. in python, identifiers have function scope. that means that anything initialized inside a function is only available within the body of that function and any nested functions. no code outside of the function body can access that identifier.

consider this code:

copy code
```
def well_howdy(who):
    greeting = 'howdy'
    print(f'{greeting}, {who}')

well_howdy('angie')
print(greeting)
```

in this code, we have a greeting variable in the body of well_howdy. when we call the function, howdy is assigned to greeting, and a more complete greeting message is printed.

when the function is done running, we try to print the value of the greeting variable. however, instead of seeing howdy, we get an error message instead:

copy code
nameerror: name 'greeting' is not defined
the error occurs since the greeting variable is only available inside the well_howdy function. the surrounding code has no way to access the variable.

what happens if we define our own greeting variable in the outer scope?

copy code
```
greeting = 'salutations'

def well_howdy(who):
    greeting = 'howdy'
    print(f'{greeting}, {who}')

well_howdy('angie')
print(greeting)
```
this time, the code prints salutations on line 8, thus showing that greeting is in scope on line 8. however, the outer greeting variable plays no part in the function's body. the assignment on line 4 hides the greeting variable from line 1. we call this variable shadowing.

let's remove the assignment on line 4 and see what happens:

copy code
```
greeting = 'salutations'

def well_howdy(who):
    print(f'{greeting}, {who}')

well_howdy('angie')
print(greeting)
```
this time, we get:

copy code
```
salutations, angie
salutations
```
the key difference here is that we are no longer assigning the greeting variable in this function. instead, we're just accessing its current value. it's the assignment in the first example in this section that causes python to create a new local variable named greeting. without any assignments in the function body, python looks for greeting in the lexical scope, which means it searches the outer scopes for the nearest definition of greeting. in this example, the only outer scope of concern is the topmost scope, i.e., the global scope.

if you're familiar with some other programming languages, the following example may be a little surprising:

copy code
```
def scope_test():
    if true:
        foo = 'hello'
    else:
        bar = 'goodbye'

    print(foo)
    print(bar)
```

scope_test()
outputcopy code
hello
unboundlocalerror: cannot access local variable
'bar' where it is not associated with a value
in some languages, the corresponding print code may treat both foo and bar as out of scope. both names may be in scope in other languages, but one will have a default value such as nil or undefined. in still other languages, only one name is in scope. python comes closest to this last approach, but both names are technically in scope. however, since the else block never runs, bar is left unassigned. thus, we get the unboundlocalerror message when we try to print it.

as you might expect, constants have the same scoping behavior as variables. in fact, so do function, parameter, class, and module names. method names act a little differently, but that's a topic for another time.

namespaces
closely related to the concept of scope is the concept of a namespace. in python, a namespace is defined as a mapping of identifiers to objects. in essence, namespaces define the scopes in which python will look for identifiers. when you refer to an identifier, python first looks for the identifier in the local namespace. that is, it looks in the local scope. if the identifier is not found in the local namespace, python next looks in any enclosing namespaces, i.e., the outer scopes (but not including the global scope). next, it searches the global namespace, which is equivalent to the global scope, and, finally, the built-in namespace.

we will not often refer to namespaces at launch school, but you may encounter the term when looking at other material.

arguments & parameters
create a new file named say2.py with the following code:

say2.pycopy code
```
print('hello')
print('hi')
print('how do you do')
print('quite all right')
```
go ahead and run it if you want. notice how we've duplicated the print call several times. instead of calling it every time we want to output something, we want to have code we can call when we need to print something.

now, let's update say2.py as follows:

say2.pycopy code
```
def say(text):
    print(text)

say('hello')
say('hi')
say('how do you do')
say('quite all right')
```
at first glance, this program seems silly. it doesn't reduce the amount of code. in fact, it adds more. also, the say function doesn't provide any functionality that print doesn't already offer. however, there is a benefit here. we've extracted the logic for displaying text in a way that makes our program more flexible. if we need to change the output style, we can change it in one place: the say function. we don't have to change any other code to get the updated behavior. we'll see such an update in a few minutes.

as we saw earlier, we invoke functions by typing their name and a pair of parentheses. say2.py illustrates how we define and call a function that takes a value known as an argument. arguments let you pass data from outside the function's scope into the function so it can access that data. you don't need

arguments when the function doesn't need access to outside data.

in say2.py, the function definition includes (text) after the function name. this syntax tells us that we should supply (pass) a single argument to the function when we call it. we include an argument between the invocation parentheses to pass it to say. thus, say('hello') passes the argument 'hello' to the function. we assign the argument's value to the text parameter inside say.

the names between parentheses in the function definition are called parameters. you can think of parameters as placeholders for potential arguments, while arguments are the values assigned to those placeholders.

function names and parameters are both considered variable names in python. parameters, in particular, are local variables. they are defined locally within the function's body. a function's name is global or local, depending on whether it is at the program's top level or nested inside a class, module, or another function. we'll see several exercises about these issues below; they are essential even if there isn't much to say about them.

since parameters are merely placeholders, we typically talk of them as declarations: they are being used to introduce those names as local variables into a function, but don't obviously provide an immediate value until the function is called. that is, they simply declare variable names.

arguments are objects passed to a function during invocation. parameters are placeholders for the objects that will be passed to the function when it is invoked.

many developers conflate the terms "parameter" and "argument" and use them interchangeably. the difference is typically not crucial to understanding, but using correct terminology is a good idea. we expect you to do so on launch school assessments. parameters are not arguments. arguments are not parameters.

you can name functions and parameters with any valid variable name. however, you should use meaningful, explicit names that follow the same naming conventions discussed in the variables chapter. in the above example, we name the parameter text since the say function expects the caller to pass in some text. other suitable names might be sentence or message.

when we call the say function, we should provide a value -- the argument -- to assign to the text parameter. for instance, on line 4 of say2.py, we pass the value 'hello' to the function, which it uses to initialize text. the function can use the value in any way it requires by referencing the parameter name. it can even ignore the argument entirely.

remember: a parameter's scope is the function where the parameter is used. thus, the scope of the text parameter in say2.py is the say function. you can't reference the parameter outside of the function's body.

you can define functions that take any number of parameters and accept any number of arguments. for instance, the add function defined below takes two parameters, and we invoke it with 2 arguments:

```
add.pycopy code
def add(left, right):
    sum = left + right
    return sum

sum = add(3, 6)
```

in the above code, left and right are parameters on line 1. on line 2, however, they are local variables that contain the argument values passed to add on line 5.

python will raise an error if you pass too many or too few arguments to a

function. however, it is possible to bypass this restriction, as we'll see later.

let's return to say2.py. when we called say('hello'), we passed the string 'hello' as the argument. thus, 'hello' was assigned to the text parameter.

as mentioned earlier, one benefit that functions give us is the ability to make changes in one location. suppose we want to add a ==> to the beginning of every line that say outputs. all we have to do is change one line of code -- we don't have to change the function invocations:

say2.pycopy code
```
def say(text):
    print('==> ' + text)

say('hello')        # ==> hello
say('hi')           # ==> hi
say('how are you')  # ==> how are you
say("i'm fine")     # ==> i'm fine
```
run this code to see the result. we've now added a ==> to the beginning of each line. we only had to change a single line of code. now, you're starting to see the power of functions.

return values
as we've seen in some earlier examples, functions can perform an operation and return a result to the caller for later use. we do that with return values and the return statement.

all python function calls evaluate to a value, provided the function doesn't raise an exception (an error). by default, that value is none; this is the implicit return value of most python functions. however, when you use a return statement, you can return a specific value from a function. this is an explicit return value. there is no distinction between implicit and explicit return values outside the function. still, it's important to remember that all functions return something unless they raise an exception, even if they don't explicitly execute a return statement.

let's create an add function that returns the sum of two numbers:

copy code
```
def add(a, b):
    return a + b

add(2, 3)           # returns 5
```
when you run this program, it doesn't print anything since add doesn't call print or any other output functions. however, the function does return a value: 5. when python encounters the return statement, it evaluates the expression, terminates the function, then returns the expression's value to the location where we called add.

let's capture the return value in a variable and print it to verify that:

copy code
```
def add(a, b):
    return a + b

two_and_three = add(2, 3)
print(two_and_three)  # 5
```
python uses the return statement to return a value to the code that invoked the function. if you don't specify a value, it returns none. either way, the return statement terminates the function and returns control to the calling function.

functions that always return a boolean value, i.e., true or false, are called predicates. for example, the following function is a predicate:

```
copy code
def is_digit(char):
    if char >= '0' and char <= '9':
        return true

    return false
```
you will almost certainly encounter this term in future readings and videos, so commit it to memory.

default parameters
it's sometimes helpful to invoke a function without some of its arguments. you can do that by providing default values for the function's parameters. let's update say to use a default value when the caller omits the argument.

```
copy code
def say(text='hello'):
    print(text + '!')

say('howdy') # howdy!
say()        # hello!
```
notice that invoking say without arguments, as on line 5, prints 'hello!'. we can invoke say without arguments since we've provided a default value for text. nice!

you can provide defaults for any or all of a function's parameters. however, once you specify a default value for a parameter, all subsequent positional parameters must also have default values:

```
copy code
def say(msg1, msg2, msg3='dummy message', msg4):
    pass
# syntaxerror: non-default argument follows default argument
```
it's also worth noting that you can't accept the default value for a parameter and provide an explicit value for a subsequent parameter:

```
copy code
def say(msg1, msg2, msg3='dummy message',
                    msg4='omitted message'):
    print(msg1)
    print(msg2)
    print(msg3)
    print(msg4)

say('a', 'b', 'c', 'd')
# a
# b
# c
# d

say('a', 'b', 'c')
# a
# b
# c
# omitted message

say('a', 'b')
# a
# b
# dummy message
# omitted message

say('a', 'b', , 'd')
# a
```

```
# b
# d               # oops - expecting 'dummy message'
# omitted message # oops again - expected 'd'
```
python has a variety of ways to specify parameters. the easiest is with positional parameters. with positional parameters, the parameter values are taken from the corresponding argument position. thus, if you have a function that takes 3 parameters, the first parameter is set to the first argument, the second parameter to the second argument, and the third parameter to the third argument. for instance, in the say function above, the 4 parameters are assigned based on the order of the arguments. of course, default parameters mean you can omit some arguments.

we'll see some of python's other parameter types later in the core curriculum.

functions vs. methods

thus far, all our function calls have used the function_name(obj, ...) syntax. we call a function by writing parentheses after its name. any arguments the function needs are provided inside the parentheses.

however, suppose you want to convert a string to all uppercase letters. you might expect to use a function invocation like upper(my_str). however, that won't work. you must use a different syntax called method invocation.

method invocations occur when you prepend an object followed by a period (.) to a function invocation, e.g., 'xyzzy'.upper(). we call such function invocations method calls. we also speak of the function as a method. we cover this topic in more detail in the core curriculum when we get to object-oriented programming. you can think of the previous code as the function upper returning a modified version of the string 'xyzzy'.

methods provide the same benefits as functions. however, methods work with specific objects. all methods are functions but not vice versa. every method belongs to a class and requires an object of that class to call it. for instance, in the following example, we're using the string object represented by my_str to call the upper method from the str class:

copy code
```
my_str = 'abc-123-def'
print(my_str.upper())         # abc-123-def
```
writing your own methods requires classes, which is how you create custom data types in python. we're not ready to write our own classes and methods at this point; you'll learn how to do that in our object oriented programming in python book.

in python, the distinction between functions and methods may sometimes seem fuzzy. some function invocations look like method invocations. for instance, consider the math module from python's standard libraries. the module provides some mathematical functions that any program can use. once you import the module, you just call the functions you need:

copy code
```
import math

print(math.sqrt(5))           # 2.23606797749979
```
wait. is that a method call? it sure looks like one. it quacks like one. since math references an object, math.sqrt() must be a method call. however, it is not. while math is technically a reference to a module object, we're not using it to perform operations on that object. the sole purpose of the math object here is to tell python where to look for those functions.

for brevity, we often speak of functions when discussing functions and methods together. however, you should say methods when discussing functions explicitly designed to require calling objects.

mutating the caller
sometimes, a method mutates the object used to invoke the method: it mutates the caller. in the following example, the pop method mutates the caller (the list referenced by odd_numbers):

copy code
```
odd_numbers = [1, 3, 5, 7, 9]
odd_numbers.pop()
print(odd_numbers)  # [1, 3, 5, 7]
```
we can also talk about whether functions mutate their arguments. the following function illustrates this concept:

copy code
```
def add_new_number(my_list):
    my_list.append(9)

numbers = [1, 2, 3, 4, 5]
add_new_number(numbers)
print(numbers) # [1, 2, 3, 4, 5, 9]
```
this code uses the list.append method to add a new number to the list my_list.thus, list.append mutates its caller. however, the add_new_number function doesn't mutate a caller (there is no caller). instead, it mutates its argument.

in general, mutating the caller is acceptable practice; many built-in functions and methods do just that. however, you should avoid mutating arguments since such functions can be tough to debug and is considered poor practice. almost no built-in functions mutate their arguments.

copy code
```
# returning a new object
def add_new_number(my_list):
    return my_list + [9]

numbers = [1, 2, 3, 4, 5]
new_numbers = add_new_number(numbers)
print(new_numbers) # [1, 2, 3, 4, 5, 9]
print(numbers)     # [1, 2, 3, 4, 5]
```
how do you know which methods mutate the caller and which don't? that's easy when the caller is immutable: the answer is always, "this method does not mutate the caller." however, when the caller is mutable, there's no way to say without inspecting the method's code or checking the documentation.

Flow Control
A computer program is like a journey for your data. Data encounters situations that impact it during this journey, changing it forever. Like any journey, one has a choice of routes to reach the destination. Sometimes, data takes one path; sometimes, another. Which roads the data takes depends on the program's end goal.

When writing programs, you want your data to take the correct path. You want it to turn left or right, up, down, reverse, or proceed straight ahead when it's supposed to. We call this flow control.

How do we make data do the right thing? We use conditionals.

Conditionals
A conditional is a fork, or multiple forks, in the road. When your data arrives at a conditional, Python evaluates the conditional and tells the data where to go. The simplest conditionals use a combination of if statements with comparison and logical operators (<, >, <=, >=, ==, !=, and, or, and not) to direct traffic. They use the keywords if, elif, and else.

That's enough talking; let's write some code. Create a file named conditional.py

with the following content:

conditional.pyCopy Code
```
value = int(input('Enter a number: '))

if value == 3:
    print('value is 3')
```
Run conditional.py at least twice.

The first time, enter the value 3.
The second and subsequent times, input any other integer value.
This example shows the simplest of if statements: it has a single block (one or more Python statements or expressions) that executes when the condition (value == 3) is True. Otherwise, Python bypasses the block. Regardless, execution eventually resumes with the first statement or expression after the if statement.

Thus, if the input value is 3, this code prints value is 3. The code doesn't print anything if the user inputs any other integer.

We can expand the if statement to include some code that runs when value is not 3:

conditional.pyCopy Code
```
value = int(input('Enter a number: '))

if value == 3:
    print('value is 3')
else:
    print('value is NOT 3')
```
Here, Python prints value is 3 if the user enters 3; otherwise, it prints value is NOT 3.

Note that the else block isn't a proper statement; it's part of the if statement.

We can nest if statements inside an outer if. In this next example, we nest an if statement inside the else block of the outer if:

conditional.pyCopy Code
```
value = int(input('Enter a number: '))

if value == 3:
    print('value is 3')
else:
    if value == 4:
        print('value is 4')
    else:
        print('value is NOT 3 or 4')
```
This time, run conditional.py at least three times:

The first time, enter the value 3.
The second time, enter the value 4.
The third and subsequent times, input any other integer value.
The indentation levels show how the code is supposed to work. In this case, Python:

prints value is 3 if the user inputs 3.
prints value is 4 if the user inputs 4.
prints value is NOT 3 or 4 if the user enters any other integer.
The sequence of operations begins on line 3, where we compare the user input against 3. If yes, line 4 runs; otherwise, we drop down to the else block. In the else block, we compare the input against 4 on line 6. If yes, line 7 runs; otherwise, we drop down to the inner else block and run the code on line 9.

We recommend avoiding nested if statements when possible. They quickly become difficult to read with multiple levels of nesting or longish code blocks. However, don't get twisted up trying to avoid them entirely. Keep the nesting to a modest 2 or 3 levels deep and use functions to isolate some of the more complex code.

You can rewrite the previous example using an elif block:

conditional.pyCopy Code
```
if value == 3:
    print('value is 3')
elif value == 4:
    print('value is 4')
else:
    print('value is NOT 3 or 4')
```
The elif block runs if value == 3 is False and value == 4 is True. The code produces the same results as the nested if.

You can have as many elif blocks as you need, but they all need to be after the if block and, if the code has one, before the else block. The elif conditionals are evaluated in the order they appear in the code.

Finally, if statement blocks may contain as many lines as you need:

conditional.pyCopy Code
```
if value == 3:
    print('value is 3')
    print('value is an odd number')
    print('value is a prime number')
elif value == 4:
    print('value is 4')
    print('value is an even number')
    print('value is NOT a prime number')
elif value == 9:
    print('value is 9')
    print('value is an odd number')
    print('value is NOT a prime number')
else:
    print('value is something else')
```
Every once in a while, you may want to create a block in an if statement that does nothing. We usually do this for readability purposes. However, blocks can't be empty. Instead, you have to use a pass statement.

Copy Code
```
if value == 3:
    print('value is 3')
elif value == 4:
    print('value is 4')
elif value == 9:
    pass # We don't care about 9
else:
    print('value is something else')
```
Adding a comment to a pass is good practice so future programmers know why it is there.

All statements in a block must be indented from the statement that begins the block. The indentation in a block must be consistent. If the first line of the block is indented 4 spaces, all statements in the block must be indented 4 spaces.

Nested blocks should be indented more than the containing block. For instance, if the current block is indented by 4 spaces from the outer block (the conventional amount of indentation), then a nested block inside that block

should be indented by 8 spaces. Another nested block would bring the indentation to 12 spaces.

Be careful with your indentation. If you accidentally outdent some code, that will end the block. For instance:

Copy Code
```
if value == 1:
    print('value is one')
print('the value is odd')
```
If you meant to print both messages when value is 1, that's what will happen. However, Python will display the second message even if value is not 1.

Comparisons
Let's look at the comparison operators in some more depth so you can build more complicated conditional statements. Remember that comparison operators return a Boolean value: True or False.

The expressions to the left and right of an operator are its operands. For instance, the equality comparison x == y uses the == operator with two operands, x and y.

==

The equality operator returns True when the operands have equal values, False otherwise. We discussed == briefly in the Basics Operations chapter. It should be familiar, even if it still looks strange.

Copy Code
```
print(5 == 5)                # True
print(5 == 4)                # False

print('abc' == 'abc')        # True
print('abc' == 'abcd')       # False

print(5 == '5')              # False

print([1, 2, 3] == [1, 2, 3]) # True
print([1, 2, 3] == [3, 2, 1]) # False
```
In most cases, operands must have the same type and value to be equal. Thus, 5 is not equal to '5'. There are some places where you can mix types, however. For instance, integers and floats that are mathematically equivalent are usually, but not always, considered equal:

Copy Code
```
print(5 == float(5))               # True

big_num = 12345678901234567
print(float(big_num) == big_num)    # False
```
Enormous floats lack precision at around 18 significant digits on most modern machines. That can lead to surprises if you happen to work with big numbers.

Comparisons with strings are case-sensitive. Thus, 'abc' is not equal to 'aBc'. You can use the str.lower and str.upper methods to achieve a case-insensitive comparison:

Copy Code
```
print('abc' == 'aBc')                 # False
print('abc'.lower() == 'aBc'.lower()) # True
print('abc'.upper() == 'aBc'.upper()) # True
```
In some non-US alphabets, converting text to upper or lower case isn't straightforward. For instance, in German, 'straÃŸe' and strasse are considered equivalent. However, the following code prints False:

Copy Code
```
'straÃŸe'.lower() == 'strasse'.lower()
```
The str.casefold method makes allowances for this issue and does the right thing:

Copy Code
```
'straÃŸe'.casefold() == 'strasse'.casefold()
```
While casefold is only needed when working with non-US characters, it's best practice in Python to use casefold instead of lower or upper, especially when comparing strings.

!=

The inequality operator, !=, is =='s inverse: It returns False when == would return True, and True when == would return False. It returns False when the operands have the same type and value, True otherwise. Other than the return value, the behaviors of == and != are identical.

Copy Code
```
print(5 != 5)             # False
print(5 != 4)             # True
print('abc' != 'abc')     # False
print('abc' != 'aBc')     # True
print(5 != '5')           # True
```
< and <=

The less than operator (<) returns True when the value of the left operand has a value that is less than the value on the right, False otherwise. The less than or equal to operator (<=) is similar, but it also returns True when the values are equal; < returns False when the operands are equal.

Copy Code
```
print(4 < 5)              # True
print(5 < 4)              # False
print(5 < 5)              # False

print(4 <= 5)             # True
print(5 <= 4)             # False
print(5 <= 5)             # True

print('4' < '5')          # True
print('5' < '4')          # False
print('5' < '5')          # False

print('42' < '402')       # False
print('42' < '420')       # True
print('420' < '42')       # False
```
The examples with strings are especially tricky here! Make sure you understand them. Python compares strings character-by-character, moving from left to right. It looks for the first character that differs from its counterpart in the other string. Once it finds differing characters, it compares them to determine the relationship.

If both strings are equal up to the shorter string's length, as in the last two examples, the shorter one is considered less than the longer one.

> and >=

The greater than operator (>) returns True when the value of the left operand has a value that is greater than the value on the right, False otherwise. The greater than or equal to operator (>=) is similar, but it also returns True when the values are equal; > returns False when the operands are equal.

Copy Code

```
print(4 > 5)              # False
print(5 > 4)              # True
print(5 > 5)              # False

print(4 >= 5)             # False
print(5 >= 4)             # True
print(5 >= 5)             # True

print('4' > '5')          # False
print('5' > '4')          # True
print('5' > '5')          # False

print('42' > '402')       # True
print('42' > '420')       # False
print('420' > '42')       # True
```

As with < and <=, you can compare strings with the > and >= operators; the rules are similar.

Logical Operators

You're beginning to get a decent grasp of basic conditional flow. Let's take a few minutes to see how we can combine conditions to create more complex scenarios. The not, and, and or logical operators provide the ability to combine conditions:

not

The not operator returns True when its operand is False and returns False when the operand is True. That is, it negates its operand.

Copy Code

```
print(not True)           # False
print(not False)          # True
print(not(4 == 4))        # False
print(not(4 != 4))        # True
```

In these examples, Python first evaluates the expression on the right, then applies not to the result, thus negating it. For instance, we know that 4 == 4 is True, so not(4 == 4) is False.

You can omit the parentheses in the last two examples. However, we don't recommend it. Operator precedence issues may occur if you let Python decide which operator to evaluate first. Use parentheses if you have anything more complex than a single identifier or literal to the right of not.

If you use the parentheses, you should not add a space after not. For instance, write not(4 == 4); don't write not (4 == 1).

Unlike most operators, not takes a single operand; it appears to the operator's right. Operators that take only one operand are called unary operators. Operators that take two operands are binary operators, though you'll rarely hear that term.

and and or

The and operator returns True when both operands are True. It returns False when either operand is False. The or operator returns True when either operand is True and False when both operands are False.

The following truth table shows how True and False interact with the and and or operators. You should memorize this table:

A	B	A and B	A or B
True	True	True	True
True	False	False	True
False	True	False	True

False False False False
For completeness, let's see a few examples:

Copy Code
```
print((4 == 4) and (7 == 7))        # True
print((4 == 4) and (7 == 3))        # False
print((4 == 9) and (7 == 7))        # False
print((4 == 9) and (7 == 3))        # False
```
Copy Code
```
print((4 == 4) or (7 == 7))         # True
print((4 == 4) or (7 == 3))         # True
print((4 == 9) or (7 == 7))         # True
print((4 == 9) or (7 == 3))         # False
```
As with not, parentheses aren't always needed. However, the same "precedence" issues may occur. Always use parentheses if the corresponding operand is not a literal or identifier.

Short Circuits
The and and or operators use a mechanism called short circuit evaluation to evaluate their operands. Consider these two expressions:

Copy Code
```
is_red(item) and is_portable(item)
is_green(item) or has_wheels(item)
```
The first expression returns True when item is red and portable. If either condition is False, then the overall result must be False. Thus, if the program determines that item is not red, it doesn't have to determine whether it is also portable. Python short-circuits the rest of the expression by terminating evaluation if it determines that item isn't red. It doesn't need to call is_portable() since it already knows the expression must be False.

Similarly, the second expression returns True when item is either green or has wheels. When either condition is True, the overall result must be True. Thus, if the program determines that item is green, it doesn't have to decide whether it has wheels. Again, Python short-circuits the entire expression once it knows that item is green; the expression must be True.

Truthiness
Many languages can evaluate objects and values as either truthy or falsy. Python is no slouch here; it can too. It can evaluate every object's truthiness. Note that these terms are not synonymous with True, False, and Boolean. In addition, truthy and falsy are not actual objects or values. Instead, they are terms that describe how specific objects behave in a Boolean context.

Truthiness arises in conditional expressions, such as if and while statements. Conditional expressions don't need to produce Boolean values. Instead, Python only needs to determine their truthiness. In an if statement, a conditional expression that evaluates as truthy causes the if block to execute. The else or elif block runs when the expression evaluates as falsy.

Since conditionals only care about truthiness, we can use any expression as the condition. The expression will always be evaluated as truthy or falsy.

So, which values are truthy? Which are falsy? The built-in falsy values are as follows:

False, None
all numeric 0 values (integers, floats, complex)
empty strings: ''
empty collections: [], (), {}, set(), frozenset(), and range(0)
Custom data types can also define additional falsy value(s).
Okay, now that we know what's falsy, what's truthy? Everything else.

Enough yammering. Let's see some examples:

Copy Code
```
value = 5                    # 5 is truthy
if value:
    print(f'{value} is truthy')
else:
    print(f'{value} is falsy')
```
Copy Code
```
value = 0                    # 0 is falsy
if value:
    print(f'{value} is truthy')
else:
    print(f'{value} is falsy')
```
The first example prints 5 is truthy while the second prints 0 is falsy. This works since Python evaluates 5 as truthy and 0 as falsy.

You may sometimes see articles that speak of true and false values; this even happens in the Python documentation. The authors should probably talk of truthy and falsy evaluations, instead, not true and false. At Launch School, we want you to use truthy and falsy when speaking of truthiness, True and False when talking of booleans, and true and false when discussing truths and falsehoods.

You may have noticed how we took care to say things like evaluates as truthy. For brevity, you can simply describe expressions as either truthy or falsy. The "evaluates as" terminology is unnecessary.

Truthiness and Short-Circuit Evaluation

You may recall that the and and or logical operators cause short-circuit evaluation. You may not realize that the logical operators don't always return True or False. Both operators care only about the truthiness of their operands. The final value returned by an expression using and and or is the value of the final sub-expression evaluated by Python:

Copy Code
```
print(3 and 'foo')   # last evaluated op: 'foo'
print('foo' and 3)   # last evaluated op: 3
print(0 and 'foo')   # last evaluated op: 0
print('foo' and 0)   # last evaluated op: 0
```
Copy Code
```
print(3 or 'foo')    # last evaluated op: 3
print('foo' or 3)    # last evaluated op: 'foo'
print(0 or 'foo')    # last evaluated op: 'foo'
print('foo' or 0)    # last evaluated op: 'foo'
print('' or 0)       # last evaluated op: 0
print(None or [])    # last evaluated op: []
```
Suppose you have a logical expression that returns a non-Boolean object instead of a Boolean:

Copy Code
```
foo = None
bar = 'qux'
is_ok = foo or bar
```
In this code, is_ok gets set to the truthy value, 'qux'. In most cases, you can use 'qux' as though it were a Boolean True value. However, using a string as a Boolean isn't always the best way to write your code. It may even look like a mistake to another programmer trying to track down a bug. In some strange cases, it may even be a mistake.

You can readily address this with an if/else statement:

Copy Code
```
if foo or bar:
    is_ok = True
```

```
else:
    is_ok = False
```
This snippet sets is_ok to either True or False based on the truthiness of foo or bar. However, it is wordy. Many Python programmers would write this more concisely as:

Copy Code
```
is_ok = bool(foo or bar)
```
Logical Operator Precedence
As discussed in an earlier chapter, Python has precedence rules for evaluating expressions that use multiple operators and sub-expressions. The following list shows the precedence of the comparison operators from highest (top) to lowest (bottom).

==, !=, <=, <, >, >= - Comparison
not - Logical NOT
and - Logical AND
or - Logical OR
Thus, if you have an expression like not x == y, you can know that x == y is evaluated first, then not is applied to the overall result. That is, not x == y is equivalent to not(x == y).

Things get really confusing when combining and and or in an expression. Even though and has higher precedence than or, the fact that both are short-circuiting operators adds a whole lot of complexity. For example, can you determine what the following code prints?

Copy Code
```
print(1 or 2 and 3)
print(0 or 2 and 3)
print(1 or 0 and 3)
print(1 or 2 and 0)
print(0 or 0 and 3)
print(0 or 2 and 0)
print(1 or 0 and 0)
print(0 or 0 and 0)

print(1 and 2 or 3)
print(0 and 2 or 3)
print(1 and 0 or 3)
print(1 and 2 or 0)
print(0 and 0 or 3)
print(0 and 2 or 0)
print(1 and 0 or 0)
print(0 and 0 or 0)
```
Go ahead and guess what will be output, then try running the code to see the results. In all likelihood, you will guess incorrectly at least once.

We're not going to try to explain what's happening with this code. While you might encounter some code like this in the future, the mixed and/or code will likely only be a small part of your problems.

In short: do not write code like this! If you must mix and and or, use parentheses to control how the code gets written. For instance, compare these two lines of code:

Copy Code
```
print((a and b) or (c and d))
print(a and b or c and d)
```
The first line, while complex, is easier to understand than the second.

To repeat, avoid mixing and and or in a single expression unless you use parentheses to control the order of evaluation.

match/case Statement
The last conditional flow structure we want to discuss is the match/case statement (or, more concisely, the match statement). A match statement, in its most basic form, is similar to an if statement but has a different interface. It compares a single value against multiple values, whereas if can test multiple expressions with any condition.

The Python match statement was introduced in Python 3.10. You won't be able to use it in earlier versions, such as the 3.9 version provided by Cloud9 with the Amazon Linux 2023 operating system. However, it's straightforward to rewrite the match statements we will use as if statements.

match statements use the reserved words match and case. It's often easier to show rather than tell, and that's certainly the case with the match statement. First, create a file named match.py with this content:

match.pyCopy Code
```
value = 5

match value:
    case 5:
        print('value is 5')
    case 6:
        print('value is 6')
    case _: # default case
        print('value is neither 5 nor 6')
# value is 5
```
This example is functionally identical to the following if/else statement:

Copy Code
```
value = 5

if value == 5:
    print('value is 5')
elif value == 6:
    print('value is 6')
else:
    print('value is neither 5 nor 6')
# value is 5
```
You can see how similar they are, but you can also see how they differ. The match statement evaluates the expression value, compares its value to the value in each case, and executes the block associated with the first matching case. In this example, the value of the expression is 5; thus, the program executes the statements associated with case 5. The statements in the case _ block run when the expression doesn't match any other case blocks. It acts like the final else in an if statement and must be the last case block in the match statement.

If you want to match multiple values in a case, you can do so by using the | character to separate item values:

Copy Code
```
value = 5

match value:
    case 1 | 2 | 3 | 4:
        print('value is < 5')
    case 5 | 6:
        print('value is 5 or 6')
    case _: # default case
        print('value is not 1, 2, 3, 4, 5, or 6')
# value is 5 or 6
```
There are plenty of uses for match statements. They also have "pattern matching" abilities (which are beyond the scope of this book). They're potent tools in Python. If you're uncomfortable with them, play with the ones we presented above

and watch how they respond to your changes. Test their boundaries and learn their capabilities. Curiosity will serve you well in your journey towards mastering Python. There is much to discover!

Ternary Expressions

We wanted to briefly show you something we call ternary expressions. The official name is conditional expression, but that's confusing since something like a == b is also a conditional expression. It is also sometimes called a ternary operator. However, it's not an operator.

A ternary expression is a concise way to choose between two values based on some condition. They are often used as an expression on the right side of an assignment, as function arguments, and as function return values.

Ternary expressions have the following structure:

Copy Code

```
value1 if condition else value2
```

Python first evaluates the condition. If it's truthy, the expression returns value1. Otherwise, it returns value2. Note that only one of value1 and value2 will be evaluated.

Consider the following code:

Copy Code

```
if shape.sides() == 3:
    print("Triangle")
else:
    print("Square")
```

That's easy enough to understand, but it is a bit wordy. We can eliminate the wordiness at the sacrifice of a little clarity:

Copy Code

```
print("Triangle" if shape.sides() == 3 else "Square")
```

Ternaries work particularly well when you need a way to handle missing or invalid data in output:

Copy Code

```
print('N/A' if value == None else value)
```

Should you use ternary expressions in your code? We recommend using them only when it doesn't sacrifice too much clarity. Don't use them as a substitute for every 4-line if/else statement or as a way to save keystrokes: they work best as expressions.

Remember that many Python programmers dislike ternary expressions since they are hard to read. If you decide that you like ternary expressions, that's fine. However, use them judiciously. In particular, ternaries should almost always be extremely simple and fit entirely on one 79-column line of code.

Introduction to Collections

Collection Types

Python has a surprisingly large number of built-in and auxiliary collection types. Collections are objects that contain zero or more member objects, often called elements. There are 3 main categories of collection: sequences, mappings, and sets.

Let's recap the types table from the Data Types chapter. We've reproduced a portion of the table below:

Type	Class	Category	Kind	Mutable
ranges	range	sequences	Non-primitive	No
tuples	tuple	sequences	Non-primitive	No
lists	list	sequences	Non-primitive	Yes
dictionaries	dict	mappings	Non-primitive	Yes

sets set sets Non-primitive Yes
frozen sets frozenset sets Non-primitive No
As you can see, the collections fall into several categories: sequences, mappings, and sets.

We often also include text sequences when talking about collections:

Type Class Category Kind Mutable
strings str text sequences Primitive No
The primary text sequence of interest is strings. Strings and other text sequences are not collections, however. They are sufficiently similar that we can treat them as sequence collections, but it's important to remember that they are not. For the purposes of this chapter, we will discuss text sequences as though they were collections.

Some collections are mutable; others are immutable. Keeping them straight takes some practice; be patient. In particular, lists and tuples differ only in that lists are mutable; tuples are not. The set collections differ similarly: sets are mutable; frozen sets are not.

We've met all these types earlier in this book. In this lesson, we'll explore them in depth.

What are Sequences?
Sequences are types that maintain an ordered collection of objects (also: elements or values) that can be indexed by whole numbers. Ordered collections are collections organized in some sequence: a first element, a second element, a third element, and so on. Indexed sequences associate every object member with a whole number (0, 1, 2, etc.) that can be used to access or modify that object. The built-in sequences (ranges, lists, and tuples) always have the first object at index 0, the second at index 1, and so on.

Copy Code
```
letters = ['a', 'b', 'c']
print(letters[0])         # a (first element)
print(letters[1])         # b (second element)
print(letters[2])         # c (third element)
```
Lists and tuples are heterogeneous; they may contain different kinds of objects, including other sequences:

Copy Code
```
my_list = [
    'May',
    2.99,
    {None, True, False},
    [1, 2, 3],
]
```
Ranges are homogenous; they always contain integers.

Strings are a form of sequence called a text sequence. They differ from ordinary sequences in several ways:

Strings are homogenous; all characters in a string are, um, characters.
Characters are not a distinct kind of object; they are merely strings of length 1.
A string's individual characters are not separate strings until you reference a character.
Strings are not actual collections since the characters inside the string aren't objects.
Bullet 3 merits an example, but it requires using Unicode. The Greek letter theta (Î¸) is a fine choice. Let's look at two snippets, one using a list and the other a string:

Example 1Copy Code

```
letters = ['a', 'b', 'Î¸', 'd', 'e']
char = letters[2]
print(char is letters[2])     # True
```
Example 2Copy Code
```
letters = 'abÎ¸de'
char = letters[2]
print(char is letters[2])     # False
```
Example 1 shows that letters[2] returns the object at index 2 of the letters list. Both char and letters[2] reference the same object, as shown on line 3.

However, in Example 2, the two characters are not the same object. They have the same value but are distinct objects.

We used Î¸ to avoid interning, a topic we discussed earlier. Python would have interned any character from the extended ASCII character set, and both snippets would have returned True. By switching to a non-ASCII character, we could show that characters in a string aren't objects.

While there are differences, we can usually treat strings as ordinary sequences.

What are Sets?
Sets are types that maintain an unordered collection of unique objects (also called elements or members). Unlike sequences, sets cannot be indexed. Unordered means no well-defined order exists for the objects in a set. In fact, the sets {1, 2, 3} and {3, 1, 2} are equal since the order doesn't matter. By unique, we mean a set can not have duplicate members.

Python has two main built-in set types: sets and frozen sets. Regular sets are mutable; frozen sets are immutable. This is the only significant difference between the two:

Copy Code
```
letters = {'a', 'b', 'c'}
letters.add('d')
print(letters)
# {'a', 'b', 'c', 'd'} (order may differ)

frozen_letters = frozenset(letters)
frozen_letters.add('e')
# AttributeError: 'frozenset' object has no
# attribute 'add'
```
Sets and frozen sets, like Lists and tuples, are heterogeneous; they may contain different kinds of objects, including other hashable collections:

Copy Code
```
my_set = {
    'May',
    2.99,
    (None, True, False),
    range(5),
}
```
Python will ignore any attempt to add duplicate members to a set:

Copy Code
```
letters = {'a', 'b', 'c', 'b', 'a'}
print(letters)
# {'a', 'c', 'b'} (order may differ)

letters.add('c')
print(letters)
# {'a', 'c', 'b'} (order may differ)
```
Since Python 3.7, sets of integers sometimes seem to be ordered. For instance:

Copy Code

```
numbers = { 1, 2, 3, 4, 5 }
print(numbers)      # { 1, 2, 3, 4, 5 }

numbers = { 5, 3, 1, 2, 4 }
print(numbers)      # { 1, 2, 3, 4, 5 }
```
No matter how often you run that code, it prints both sets with the same ordered values. It's an illusion, however, produced by an implementation detail. The behavior isn't guaranteed: it's easy to construct sets of integers that are clearly unordered:

Copy Code
```
numbers = { 1, 2, 37, 4, 5 }
print(numbers)      # {1, 2, 4, 37, 5}
```
The order of the numbers you see may not match ours, but you will likely see that the numbers are no longer predictably ordered.

What are Mappings?
Mappings are types that maintain an unordered collection of key/value pairs (also called elements or members). Unlike sequences, mappings are accessed by their keys, which usually are not numbers. While Python has several mapping types, the only one we'll meet in this book is the dictionary or dict type.

Dicts have the following characteristics:

Dicts are mutable.
The keys in a dict must be unique. Trying to add a duplicate key to a dict merely replaces the existing key's associated value.
Keys must be "hashable" values. We won't define "hashable" right now. However, hashable values are usually (not always) immutable. All built-in immutable types (numbers, strings, booleans, tuples, frozen sets, and None) are hashable and can be dict keys.
The values in each key/value pair may be any object.
Copy Code
```
d = {'a': 1, (1, 3): 3, 1: 'x'}
print(d)         # {'a': 1, (1, 3): 3, 1: 'x'}
print(d['a'])    # 1
print(d[(1, 3)]) # 3
print(d[1])      # 'x'

d['a'] = 'A'
print(d)        # {'a': 'A', (1, 3): 3, 1: 'x'}
```
Since version 3.7, Python dicts maintain the insertion order of the pairs. Python guarantees it. Thus, you can iterate over the pairs in the same order they were inserted into the dictionary. However, this behavior doesn't fundamentally alter whether dicts are unordered collections. In practice, you should mentally think of dicts as unordered collections, but it's OK to rely on the ordering in contexts where the insertion order matters.

Copy Code
```
d = {'a': 1, (1, 3): 3, 1: 'x'}

del d['a']      # Deletes key/value pair 'a': 1
print(d)        # {(1, 3): 3, 1: 'x'}

d['a'] = 42
print(d)        # {(1, 3): 3, 1: 'x', 'a': 42}
```

Sequence Constructors
Most built-in Python data types let the programmer create new objects using literal values. Literals are great. However, you can also use special functions called constructors to create new objects. In fact, sometimes you can't use literals; you must use constructors to create ranges, frozen sets, and empty sets.

Why do you think we must use a constructor function to create an empty set?

Click here to ask LSBot this question
Type your answer to the question above... (Press Cmd/Ctrl+Enter to send)
Check My Answer
Let's examine the constructors used with collections.

String Constructor
The string constructor, str, has two basic formats, str() and str(something), where something is any Python object. Regardless of what you pass to str, it returns a string. For instance:

Copy Code
```
str()            # returns '' (empty string)
str('abc')       # returns 'abc'
str(42)          # returns '42'
str(3.141592)    # returns '3.141592'
str(False)       # returns 'False'
str(None)        # returns 'None'
str(range(3, 7)) # returns 'range(3, 7)'
str([1, 2, 3])   # returns '[1, 2, 3]'
str(int)         # returns "<class 'int'>"
str(list)        # returns "<class 'list'>"

class Person: pass
str(Person())
# returns "<__main__.Person object at 0x1006d21e0>"
```
In most cases, the values returned by str give a good idea of the original value. However, the last 6 lines show that the return value isn't always helpful. This is especially true with custom types like the Person class above (we'll learn about custom types in the OO Python book). You can fix this issue with custom types, but we won't discuss that until later in the Core Curriculum.

Range Constructor
The range constructor comes in 3 forms:

```
range(start, stop, step)
```

This constructor generates a sequence of integers between start and stop - 1 with an increment of step between each consecutive integer. You can use a negative step to generate a sequence in reverse order. In this case, the range ends at stop + 1. For instance:

Copy Code
```
r = range(5, 12, 2)
print(list(r))            # [5, 7, 9, 11]

r = range(12, 8, -1)
print(list(r))            # [12, 11, 10, 9]

r = range(12, 5, -2)
print(list(r))            # [12, 10, 8, 6]
```
You can create empty ranges by giving values where start >= stop when step is positive or start <= stop when step is negative. Empty ranges are often bugs.

Copy Code
```
r = range(5, 5, 1)
print(list(r))          # []

r = range(5, 7, -1)
print(list(r))            # []
range(start, stop)
```

When you omit the step argument, Python uses a default value of 1. Hence,

range(start, stop) is identical to range(start, stop, 1).

range(stop)

When you omit the start argument, Python uses a default value of 0 for start. Hence, range(stop) is identical to range(0, stop, 1).

Ranges are lazy sequences: they don't create any element values until your program needs them. This is why we use list(r) above; without it, print(r) prints something like range(5, 12, 2), which may not be helpful. You can also use the tuple constructor to do the same thing. The set and frozenset constructors also work, but the sets may not be in the same order.

The List, Tuple, Set, and Frozen Set Constructors
Lists, tuples, sets, and frozen sets share two common constructor forms:

list() or list(iterable)
tuple() or tuple(iterable)
set() or set(iterable)
frozenset() or frozenset(iterable)
The constructors with no arguments create an empty list, tuple, set, or frozen set, as appropriate: a sequence or set with no members.

The constructors that take an iterable argument expect an object that Python can iterate: an iterable. From the built-in types, the iterables include all sequences, strings, sets, and mappings. (Note that iterating a mapping iterates the mapping's keys.) Thus, you can use these constructors to convert lists to tuples, tuples to sets, strings to lists, and so on:

Copy Code
```
my_str = 'Python'

my_list = list(my_str)
print(my_list)  # ['P', 'y', 't', 'h', 'o', 'n']

my_tuple = tuple(my_list)
print(my_tuple) # ('P', 'y', 't', 'h', 'o', 'n')

my_set = set(my_tuple)
print(my_set)   # {'t', 'o', 'n', 'h', 'P', 'y'}
```
You can also use these constructors to duplicate a collection:

Copy Code
```
my_list = [5, 12, 2]
another_list = list(my_list)

print(my_list)                          # [5, 12, 2]
print(another_list)                     # [5, 12, 2]

print(my_list == another_list)          # True
print(my_list is another_list)          # False
```
As you can see in lines 4, 5, and 7, the lists are identical. However, line 8 shows that the list objects are not the same object.

Let's look at a more complex example involving nested lists:

Copy Code
```
my_list = [[1, 2, 3], [4, 5, 6]]
another_list = list(my_list)

print(my_list)                          # [[1, 2, 3], [4, 5, 6]]
print(another_list)                     # [[1, 2, 3], [4, 5, 6]]

print(my_list == another_list)          # True
```

```
print(my_list is another_list)          # False
print(my_list[0] is another_list[0])    # True
print(my_list[1] is another_list[1])    # True
```
In this case, we can see from lines 4, 5, and 7 that the lists have the same values, and from line 8, we can see that they are different objects. Strangely, in lines 9 and 10, we can see that their elements are the same objects. That's because the list constructor performs what is known as a shallow copy. In a later chapter, we'll discuss shallow and deep copies and what is happening here. For now, remember that nested collections are subject to shallow copies. That's often fine. However, it can lead to problems that are difficult to diagnose.

Passing a string to any of these constructors causes it to iterate over the characters in the string. It places each character in a separate element of the resulting collection:

Copy Code
```
print(tuple('Python'))
# ('P', 'y', 't', 'h', 'o', 'n')
```

Using Collections

In this chapter, we'll learn how to work with Python's collections. In particular, we'll explore indexing and key-based access, then explore some of the many operations most collections have in common.

Indexing

Indexing is the process of using a whole number to access and perhaps alter an element of a sequence. All sequences, including strings, support indexing. Python uses index 0 as the first element of all built-in sequences. To access the first element of a sequence assigned to variable seq, we use seq[0]. We use seq[1] to access the second element. This process repeats until we exhaust the sequence:

Copy Code
```
seq = ('a', 'b', 'c')
print(seq[0])  # a (1st element)
print(seq[1])  # b (2nd element)
print(seq[2])  # c (3rd element)
print(seq[3])  # IndexError: tuple index out of range
```
Pictorially, the indexing looks like this:

Indexing

In the above example, we used a tuple as our sequence. However, we could have used any other sequence, such as a list or string. It's worth noting that the index of the final element is one less than the sequence's length.

An error occurred when accessing index 3. The last element in a 3-element sequence has index 2, not 3. The len function can determine a sequence's length. You can use its return value to determine whether an index is out of range:

Copy Code
```
seq = ('a', 'b', 'c')
if len(seq) > 3:
    print(seq[3])
```
Note that we tested whether the length was greater than the intended index. If you want to access the element at index 3, the sequence must have at least four elements.

Suppose we want to access the last element in a sequence? To do that, we can compute the index of the last element and then use that value:

Copy Code
```
seq = ('a', 'b', 'c')
last_index = len(seq) - 1
```

```
print(seq[last_index])        # c
```
You can also use negative indexes, which are often easier to work with:

Copy Code
```
seq = ('a', 'b', 'c')
print(seq[-1])  # c (last element)
print(seq[-2])  # b (next to last element)
print(seq[-3])  # a (2nd to last element)
```
Pictorially, negative indexing looks like this:

Negative Indexing

If you want to change the value of an element in a mutable sequence (i.e., a list), you can use indexing on the left side of an assignment:

Copy Code
```
seq = ['a', 'b', 'c']
seq[1] = 'B'
print(seq)      # ['a', 'B', 'c']
```
This operation mutates the list but merely reassigns seq[1].

Slicing
The indexing syntax also supports a slicing augmentation. Slicing can extract (or modify) any number of consecutive elements simultaneously. For instance, the syntax seq[start:stop] retrieves the elements from seq whose index is between start and stop - 1, inclusive. You can also use negative indexes for the slice. Finally, you can use the seq[start:stop:step] syntax to slice every "step-th" element.

Copy Code
```
string = 'abcdefghi'
print(string[3:7])       # defg
print(string[-6:-2])     # defg
print(string[2:8:2])     # ceg
print(repr(string[3:3])) # ''
print(string[:])         # abcdefghi
print(string[::-1])      # ihgfedcba

seq = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print(seq[3:7])          # [4, 5, 6, 7]
print(seq[-6:-2])        # [5, 6, 7, 8]
print(seq[2:8:2])        # [3, 5, 7]
print(seq[3:3])          # []
print(seq[:])            # [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print(seq[::-1])         # [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]

seq = [[1, 2], [3, 4]]
seq_dup = seq[:]
print(seq[0] is seq_dup[0])   # True
```
Line 5 shows that you get an empty slice when the start and stop values are the same.

Line 6 returns a duplicate of the string. It is equivalent to string[0:len(string)]. This syntax creates a shallow copy, which we'll discuss later.

Line 7 returns a reversed copy of a string. Unlike seq[::-1] (described below), there is no string.reverse function.

Lines 9-15 demonstrate that slicing also works with other sequences (a list in this case).

Note that line 15 returns a reversed copy of a sequence, similar to the way that string[::-1] returns a reversed copy of a string. Unlike strings, however,

sequences also have a seq.reverse() method that we'll describe later. seq[::-1] returns a new sequence; seq.reverse() mutates the original. seq.reverse() is much easier to read, but the mutation should be intentional.

Lines 17-19 demonstrate that slicing performs a shallow copy if the sequence contains any collections, such as lists or tuples. Custom objects, which we'll meet in our Object Oriented Programming with Python book, are also subject to shallow copying.

Slicing also works as an assignment's target (the left side of an =). However, we'll refrain from demonstrating that as it often leads to code that is hard to read.

Key-Based Access

Indexing uses whole numbers and only works with sequences and strings. However, mappings use a syntax called key-based access that looks like indexing. However, keys are usually strings, though not always.

Instead of limiting ourselves to whole numbers, we can use any hashable object as a key, which includes the built-in immutable types. We sometimes use integer keys, which, confusingly, look like indexing. Other types are rarely used but can be helpful (especially tuples).

With dicts, we use key-based access like this:

Copy Code

```
my_dict = {
    'a': 'abc',
    37: 'def',
    (5, 6, 7): 'ghi',
    frozenset([1, 2]): 'jkl',
}

print(my_dict['a'])               # abc
print(my_dict[37])                # def
print(my_dict[(5, 6, 7)])         # ghi
print(my_dict[frozenset([1, 2])])  # jkl
print(my_dict['nothing'])     # KeyError: 'nothing'
```

We've used a string, an integer, a tuple, and a frozen set as keys in this dict. We've also seen that we get a KeyError if we try to use a non-existent key. If there's a chance you might get a KeyError, consider using the dict.get method. It returns the value associated with a given key if the key exists. Otherwise, it produces a default return value (usually None, but other values can be specified):

Copy Code

```
my_dict = {
    'a': 'abc',
    37: 'def',
    (5, 6, 7): 'ghi',
    frozenset([1, 2]): 'jkl',
}

print(my_dict.get('a'))                # abc
print(my_dict.get('nothing'))          # None
print(my_dict.get('nothing', 'N/A'))    # N/A
print(my_dict.get('nothing', 100))      # 100
```

We can also use key-based access to the left of the = operator:

Copy Code

```
my_dict = {
    'a': 'abc',
    37: 'def',
    (5, 6, 7): 'ghi',
```

```
    frozenset([1, 2]): 'jkl',
}

my_dict['a'] = 'ABC'
my_dict[37] = 'DEF'
my_dict[(5, 6, 7)] = 'GHI'
my_dict[frozenset([1, 2])] = 'JKL'
print(my_dict)
# Pretty printed for clarity
# {
#     'a': 'ABC',
#     37: 'DEF',
#     (5, 6, 7): 'GHI',
#     frozenset({1, 2}): 'JKL'
# }
```

Can we assign new keys to a dict?

Copy Code

```
my_dict['xyz'] = 'Hey there!'
print(my_dict['xyz'])         # Hey there!
```

By all means, we can!

Can we use a mutable key?

Copy Code

```
my_dict[[1, 2, 3]] = 'Hey there!'
# TypeError: unhashable type: 'list'
```

Nope. It doesn't work. Dictionary keys must be immutable.

Common Collection Operations

Most Python collections support various operations, functions, and methods, some of which only apply to mutable collections. We'll examine the non-mutating operations first. We won't cover everything you can do. However, we'll see most of what you will encounter at Launch School.

Non-Mutating Operations for Collections

All of the operations in this section work equally well with mutable and immutable collections. They never modify the original collection. Instead, they return new objects.

Collection Membership

The `in` operator determines whether the object to the operator's left is in the iterable collection on the right. It returns `True` if the item on the left is in the collection on the right, `False` otherwise.

The `not in` operator is the inverse of `in`. It returns `False` if the object is in the collection; `True` if not.

With sequences and sets, these operators compare the object for equality against each collection element. For mappings (dicts), it checks whether the item is a key in the dictionary. For strings, it determines whether the right string contains the left string.

Copy Code

```
seq = [4, 'abcdef', (True, False, None)]
print(4 in seq)                         # True
print(4 not in seq)                     # False
print('abcdef' in seq)                  # True
print('abcdef' not in seq)              # False
print('cde' in seq[1])                  # True
print('cde' not in seq[1])              # False
print('acde' in seq[1])                 # False
print('acde' not in seq[1])             # True
print((True, False, None) in seq)       # True
```

```
print((True, False, None) not in seq)   # False
print(3.14 in seq)                      # False
print(3.15 not in seq)                  # True
```

Common Collection Operations
Most Python collections support various operations, functions, and methods, some of which only apply to mutable collections. We'll examine the non-mutating operations first. We won't cover everything you can do. However, we'll see most of what you will encounter at Launch School.

Non-Mutating Operations for Collections
All of the operations in this section work equally well with mutable and immutable collections. They never modify the original collection. Instead, they return new objects.

Collection Membership
The in operator determines whether the object to the operator's left is in the iterable collection on the right. It returns True if the item on the left is in the collection on the right, False otherwise.

The not in operator is the inverse of in. It returns False if the object is in the collection; True if not.

With sequences and sets, these operators compare the object for equality against each collection element. For mappings (dicts), it checks whether the item is a key in the dictionary. For strings, it determines whether the right string contains the left string.

Copy Code
```
seq = [4, 'abcdef', (True, False, None)]
print(4 in seq)                         # True
print(4 not in seq)                     # False
print('abcdef' in seq)                  # True
print('abcdef' not in seq)              # False
print('cde' in seq[1])                  # True
print('cde' not in seq[1])              # False
print('acde' in seq[1])                 # False
print('acde' not in seq[1])             # True
print((True, False, None) in seq)       # True
print((True, False, None) not in seq)   # False
print(3.14 in seq)                      # False
print(3.15 not in seq)                  # True
```

Minimum and Maximum Members
min and max return the minimum and maximum members in an iterable collection. The only requirement is that any pair of the collection's elements are comparable with the < and > operators.

Copy Code
```
my_set1 = {1, 4, -9, 16, 25, -36, -63, -1}
my_set2 = {'1', '4', '-9', '16', '25', '-36', '-1'}

print(min(my_set1), max(my_set1))     # -63 25
print(min(my_set2), max(my_set2))     # -1 4
```
As you can see, min and max know how to compare the members of our sets. It determines the type of comparison by looking at the element types.

In most cases, you can't use min and max with heterogenous collections:

```
>>> my_set = {1, 4, '-9', 16, '25', -36, -63, -1}
>>> min(my_set)
TypeError: '<' not supported between instances of
'str' and 'int'
```

```
>>> max(my_set)
TypeError: '>' not supported between instances of
'str' and 'int'
```
However, it is possible in some situations:

Copy Code
```
my_set = {1, 3.14, -2.71}
print(min(my_set), max(my_set))      # -2.71 3.14
```
You can also use min and max with multiple arguments instead of an iterable.

Copy Code
```
print(min(3, 5, -1), max(3, 5, -1))  # -1 5
```
Summation
The sum function is used in conjunction with iterable collections that consist entirely of numeric values. It computes and returns the sum of all the collection's numbers.

Copy Code
```
numbers = (1, 1, 2, 3, 5, 8, 13, 21, 34)
print(sum(numbers))                         # 88
```
Despite what Python's official documentation says, sum cannot be used with strings. It only works with numeric types. Use str.join if you want to concatenate strings

Locating Indices and Counting
Two helpful sequence methods are the index and count methods. seq.index returns the index of the first element in the sequence that matches a given object. It raises a ValueError exception if the object is not found. seq.count returns the number of times a value occurs in the sequence.

Copy Code
```
names = ['Karl', 'Grace', 'Clare', 'Victor',
         'Antonina', 'Allison', 'Trevor']
print(names.index('Clare'))   # 2
print(names.index('Trevor'))  # 6
print(names.index('Chris'))
# ValueError: 'Chris' is not in list
```
Copy Code
```
numbers = [1, 3, 6, 5, 4, 10, 1, 5, 4, 4, 5, 4]
print(numbers.count(1))       # 2
print(numbers.count(3))       # 1
print(numbers.count(4))       # 4
print(numbers.count(7))       # 0
```
index also works with strings. It searches for the first matching substring of a string:

Copy Code
```
names = 'Karl Grace Clare Victor Antonina Trevor'
print(names.index('Clare'))   # 11
print(names.index('Trevor'))  # 33
print(names.index('Chris'))
# ValueError: substring not found
```

Merging Collections
One of the most impressively helpful functions is zip, which works with all iterables. It lets you merge the members of multiple iterables into a single list of tuples. zip makes it easy to iterate through many collections simultaneously.

zip iterates through 0 or more iterables in parallel and returns a list-like object of tuples. Each tuple contains a single object from each of the iterables. That's a mouthful, so let's take a look at an example:

Copy Code
```
iterable1 = [1, 2, 3]
iterable2 = ('Kim', 'Leslie', 'Bertie')
iterable3 = [None, True, False]

zipped_iterables = zip(iterable1, iterable2, iterable3)
print(list(zipped_iterables))
# Pretty printed for clarity
# [
#   (1, 'Kim', None),
#   (2, 'Leslie', True),
#   (3, 'Bertie', False)
# ]
```
Here, we've combined 3 iterables (two lists and a tuple) into a list-like object of tuples. Each tuple in the return value has 3 members. The first member is from iterable1, the second from iterable2, and the third from iterable3. The first tuple contains the first element of each of the iterables, the second tuple contains the second element of the iterables, and the third tuple contains the third element. Thus, our first tuple contains the first elements of iterable1 (1), iterable2 ('Kim'), and iterable3 (None).

Note that we referred to zips return value as a list-like object. It's not a true list, but a lazy sequence much the same as a range. You must request values explicitly, which you can do with a loop or iterable constructor. That's why we call list(zipped_iterables) on line 6 above.

zip's collection arguments are usually the same length but don't have to be. If you want to enforce identical lengths, add a strict=True keyword argument to the invocation:

Copy Code
```
zipped_iterables = zip(iterable1, iterable2, strict=True)
```
With strict=True, zip raises an exception if the iterables don't all have the same length.

So, what happens if the lengths differ and strict=True isn't given? In this case, zip stops after exhausting the shortest iterable:

Copy Code
```
result = list(zip('12345',
             ['chicken', 'banana', 'apple'],
             {True, False}))

print(result)
# [('1', 'chicken', False), ('2', 'banana', True)]
```
Since {True, False} only has a length of 2 and strict=True was omitted, the result list only has 2 elements.

The zip function's canonical application is to simultaneously iterate over multiple collections. We'll demonstrate that in the next chapter.

It's worth noting that zip returns what is known as an iterator. We'll discuss iterators in more detail in the Core curriculum. However, one characteristic of iterators that is important to be aware of is that they can only be consumed once. If you iterate over the iterator object, subsequent attempts to iterate will fail. For instance:

Copy Code
```
result = zip(range(5, 10),    # length is 5
             range(1, 3),     # length is 2 (shortest)
             range(3, 7))     # length is 4
print(list(result)) # [(5, 1, 3), (6, 2, 4)]
print(list(result)) # []
```
As you can see, once we've consumed the return value of zip, we can't do it

again. Most Python functions and methods that return lazy sequences may only be consumed once; range objects are one of the few exceptions to this rule.

Operations on Dictionaries
Python provides 3 methods to get lists of the keys, values, and key/value pairs from a dictionary. Those methods are dict.keys, dict.values, and dict.items. Let's see them in action:

Copy Code
```
people_phones = {
    'Chris': '111-2222',
    'Pete':  '333-4444',
    'Clare': '555-6666',
}

print(people_phones.keys())
# dict_keys(['Chris', 'Pete', 'Clare'])

print(people_phones.values())
# Pretty printed for clarity
# dict_values([
#     '111-2222',
#     '333-4444',
#     '555-6666'
# ])

print(people_phones.items())
# Pretty printed for clarity
# dict_items([
#     ('Chris', '111-2222'),
#     ('Pete',  '333-4444'),
#     ('Clare', '555-6666')
# ])
```
The lists produced by these methods aren't ordinary lists. Python wraps the output for each list with dict_keys(), dict_values(), or dict_items() to show that these aren't regular lists. They are actually dictionary view objects that are tied to the dictionary. If you add a new key/value pair to the dictionary, remove an element, or update a value, the corresponding lists are updated immediately:

Copy Code
```
people_phones = {
    'Chris': '111-2222',
    'Pete':  '333-4444',
    'Clare': '555-6666',
}

keys = people_phones.keys()
values = people_phones.values()

print(keys)    # dict_keys(['Chris', 'Pete', 'Clare'])
print(values)
# dict_values(['111-2222', '333-4444', '555-6666'])

people_phones['Max'] = '123-4567'
people_phones['Pete'] = '345-6789'
del people_phones['Chris']

print(keys)    # dict_keys(['Pete', 'Clare', 'Max'])
print(values)
# dict_values(['345-6789', '555-6666', '123-4567'])
```

Operations for Mutable Sequences
Let's see some of the mutating operations we can use with mutable sequences, the

most common of which are lists. All methods in this section mutate the collection used to call them.

Keep in mind that all these methods work with any properly defined mutable sequence, such as deques and arrays, both of which are available for Python.

Adding Elements to Mutable Sequences
We can use the append, insert, and extend methods to add new elements to a mutable sequence, such as a list.

seq.append appends a single object to the end of a mutable sequence, such as a list:

Copy Code
```
numbers = [1, 2]

numbers.append(10)      # Append the number 10
print(numbers)          # [1, 2, 10]
```
seq.insert inserts an object into a mutable sequence before the element at a given index. If the given index is greater than or equal to the sequence's length, the object is appended to the sequence. If the index is negative, Python counts from the end of the sequence.

Copy Code
```
numbers = [1, 2]

numbers.insert(0, 8)    # Insert 8 before numbers[0]
print(numbers)          # [8, 1, 2]
numbers.insert(2, 6)    # Insert 6 before numbers[2]
print(numbers)          # [8, 1, 6, 2]
numbers.insert(100, 55) # Insert 55 before numbers[100]
print(numbers)          # [8, 1, 6, 2, 55]
numbers.insert(-3, 33)  # Insert 33 before the 3rd element
                        # from the end.
print(numbers)          # [8, 1, 33, 6, 2, 55]
```
seq.extend appends the contents of an iterable sequence to the calling iterable sequence.

Copy Code
```
numbers = [1, 2]

numbers.extend([7, 8])  # Append 7 and 8 to numbers
print(numbers)          # [1, 2, 7, 8]
```
Removing Elements from Mutable Sequences
We can use the remove, pop, and clear methods to remove elements from a mutable sequence:

seq.remove searches a sequence for a specific object and removes the first occurrence of that object. It raises a ValueError if there is no such object.

Copy Code
```
my_list = [2, 4, 6, 8, 10]

my_list.remove(8)
print(my_list)            # [2, 4, 6, 10]

my_list.remove(8)
# ValueError: list.remove(x): x not in list
```
seq.pop removes and returns an indexed element from a mutable sequence. If no index is given, it removes the last element in the sequence. It raises an error if the index is out of range. pop only works with mutable indexed sequences.

Copy Code

```
my_list = [2, 4, 6, 8, 10]

print(my_list.pop(1))         # 4
print(my_list)                # [2, 6, 8, 10]

print(my_list.pop())          # 10
print(my_list)                # [2, 6, 8]

print(my_list.pop(4))
# IndexError: pop index out of range
```

seq.clear removes all elements from a sequence, leaving it empty.

Copy Code

```
my_list = [2, 4, 6, 8, 10]

my_list.clear()
print(my_list)                # []
```

Sorting Collections

You can use the sorted function to create a sorted list from any iterable collection, mutable or immutable. It creates and returns a sorted list from the elements in the collection. The original collection is unchanged.

You can also sort lists using the list.sort method. However, this method mutates the list. It's also worth noting that my_list.sort() is a bit faster and less memory intensive than sorted(my_list) since the method does an in-place sort, so doesn't have to build a completely new list.

Copy Code

```
names = ('Grace', 'Clare', 'Allison', 'Trevor')
print(sorted(names))
# ['Allison', 'Clare', 'Grace', 'Trevor']

print(names)
# ('Grace', 'Clare', 'Allison', 'Trevor')

names = list(names)
print(names)
# ['Grace', 'Clare', 'Allison', 'Trevor']

print(names.sort())   # None
print(names)
# ['Allison', 'Clare', 'Grace', 'Trevor']
```

In the sorted example, we've sorted and printed the tuple of names. Note that the result is a list, not a tuple. We've then shown that the original names tuple is unchanged.

Next, we converted the names tuple to a list and sorted it with names.sort. Note that sort returned None, which strongly hints that the collection was mutated. When we print the names, we see that the list was mutated.

By default, both sort and sorted do an ascending sort using the < operator to compare elements from the collection. You can reverse the sort by adding a reverse=True keyword argument to the argument list:

Copy Code

```
names = ['Grace', 'Clare', 'Allison', 'Trevor']
print(sorted(names, reverse=True))
# ['Trevor', 'Grace', 'Clare', 'Allison']

names.sort(reverse=True)
print(names) # ['Trevor', 'Grace', 'Clare', 'Allison']
```

You can also pass a key=func keyword argument to tell sort or sorted how to determine what values it should sort. For instance, if you want to perform a case-insensitive sort on a list of strings, you can specify key=str.casefold:

Copy Code
```
words = ['abc', 'DEF', 'xyz', '123']
print(sorted(words))
# ['123', 'DEF', 'abc', 'xyz']

print(sorted(words, key=str.casefold))
# ['123', 'abc', 'DEF', 'xyz']
```
In most cases, you can also use str.lower instead of str.casefold. However, using str.casefold is considered best-practice since sort will be comparing the strings.

You can also sort a list of numeric-valued strings by passing key=int to the function or method:

Copy Code
```
numbers = ['1', '5', '100', '15', '534', '53']
numbers.sort()
print(numbers)   # ['1', '100', '15', '5', '53', '534']

numbers.sort(key=int)
print(numbers)   # ['1', '5', '15', '53', '100', '534']
```
This is probably your first time seeing that Python's functions and methods are objects that can be passed around as arguments to a function. (They can also be passed around as return values.) Remember this moment. The technique is potent.

If you use sorted on a dictionary, it returns a sorted list of the dictionary's keys.

Reversing Sequences and Dictionaries
You can use the reversed function to reverse the order of elements in a sequence or dictionary. The returned value is a lazy sequence that contains the elements in the sequence or the keys from a dictionary. Since the result is lazy, you need to iterate over the result or expand it with a function list list or tuple.

You can also reverse lists using the list.reverse method. However, this method mutates the list. In general, you should use list.reverse when you really need to reverse the list's contents, and don't need to preserve the original order. You should use reversed when all you need to do is iterate over the list in reverse. Don't use reversed if you eventually want to convert the result to a non-lazy sequence such as a list or tuple:

Copy Code
```
names = ('Grace', 'Clare', 'Allison', 'Trevor')
reversed_names = reversed(names)
print(reversed_names)
# <reversed object at 0x102848e50>
print(tuple(reversed(names))) # Requires extra memory
# ('Trevor', 'Allison', 'Clare', 'Grace')
print(names)
# ('Grace', 'Clare', 'Allison', 'Trevor')

names = list(names)
print(names.reverse())   # None
print(names)
# ['Trevor', 'Allison', 'Clare', 'Grace']

my_dict = {'abc': 1, 'xyz': 23, 'pqr': 0, 'jkl': 5}
reversed_dict = reversed(my_dict)
print(reversed_dict)
# <dict_reversekeyiterator object at 0x100d19f80>

print(list(reversed_dict))    # Requires extra memory
# ['jkl', 'pqr', 'xyz', 'abc']
```

In the reversed example using a tuple, we've reversed and printed the tuple of names. We've then shown that the original names tuple is unchanged.

Next, we converted the names tuple to a list and reversed it with names.reverse. Note that reverse returned None, which strongly hints that the collection was mutated. When we print the names, we see that the list was mutated.

Finally, we demonstrated using reversed with a dictionary.

Think of the reversed function as a looping aid. You sometimes want to iterate over a collection in reverse. reversed makes that easy:

Copy Code
```
names = ('Grace', 'Clare', 'Allison', 'Trevor')
for name in reversed(names):
    print(name)
# Trevor
# Allison
# Clare
# Grace
```
String Operations
As we've seen, Python has many built-in functions for manipulating sequences and other collections. You can use many of these functions with strings, even though strings are not proper collections or sequences.

The str class also offers a veritable goldmine of methods for using and manipulating strings in myriad ways. Let's take a look at some of these. We cannot cover them all, but we'll look at some of the most useful. We'll skip over anything we've already discussed in any detail.

Letter Case
We've already seen how the str.lower and str.upper methods work. Let's check out two related methods:

str.capitalize returns a copy of the string with the first character capitalized and the remaining characters converted to lowercase.

Copy Code
```
print("what's up?".capitalize())        # What's up?
print('456ABC'.capitalize())            # 456abc
```
str.title returns a copy of the string with the first character of every word in the string capitalized. The remaining characters are converted to lowercase.

Copy Code
```
print("four SCORE and sEvEn".title())
# Four Score And Seven
```
str.title's idea of what constitutes a word can lead to unexpected results. In particular, it uses whitespace and certain punctuation characters as word boundaries:

Copy Code
```
print("i can't believe it's already mid-july.".title())
# I Can'T Believe It'S Already Mid-July.
```
If you only want to break at whitespace, you can use the capwords function from the string module:

Copy Code
```
import string
print(string.capwords("i can't believe it's already mid-july."))
# I Can't Believe It's Already Mid-july.
```
str.swapcase returns a copy of the string with every uppercase letter converted to lowercase, and vice versa.

Copy Code

```
print("What's up?".swapcase())          # wHAT'S UP?
print('456ABC'.swapcase())              # 456abc
print('456ABC'.swapcase().swapcase())   # 456ABC
```
The first call to swapcase on line 3 returns the original string with the case swapped; the second swaps the case on that return value.

Note that there are situations where str.swapcase().swapcase() does not return the original value of the calling string. For instance:

Copy Code
```
print('StraÃŸe'.swapcase().swapcase())   # Strasse
```
In this case, the German eszet character (ÃŸ), which represents a doubled lowercase s (ss), does not have an uppercase counterpart. Thus, the above example prints Strasse instead of StraÃŸe.

Character Classification
Python has a generous suite of methods that test what sort of characters are present in a string. We'll look at just a few of these -- there are many more.

str.isalpha() returns True if all characters of the string are alphabetic, False otherwise. It returns False if the string is empty.

Copy Code
```
'Hello'.isalpha()      # True
'Good-bye'.isalpha()   # False: `-` is not a letter
'Four score'.isalpha() # False: space is not a letter
''.isalpha()           # False
```
str.isdigit() returns True if all characters of the string are digits, False otherwise. It returns False if the string is empty.

Copy Code
```
'12340'.isdigit()      # True
'123.4'.isdigit()      # False: `.` is not a digit
'-1234'.isdigit()      # False: `-` is not a digit
''.isdigit()           # False
```
str.isalnum() returns True if a string is composed entirely of letters and/or digits, False otherwise. It returns False if the string is empty.

str.islower() returns True if all cased characters in a string are lowercase letters, False otherwise. It returns False if the string contains no case characters.

str.isupper() returns True if all cased characters in the string are uppercase, False otherwise. It returns False if the string contains no case characters.

str.isspace() returns True if all characters in the string are whitespace characters, False otherwise. It returns False if the string is empty. The whitespace characters include ordinary spaces ( ), tabs (\t), newlines (\n), and carriage returns (\r). It also includes two rarely used characters: vertical tabs (\v) and form feeds (\f), as well as some foreign characters that count as whitespace.

Be careful with these methods: they're all Unicode-aware. Thus, 'Î§Î±Î»Ï‰Ïƒ Î¸ÎµÏ'.isalpha() returns True since the characters are all part of the Greek alphabet. If you need to exclude non-ASCII characters, use this pattern:

Copy Code
```
text.isalpha() and text.isascii()
```
Stripping Characters
The str.strip method returns a copy of a string with all leading and trailing whitespace characters (see str.isspace above) removed. Python programmers often use this method to strip unwanted whitespace from input data, such as keyboard input. Unwanted whitespace frequently makes input data harder to work with, so

immediately removing excess whitespace is a good idea.

Copy Code
```
text = input(prompt).strip()
```
In some of the examples in this section, we use the built-in repr function when printing strings. repr formats strings with quotes and clearly shows the presence of whitespace characters:

Copy Code
```
text = ' \t  abc def    \n\r'
print(repr(text))             # ' \t  abc def    \n\r'
print(repr(text.strip()))     # 'abc def'
```
You can also tell strip to remove other characters by providing a string argument. The characters inside this string are the ones you want removed.

Copy Code
```
text = ' \t  abc def    \n\r'
print(repr(text.strip('abc'))) # ' \t  abc def    \n\r'

text = 'aaabaacccabxyzabccba'
print(text.strip('a'))         # baacccabxyzabccb
print(text.strip('ab'))        # cccabxyzabcc
print(text.strip('ba'))        # cccabxyzabcc
print(text.strip('abc'))       # xyz
print(text.strip('bc'))        # aaabaacccabxyzabccba

print(repr(text.strip('abcxyz'))) # ''
```
There are things to note with the above examples:

Lines 2 and 9 show that only leading and trailing characters that match the argument are removed.
Lines 6-8 show that strip removes individual characters, not substrings. That is, the order of the characters in the argument doesn't matter. Thus, on line 8, we remove all of the leading and trailing 'a', 'b', and 'c' characters, leaving nothing but 'xyz'.
The str.lstrip method is identical to str.strip except it only removes leading characters (the leftmost). Similarly, str.rstrip removes trailing characters (the rightmost).

Copy Code
```
text = 'aaabaacccabxyzabccba'

print(text.lstrip('a'))       # baacccabxyzabccba
print(text.rstrip('a'))       # aaabaacccabxyzabccb

print(text.lstrip('ab'))      # cccabxyzabccba
print(text.rstrip('ab'))      # aaabaacccabxyzabcc

print(text.lstrip('ba'))      # cccabxyzabccba
print(text.rstrip('ba'))      # aaabaacccabxyzabcc

print(text.lstrip('abc'))     # xyzabccba
print(text.rstrip('abc'))     # aaabaacccabxyz
```

startswith and endswith
Two related methods are available for determining whether a string begins or ends with a given substring. Let's examine these methods:

str.startswith returns True if the calling string begins with a certain substring, False if it does not:

Copy Code
```
'Four score and seven'.startswith('Four score')  # True
'Four score and seven'.startswith('For score')   # False
```

```
'Four score and seven'.startswith('score')       # False
```
The argument can also be a tuple of strings:

Copy Code
```
'abc def'.startswith(('abc', 'xyz', 'stu'))  # True
'def ghu'.startswith(('abc', 'xyz', 'stu'))  # False
'xyz uvw'.startswith(('abc', 'xyz', 'stu'))  # True
'stu vwx'.startswith(('abc', 'xyz', 'stu'))  # True
```
The method also accepts "start" and "end" indexes to control where the search begins and ends:

Copy Code
```
'abc def'.startswith('def', 4)           # True
'abc def ghi'.startswith('def', 4, 7)    # True
```
str.endswith returns True if the calling string ends with a certain substring, False if it does not:

Copy Code
```
'Four score and seven'.endswith('and seven')  # True
'Four score and seven'.endswith('ad seven')   # False
'Four score and seven'.endswith('score')      # False
```
As with startswith, the argument can be a tuple of strings. You can also supply "start" and "end" indexes:

Copy Code
```
'abc def'.endswith(('abc', 'xyz', 'stu'))  # False
'abc def'.endswith(('xyz', 'def'))         # True
'abc def'.endswith('def', 4)               # True
'abc def ghi'.endswith('def', 4, 7)        # True
```

Splitting and Joining Strings
The str.split method returns a list of the words in the calling string. By default, split splits the string at sequences of one or more whitespace characters:

Copy Code
```
text = '  Four     score and   seven years ago.   '
print(text.split())
# ['Four', 'score', 'and', 'seven', 'years', 'ago.']

print('no-spaces'.split()) # ['no-spaces']
```
If you want to split on something other than spaces, you can tell Python what character or character string should act as a delimiter:

Copy Code
```
text = ',Four,score,and,,,seven,years,ago,'
print(text.split(','))
# Pretty printed for clarity
# [
#     '',
#     'Four',
#     'score',
#     'and',
#     '',
#     '',
#     'seven',
#     'years',
#     'ago',
#     ''
# ]
```
Note that specifying a delimiter changes the splitting behavior. Instead of looking for runs of whitespace, it splits the string at every occurrence of the delimiter. This also applies when using a literal space character as the

delimiter. Compare the output of the code below to our first example:

Copy Code
```
text = '  Four     score and   seven years ago.   '
print(text.split(' '))
# Partially pretty printed for clarity
# ['', '', 'Four', '', '', '', '', 'score', 'and',
#  '', '', 'seven', 'years', 'ago.', '', '', '']
```
split also recognizes multi-character delimiters:

Copy Code
```
text = 'Four<>score<:>and<>seven<>years<>ago'
print(text.split('<>'))
# ['Four', 'score<:>and', 'seven', 'years', 'ago']
```
Multi-character delimiters must match exactly. Thus, the < and > in <:> aren't split.

Most major languages have a split function or method. It's worth noting that Python's differs from most others in that you can't split a string into individual characters using split. Python's str.split method doesn't allow a separator of ''. If you need to split a string into characters, use the list or tuple function:

Copy Code
```
text = 'abcde'
print(list(text))            # ['a', 'b', 'c', 'd', 'e']
print(tuple(text))           # ('a', 'b', 'c', 'd', 'e')
```
You can also iterate strings a character at a time:

Copy Code
```
text = 'abcde'
for char in text:
    print(char)
```

str.splitlines returns a list of lines from a string. splitlines looks for line-ending characters like \n (line feed), \r (carriage return), \n\r (new lines), and a variety of other line boundaries. See the documentation for a complete list.

Copy Code
```
text = '''
You were lucky to have a room. We used to have to
live in a corridor.
Oh we used to dream of livin' in a corridor!
Woulda' been a palace to us. We used to live in an
old water tank on a rubbish tip. We got woken up
every morning by having a load of rotting fish
dumped all over us.
'''

print(text.strip().splitlines())
# Pretty printed for clarity
[
    "You were lucky to have a room. We used to have to",
    "live in a corridor.",
    "Oh we used to dream of livin' in a corridor!",
    "Woulda' been a palace to us. We used to live in an",
    "old water tank on a rubbish tip. We got woken up",
    "every morning by having a load of rotting fish",
    "dumped all over us."
]
```
The final method we'll discuss in this cluster is str.join, which concatenates all strings in an iterable collection into a single lone string. Each string from the collection gets concatenated to the previous string with the value of

the calling string between them:

Copy Code
```
words = ['You', 'were', 'lucky']
print(''.join(words))         # Youwerelucky
print(' '.join(words))        # You were lucky
print(','.join(words))        # You,were,lucky
print('\n  '.join(words))
# You
#   were
#   lucky
```
Finding Substrings

We often need to search through strings looking for a particular substring. Python provides several ways to do this, including the in and not in operators we looked at earlier.

Here, we'll look at the str.find and str.rfind methods. str.find searches through a string looking for the first occurrence of the argument. str.rfind does the same, but it searches from right to left (that is, in reverse). Both methods return the index of the first matching substring. Otherwise, they return -1.

Copy Code
```
school = 'launch school'

print(school.find(' '))       # 6
print(school.find('l'))       # 0
print(school.find('h'))       # 5
print(school.find('hoo'))     # 9
print(school.find('x'))       # -1
print(school.find('N'))       # -1

print(school.rfind(' '))      # 6
print(school.rfind('l'))      # 12
print(school.rfind('h'))      # 9
print(school.rfind('hoo'))    # 9
print(school.rfind('oh'))     # -1
print(school.rfind('N'))      # -1
```
Note that both find and rfind are case sensitive, so 'N' doesn't match 'n'.

You can also search slices by adding start and end arguments to the invocation:

Copy Code
```
text = 'abc abc def abc'

print(text.find(' ', 4))         # 7
print(text.find(' ', 8))         # 11

print(text.find('c', 0, 2))      # -1
print(text.rfind('c', 3, 10))    # 6
```
Using find or rfind to search slices is not the same as using the [start:stop] syntax first and then searching the result. Compare the outputs for the two find invocations below:

Copy Code
```
text = 'abc abc def abc'

print(text[3:10].find('c'))     # 3
print(text.find('c', 3, 10))    # 6
```
If you take a slice of a string before using it to call find or rfind, the method returns the index of the search string in that slice. However, the method considers the starting index when you use the slicing arguments. Another difference is that taking a slice first creates a new string, while using slicing arguments does not.

Nested Collections
Collections can be nested inside other collections. For instance, you can have a list that contains a dict, a set, a tuple, and another list. Each of those can, in turn, also contain nested collections:

Copy Code
```
nested_list = [
    {'foo': 42, 'bar': [1, 2, 3], 'qux': None},
    {
        'Kim',
        ('Leslie', 'Les'),
        ('Pete', 'Peter'),
        ('Jonathan', 'Jon', 'Jack'),
    },
    (4, 5, (1, 2, 3), 6, 7),
    ['a', 'b', 'cde', -3.141592],
]
```
There are, however, some limitations on what you can nest in certain collections. For instance, you can't nest a mutable collection such as a list, dictionary, or another set inside a set:

```
>>> my_set = {1, 2, 3, [4, 5]}
TypeError: unhashable type: 'list'

>>> my_set = {1, 2, 3, {4, 5}}
TypeError: unhashable type: 'set'
```
However, you can nest a frozen set inside a set or frozen set:

```
>>> my_set = { 1, 2, 3, frozenset([4, 5]) }
>>> my_set          # {frozenset({4, 5}), 1, 2, 3}
```
Curiously, you can nest mutable collections inside tuples even though tuples are immutable.

```
>>> my_tuple = (1, 2, 3, [4, 5], {6, 7}, {'x': 'a dict'})
>>> my_tuple
(1, 2, 3, [4, 5], {6, 7}, {'x': 'a dict'})
```
In most cases, nested collections have some sense of structure and reason for the nesting. For instance, we might define a deck of cards as a list of dictionaries:

Copy Code
```
deck = [
    { 'suit': 'Clubs', 'value': '2' },
    { 'suit': 'Clubs', 'value': '3' },
    { 'suit': 'Clubs', 'value': '4' },
        ...
    { 'suit': 'Spades', 'value': 'Queen' },
    { 'suit': 'Spades', 'value': 'King' },
    { 'suit': 'Spades', 'value': 'Ace' },
]
```
If we want to print the fiftieth card in the deck, we can write:

Copy Code
```
print(f"{deck[49]['value']} of {deck[49]['suit']}")
# Queen of Spades
```
You can also have several layers of nesting in a sequence. In that case, you can access any item from any nested part of the sequence by just running several [] indexing operations together:

Copy Code
```
nested_seq = [
    [1, 2, [3, 4, (5, 6, 7, 8)]],
    [9, [10, (11,)]],
```

```
    [12, 13, [14, 15, (16, 17)]],
    [18, [19, 20, (21, 22)]],
]

print(nested_seq[1])          # [9, [10, (11,)]]
print(nested_seq[3][0])       # 18
print(nested_seq[0][2][2])    # (5, 6, 7, 8)
print(nested_seq[2][2][2][1]) # 17
```
Nested collections become really "fun" when you start having to iterate through them.

Comparing Collections

As you might expect, Python supports comparison operations for collections. It provides comparison mechanisms for all built-in iterable collections, and you can define comparisons for any custom iterables you create.

Equality is the most straightforward comparison. If two iterables meet all of the following requirements, they are equal. Otherwise, they are unequal.

They have the same type: (list, tuple, set, etc.) Note that sets and frozen sets are considered the same for comparison purposes.
They have the same number of elements.
For sequences, each pair of corresponding elements compares as equal.
For sets, each set has the same members (order doesn't matter).
For mappings, each key/value pair must be present and identical in both mappings (order doesn't matter).
Copy Code
```
print([2, 3] == [2, 3])    # True
print([2, 3] == [3, 2])    # False (diff sequence)
print([2, 3] == [2])       # False (diff lengths)
print([2, 3] == (2, 3))    # False (diff types)
print({2, 3} == {3, 2})    # True (same members)

dict1 = {'a': 1, 'b': 2}
dict2 = {'b': 2, 'a': 1}
dict3 = {'a': 1, 'b': 2, 'c': 3}

print(dict1 == dict2)      # True (same pairs)
print(dict1 == dict3)      # False
```
Of course, you can also use != to compare for inequality.

You can also compare sequences for <, <=, >, and >=. However, we won't go into that here. It's rarely needed.

Loops and Iterating

Most programs require code that runs repeatedly. Such code runs while a given condition remains truthy or until it becomes falsy (they mean the same thing). Most programming languages, including Python, use loops to provide this capability.

Python loops have several forms, but the main looping structures are the for and while statements. These loops execute a block repeatedly while a condition remains truthy. You can also think of the loop running until the condition becomes falsy. Both mental models are equivalent.

Python has three other looping mechanisms: comprehensions, generators, and functional loops. We'll talk about comprehensions in this chapter. However, we'll postpone the discussion of generators and functional loops to a later course in the Core Curriculum.

Let's begin with the while loop.

while Loops

A while loop uses the while keyword followed by a conditional expression, a colon (:), and a block. The loop repeatedly executes the block while the conditional expression remains truthy. In most programs, that loop should ultimately stop repeating. That means the block must do something to help Python know when to stop. That is, it must arrange to terminate the loop. That's usually triggered by evaluating a conditional expression. Otherwise, the loop is an infinite loop that never stops repeating.

To see why a while loop is useful, consider writing some code that prints numbers from 1 to 10:

Copy Code
```
print(1)
print(2)
print(3)
print(4)
print(5)
print(6)
print(7)
print(8)
print(9)
print(10)
```
While that code is straightforward and readily understood, it's easy to see why this approach is unsustainable. Suppose we need to print the numbers from 1 to 1000 or 1,000,000. If you had to write all that code for such a simple task, you would soon regret your career choice.

Let's rewrite the program with a while loop. Create a file named counter.py with the following code and run it:

counter.pyCopy Code
```
counter = 1
while counter <= 10:
    print(counter)
    counter += 1
```
This code does the same thing as the first program but more programmatically. It iterates over the block for all integer values between 1 and 10, inclusive. Each time the block runs is called an iteration.

If you want to print 1000 numbers or even a million, all you have to change is the conditional expression:

counter.pyCopy Code
```
counter = 1
while counter <= 1000:
    print(counter)
    counter += 1
```
Go ahead and run the code now using the command line:

Copy Code
```
python counter.py
```
When Python encounters this while loop, it evaluates the conditional expression, counter <= 1000. Since counter's initial value is 1, the expression is initially True, so Python executes the block. We print counter's value inside the block, then increment it by 1.

After the first iteration, Python re-evaluates the conditional expression. This time, counter is 2, which is still less than or equal to 1000; thus, the block runs again. After 1000 iterations, counter's value becomes 1001. Since the loop condition is no longer truthy, the program stops looping. It continues with the first expression or statement after the loop.

Line 4 in this example is crucial. The block must modify counter somehow, ultimately making the loop condition falsy. If it doesn't, the loop never ends,

which is usually not what you want. If your program never stops running, you probably have an infinite loop somewhere in the program. Try commenting out line 4 and rerun the code. You'll see that it continually prints the number 1. You can use the Control+c keystroke to terminate the program.

Using while Loops with Sequences
One of the most common uses of loops in programming is to iterate over a sequence's elements and perform some action on each element. For example, we may want to iterate over a list of names and create a new list that contains the names in uppercase. Here's one way to do that:

names.pyCopy Code
```
names = ['Chris', 'Max', 'Karis', 'Victor']
upper_names = []
index = 0

while index < len(names):
    upper_name = names[index].upper()
    upper_names.append(upper_name)
    index += 1

print(upper_names);
# ['CHRIS', 'MAX', 'KARIS', 'VICTOR']
```
A bit of explanation is in order here. The variable names holds a list of names. We want to append each name, in uppercase, to the initially empty upper_names list. Since list indexes are zero-based, we initialize an index variable with 0.

Next, we use a loop that executes as long as the number in index is smaller than the length of the names list. Line 8 increments the index by 1 after each iteration, which ensures that index < len(names) becomes falsy after the loop handles the last element.

Line 6 accesses the name stored at names[index] and uses it to call str.upper. That method returns the name in uppercase, which we assign to upper_name. It doesn't change the original name in the names list.

Line 7 uses list.append to append the latest uppercase name to the upper_names list. Over the four iterations of the names list, line 7 appends four uppercase names to upper_names, one per iteration, in the same order the loop processes them.

Notice that we initialized names, upper_names, and index before the loop. We don't want to initialize them inside the loop; they would get reset during every iteration. Basically, nothing would change. That wouldn't work well even if the code ran without error.

for Loops
for loops have the same purpose as while loops, but they use a condensed syntax that works well when iterating over lists and other sequences. A for loop lets you forget about indexing your sequences. You don't have to initialize or increment the index value or even need a condition. Moreover, for loops work on all built-in collections (including strings). Most loops you write in Python will be for loops.

Copy Code
```
for element in collection:
    # loop body: do something with the element
```
If collection is a sequence, this structure behaves in much the same way as:

Copy Code
```
index = 0
while index < len(collection):
    # loop body
    # do something with collection[index]
```

```
    index += 1
```
The for loop is more elegant and significantly less error-prone.

Let's rewrite names.py with a for loop to better illustrate using for:

names.pyCopy Code
```
names = ['Chris', 'Max', 'Karis', 'Victor']
upper_names = []

for name in names:
    upper_name = name.upper()
    upper_names.append(upper_name)
    # Deleted: index += 1

print(upper_names);
# ['CHRIS', 'MAX', 'KARIS', 'VICTOR']
```
The result is the same as with the while loop. However, the code is much easier to read and maintain.

For illustrative purposes, let's see what happens when we use a for loop to iterate over a string:

chars.pyCopy Code
```
for char in 'Launch School':
    print(char)
```
OutputCopy Code
```
L
a
u
n
c
h

S
c
h
o
o
l
```
If you want to work with words instead of characters, you can use the split method:

chars.pyCopy Code
```
for word in 'Launch School'.split():
    print(word)
```
OutputCopy Code
```
Launch
School
```
You can also use for loops with other collections, such as sets and dicts. Any iterable collection, in fact:

sets.pyCopy Code
```
# Looping over a set
my_set = {1000, 2000, 3000, 4000, 5000}
for member in my_set:
    print(member)
```
OutputCopy Code
```
# The output may not be in this sequence since it
# comes from a set.
4000
2000
5000
3000
1000
```

dicts.pyCopy Code
```
# Looping over a dictionary
my_dict = {'a': 1, 'b': 2, 'c': 3}
for key in my_dict:
    print(key)
```
OutputCopy Code
```
a
b
c
```
Using a for loop with a dict iterates over the dict keys by default. If you want the values or pairs, you can request them with the values or items methods:

dicts.pyCopy Code
```
# Looping over a dictionary's values
my_dict = {'a': 1, 'b': 2, 'c': 3}
for value in my_dict.values():
    print(value)
```
OutputCopy Code
```
1
2
3
```
dicts.pyCopy Code
```
# Looping over a dictionary's key/value pairs
my_dict = {'a': 1, 'b': 2, 'c': 3}
for item in my_dict.items():
    print(item)
```
OutputCopy Code
```
('a', 1)
('b', 2)
('c', 3)
```
A more Pythonic way to iterate over both the keys and values simultaneously is to use tuple unpacking:

dicts.pyCopy Code
```
# Looping over a dictionary's key/value pairs
my_dict = {'a': 1, 'b': 2, 'c': 3}
for (key, value) in my_dict.items():
    print(f'{key} = {value}')
```
OutputCopy Code
```
a = 1
b = 2
c = 3
```
We'll cover tuple unpacking in the Core Curriculum. For now, all you need to know is that each key returned by the items method gets assigned to the key variable, and each associated value gets assigned to the value variable.

By the way: you don't need the parentheses around key, value. The following code works and is more Pythonic:

dicts.pyCopy Code
```
# Looping over a dictionary's key/value pairs
my_dict = {'a': 1, 'b': 2, 'c': 3}
for key, value in my_dict.items():
    print(f'{key} = {value}')
```

Nested Loops
You will often need to nest loops within one or more outer loops. For instance, suppose you want to create a deck of cards given two lists: the suits and ranks you want to combine:

cards.pyCopy Code
```
suits = ['Clubs', 'Diamonds', 'Hearts', 'Spades']
ranks = [
    '2', '3', '4', '5', '6', '7', '8', '9', '10',
```

```
    'Jack', 'Queen', 'King', 'Ace',
]

deck = []
for suit in suits:
    for rank in ranks:
        card = f'{rank} of {suit}'
        deck.append(card)

print(deck)
```

With nested for loops, you start with the outer loop and assign the variable to the first element of its collection (suit and suits). You then process the inner loop in its entirety. Here, we match the first suit with every possible card rank, appending each card to the deck.

Once the inner loop finishes processing, control returns to the outer loop. Working with the second element of the outer loop's collection, it again iterates over the inner loop's collection. This continues until all members of the outer loop's collection have been processed. In our card deck example, the outer loop runs 4 times, while the inner loop runs 4 * 13 (52) times.

Nesting 3 or more levels deep is possible but frequently inadvisable. Too much nesting is hard to understand. Using one or more helper functions to handle the more deeply nested processing is often a good idea.

You can also nest while loops and mix for and while loops.

Controlling Loops

Python uses the keywords continue and break to provide more control over for and while loops. continue starts a new loop iteration; break terminates the loop early.

Continuing a Loop With Next Iteration

Let's stick with the names.py program. Suppose we want all the uppercase names in our upper_names list except 'Max'. The continue statement can help us do that.

names.pyCopy Code

```
names = ['Chris', 'Max', 'Karis', 'Victor']
upper_names = []

for name in names:
    if name == 'Max':
        continue

    upper_name = name.upper()
    upper_names.append(upper_name)

print(upper_names);
# ['CHRIS', 'KARIS', 'VICTOR']
```

The result doesn't contain 'MAX'.

When a loop encounters the continue keyword, it skips running the rest of the block and jumps ahead to the next iteration. In this example, we tell the loop to ignore 'Max' and skip to the next iteration without adding 'MAX' to upper_names.

You can often rewrite a loop that uses continue with a negated if conditional:

names.pyCopy Code

```
names = ['Chris', 'Max', 'Karis', 'Victor']
upper_names = []

for name in names:
```

```
    if name != 'Max':
        upper_name = name.upper()
        upper_names.append(upper_name)

print(upper_names);
# ['CHRIS', 'KARIS', 'VICTOR']
```
This code behaves like the version that uses continue but is a little more concise.

Why bother using continue if we can write looping logic without it? You don't have to use continue, of course. However, it often leads to a more elegant solution to a problem. Without continue, your loops can get cluttered with nested conditional logic.

Copy Code
```
for value in collection:
    if some_condition():
        # some code here
        if another_condition():
            # some more code here
```
We can use continue to rewrite this loop without the nested ifs:

Copy Code
```
for value in collection:
    if not some_condition():
        continue

    # some code here

    if not another_condition():
        continue

    # some more code here
```
Which of these is more readable and easier to maintain? That's not always easy to answer. As is often the case, the choice may be made for you by local coding standards. At other times, it's a matter of deciding which version looks and reads better.

The continue statement tells Python to start the next iteration of the nearest enclosing loop. You can't start a new iteration of an outer loop if you're currently in an inner (nested) loop.

Breaking Out of a Loop
Instead of exiting the current iteration, you sometimes want to stop iterating completely. For instance, when you search a list for a specific value, you probably want to stop searching once you find it. There's no reason to keep searching if you don't need any subsequent matches.

Let's explore this idea with some code. Create a file named search.py with the following code and run it from your terminal:

search.pyCopy Code
```
numbers = [3, 1, 5, 9, 2, 6, 4, 7]
found_item = -1
index = 0

while index < len(numbers):
    if numbers[index] == 5:
        found_item = index

    index += 1

print(found_item)
```
This program iterates over the elements of a list to find the element whose

value is 5. It saves the index value in found_item. However, the loop continues to iterate after finding the desired value. That seems pointless and wasteful. It's where break steps in and saves the day:

search.pyCopy Code

```
numbers = [3, 1, 5, 9, 2, 6, 4, 7]
found_item = -1
index = 0

while index < len(numbers):
    if numbers[index] == 5:
        found_item = index
        break

    index += 1

print(found_item)
```

The break statement tells Python to terminate the nearest enclosing loop once we find the desired element. You can't break out of an outer loop if you're currently in an inner (nested) loop.

Emulating Do/While Loops

A typical while loop looks something like this:

Copy Code

```
while some condition is truthy
    do some work
```

In most code, this is a perfectly fine solution; you usually want to skip over the "do some work" part if "some condition" is initially falsy. However, sometimes you want to "do some work" at least once, even if the condition is initially falsy. For that, you need something like this:

Copy Code

```
do some work
while some condition is truthy
```

In other words, you always want to "do some work" at least once. This is often called a do/while or do/until loop since many languages have a looping structure with those names. Python does not, so you must take a different approach. That typically uses a break statement at the end of the loop, like so:

Copy Code

```
do some work
    if some condition is falsy
        break
```

This often comes up in interactive programs where you may want to execute the main program loop at least once before exiting. The code to handle such a loop might look something like this:

Copy Code

```
keep_going = True
while keep_going:
    # main loop code is here

    answer = input('Play again? (y/n) ')
    if answer == 'n':
        keep_going = False
```

That's workable, albeit a little awkward. A slightly less clumsy alternative uses break and while True.

Copy Code

```
while True:
    # main loop code is here

    answer = input('Play again? (y/n) ')
```

```
    if answer == 'n':
        break
```

Simultaneous Iteration

We sometimes need to iterate through multiple collections in parallel. That is, we want to grab data from several collections during each loop iteration. If all the collections are indexed sequences, you can write a while loop using indexes. However, that's error-prone and often messy.

Meet the hero of parallel iteration: zip! The zip function is specifically designed to make simultaneous iteration easy. Let's see what this looks like.

Suppose you need to print the full names of several thousand people. For bizarre historical reasons, you have two lists: one contains the forenames, and the other contains the corresponding surnames. Without zip, you need to use a while loop and indexing. That's feasible, so let's try it. We'll use 4 names to test the concept:

Copy Code

```
forenames = ['Ken', 'Lynn', 'Pat', 'Nancy']
surnames = ['Camp', 'Blake', 'Flanagan', 'Short']

index = 0
while index < len(forenames):
    if index >= len(surnames): # surnames might be shorter.
        break

    forename = forenames[index]
    surname = surnames[index]
    print(f'{forename} {surname}')

    index += 1
```

Copy Code

```
Ken Camp
Lynn Blake
Pat Flanagan
Nancy Short
```

While it's not horrid, it's a lot of work. We can do better than that with zip:

Copy Code

```
forenames = ['Ken', 'Lynn', 'Pat', 'Nancy']
surnames = ['Camp', 'Blake', 'Flanagan', 'Short']

zipped_names = zip(forenames, surnames)
for forename, surname in zipped_names:
    print(f'{forename} {surname}')
```

That definitely looks better! However, we should explain lines 4 and 5. On line 4, zip creates a lazy sequence that acts like a list of tuples. Each tuple contains a forename and a surname. Line 5 is the start of our loop. We're taking advantage of the fact that for can assign multiple variables when the collection elements are tuples. Thus, we can grab the forename and surname from the current tuple.

Note that zip takes care of the potential problem of dealing with lists of different sizes. Our first attempt needed that if statement in the while block to guard against the surnames list being shorter than forenames.

Comprehensions

Python supports a concise and readable way to create mutable collections from existing iterable collections: comprehensions. There are 3 comprehension types: list, dict, and set. Properly used, comprehensions can simplify your code and make it easier to understand.

Unlike most for and while loops, which are statements, comprehensions are expressions. You can use a comprehension on the right side of an assignment, as

a function argument, as a return value, or any other place where you can use an expression that evaluates as a list, dict, or set. You can even use them as standalone expressions:

Copy Code
```
[print(foo) for foo in collection]
```
However, an ordinary for loop is the preferred alternative.

List Comprehensions
The most commonly used comprehensions are list comprehensions. They take an iterable collection and create a new list through iteration and optional selection. List comprehensions have the following format:

Copy Code
```
[ expression for element in iterable if condition ]
```
The if condition portion is optional: it tells Python to select only certain elements from the iterable. The for element in iterable portion describes the iteration: it looks exactly like a for loop. It can be read in much the same way. Finally, the expression is a value that gets returned by each iteration of the loop. Python collects all the return values and puts them in a new list.

The expression in a comprehension often performs a transformation. It determines a new value based on an element from the original collection. Such comprehensions are called transformations.

If the if condition portion is present, we say that the comprehension also performs selection. With selections, it's not uncommon to return the original values from the collection:

Copy Code
```
[ element for element in iterable if condition ]
```
The terms transformation and selection are handy to remember. You will encounter them in many languages.

Consider this basic example of a transformative list comprehension:

list_comp.pyCopy Code
```
squares = [ number * number for number in range(5) ]
print(squares)      # [0, 1, 4, 9, 16]
```
Here, we're iterating over the numbers in the indicated range: 0, 1, 2, 3, 4. We compute the square with number * number for each number. Finally, Python collects all the squares into a list and assigns the list to the squares variable. Voila! We now have a list of squares.

Of course, we could do the same thing with an ordinary for loop:

list_comp.pyCopy Code
```
squares = []
for number in range(5):
    square = number * number
    squares.append(square)

print(squares)      # [0, 1, 4, 9, 16]
```
That's quite a bit more code, and the effort of reading it is a bit higher.

Let's look at a selection example:

list_comp.pyCopy Code
```
multiples_of_6 = [ number for number in range(20)
                   if number % 6 == 0 ]
print(multiples_of_6)      # [0, 6, 12, 18]
```
Here, we see the pattern of using the original collection values as the expression. We've also split the comprehension over two lines, which can aid readability.

This example combines selection and transformation:

list_comp.pyCopy Code
```
even_squares = [ number * number
                 for number in range(10)
                 if number % 2 == 0 ]
print(even_squares)      # [0, 4, 16, 36, 64]
```
This code selects all of the even numbers in the specified range and then returns a list of the squares of the chosen numbers.

Let's try iterating over a dictionary. Suppose we have a dict of cats. We're using the cat names as keys; each name is associated with the cat's coat color. We want to create a list of the names in uppercase:

list_comp.pyCopy Code
```
cats_colors = {
    'Tess':   'brown',
    'Leo':    'orange',
    'Fluffy': 'gray',
    'Ben':    'black',
    'Kat':    'orange',
}

names = [ name.upper() for name in cats_colors ]
print(names)
# ['TESS', 'LEO', 'FLUFFY', 'BEN', 'KAT']
```
This is nearly identical to using the dict.keys method. The only difference is that the list comprehension returns an ordinary list, not a dictionary view object.

You can also iterate over the values by iterating in cats_colors.values() or the key/value pairs by iterating in cats_colors.items().

Suppose we now want to limit the result list to just orange cats. We can accomplish that by adding a selection criterion to the comprehension:

list_comp.pyCopy Code
```
cats_colors = {
    'Tess':   'brown',
    'Leo':    'orange',
    'Fluffy': 'gray',
    'Ben':    'black',
    'Kat':    'orange',
}

names = [ name.upper()
          for name in cats_colors
          if cats_colors[name] == 'orange' ]
print(names) # ['LEO', 'KAT']
```
It's also possible to use multiple selection criteria. Let's limit the result to cats whose name begins with L:

list_comp.pyCopy Code
```
cats_colors = {
    'Tess':   'brown',
    'Leo':    'orange',
    'Fluffy': 'gray',
    'Ben':    'black',
    'Kat':    'orange',
}

names = [ name.upper()
          for name in cats_colors
```

```
          if cats_colors[name] == 'orange'
          if name[0] == 'L' ]
print(names) # ['LEO']
```
Multiple selection criteria act like nested if statements or as and-ed conditions. The selections combine, so only collection members matching all criteria are selected.

Comprehensions can also have multiple for loop components. For instance, let's generate a deck of cards based on a list of the suits and a list of the ranks:

cards.pyCopy Code
```
suits = ['Clubs', 'Diamonds', 'Hearts', 'Spades']
ranks = [
    '2', '3', '4', '5', '6', '7', '8', '9', '10',
    'Jack', 'Queen', 'King', 'Ace',
]

deck = [ f'{rank} of {suit}'
         for suit in suits
         for rank in ranks ]
print(deck)
```
Compare this code with the one shown earlier in Nested Loops. As multiple selection criteria resemble a nested if, multiple looping components resemble a nested for loop.

Dictionary Comprehensions

Dictionary comprehensions are almost identical to list comprehensions. However, they create new dictionaries instead of lists. Syntactically, they use curly braces instead of square brackets, and the expression becomes a "key: value" pair. Dictionary comprehensions have the following format:

Copy Code
```
{ key: value for element in iterable if condition }
```
The most significant difference is that the expression component is now a key/value pair, each given by another expression.

Consider this basic example of a dictionary comprehension:

dict_comp.pyCopy Code
```
squares = { f'{number}-squared': number * number
            for number in range(1, 6) }
print(squares)
# pretty-printed for clarity.
{
    '1-squared': 1,
    '2-squared': 4,
    '3-squared': 9,
    '4-squared': 16,
    '5-squared': 25
}
```
Other than the differences mentioned above, list and dict comprehensions are identical.

Set Comprehensions

Set comprehensions look almost identical to dict comprehensions. However, they create a new set instead of a dict and only have one expression to the left of the word for:

Copy Code
```
{ expression for element in iterable if condition }
```
Previously, a colon-separated key/value expression pair was to the left of for. Now, we only have an expression.

Consider this basic example of a set comprehension:

dict_comp.pyCopy Code
```
squares = { number * number for number in range(1, 6) }
print(squares)      # {1, 4, 9, 16, 25}
```
Other than the differences mentioned above, dict and set comprehensions are identical.

Why No Tuple, Range, or String Comprehensions?
Python programmers often wonder why Python doesn't have tuple comprehensions. They can easily see that the following valid code hints that it might be a tuple comprehension:

dict_comp.pyCopy Code
```
squares = ( number * number for number in range(1, 6) )
print(squares) # <generator object <genexpr> at 0x104e39a40>
```
However, it is not a comprehension. Instead, it is a generator expression. We won't discuss generators in this book, but you'll see them later in the Core Curriculum. You'll also learn why they are helpful. The fact that a generator expression looks like it should be tuple comprehension is merely a matter of syntax. It doesn't answer the question of why.

Comprehensions don't build their results all at once. Each kind of comprehension works something like this:

Copy Code
```
result = empty_collection               # [], {}, set()
for item in collection:
    result.append(item)
```
As you can see, our result starts as an empty collection. We then modify the result collection during each iteration by appending a new item to result. From this, it's clear that the result must be a mutable type. Tuples are immutable, so Python can't have tuple comprehensions.

Since ranges and strings are also immutable, comprehensions can't create them. If you must have a tuple or string, use the tuple or str constructors to convert a list comprehension's result into a tuple or string. (Sorry, but you can't do something similar to create a range.)

Variables as Pointers
This chapter examines the concepts behind variables and pointers. Specifically, we'll see how Python variables are pointers to a location in memory (an address space) that contains the object assigned to the variable. New programmers often struggle to master this concept, but the material is crucial. We'll meet these concepts again in Core.

Developers sometimes talk about references instead of pointers. At Launch School, we use these terms interchangeably. You can say that a variable points to or references an object in memory, and you can also say that the pointers stored in variables are references. Some languages make a distinction between references and pointers, but Python does not; feel free to use either term.

Variables As Pointers
The behaviors described in this chapter arise from the interaction of variables using pointers to reference their associated objects. In Python, all variables are pointers to objects. If you assign the same object to multiple variables, every one of those variables references (points to) the same object. They act like aliases for the object.

When you reassign a variable, Python changes what object the variable references. Reassignment doesn't alter the old or new object; it simply changes which object the variable references.

When a reassignment involves the creation of a new object, Python first creates the new object. It then changes the variable's stack item to point to the new

object. As in the previous paragraph, this does not alter the object.

Things get a little messier when mutating objects through a variable. If a variable points to a mutable object and you do something to mutate that object, Python doesn't change the variable; it changes the object. Every variable that references that object will immediately see the object's new state.

Copy Code
```
numbers1 = [1, 2, 3]
numbers2 = numbers1 # numbers2 points to
                    # the same object as numbers1

numbers1.pop() # `pop` removes the last element
print(numbers2) # [1, 2]
```

The author refers to this as Einsteinian spooky action at a distance or quantum variable entanglement.

The indexing syntax can make things a little confusing. For instance, assume we have a numbers list that looks like this:

Copy Code
```
numbers = [1, 2, 3, 4]
```
Now, let's reassign numbers[2]:

Copy Code
```
numbers[2] = 3333
```
This is not a reassignment of the numbers variable. It's a mutation of the list object referenced by numbers. Technically, it's also a reassignment of the list object element at index 2, but it is not a mutation of that element.

When using variables, it's essential to remember that some operations mutate objects while others do not. For instance, list.sort mutates a list, but sorted(list) does not. In particular, you must understand how lines 2 and 3 in the following code differ:

Copy Code
```
x = [1, 2, 3, 4, 5]
x = [1, 2, 3]
x[2] = 4
```
Both lines 2 and 3 are reassignments. However, line 3 mutates the list assigned to x while line 2 does not mutate anything: it simply reassigns the variable to a new value.

We should also look at augmented assignment. When the variable on the left side references an immutable value, augmented assignment acts like reassignment. Thus, all of the augmented assignments in the following code (lines 6-10) are reassignments since the values referenced on the left are all immutable:

Copy Code
```
a = 42
b = 3.1415
c = 'abc'
d = (1, 2, 3)

a *= 2
b -= 1
b /= 3
c += 'def'
d += (4, 5)
```
However, if the variable on the left references a mutable value, the augmented assignment is usually mutated. Thus, all the augmented assignments below mutate the value referenced by the variable:

Copy Code
```
a = [1, 2, 3]
b = {'a', 'b', 'c'}

a += [4, 5]
a *= 2
b -= {'b'}
```
All built-in mutable types will be mutated when used as the target of an augmented assignment. However, this behavior is not guaranteed for custom objects, which we'll meet in the Object Oriented Programming book.

The idea that Python stores the elements of a collection in the collection object itself is a simplification of how Python works under the hood. However, it mirrors reality well enough to be a mental model for almost all situations. Don't sweat the details right now; we will expand on this later in the chapter.

It's time to jump into our first real rabbit hole: how variables and objects work in Python.

Variables and Objects

At the most basic level, variables are named locations in a computer's memory. Each variable has a unique address in memory, usually in an area called the stack, and sometimes in a different area called the heap. You don't need to understand the differences between the stack and heap at this stage, but learning that they exist will help later in Core.

We'll pretend that all variables are stored on the stack.

The space allocated for a single variable is small. On modern computers in 2023, that's typically 64 bits (or 8 bytes). Objects often far exceed that size. Thus, objects usually aren't stored on the stack. Instead, Python allocates the memory it needs for an object from the heap. Heap blocks can be pretty much any size, provided sufficient memory is available.

Once Python allocates heap space, it creates and stores the object at that location. The address of that object is then copied to the variable's stack location.

Thus, when you access a variable, Python first determines where the variable is on the stack. It then takes the object's heap address from the stack item and uses it to find the object. The variable is a pointer to a stack location, and the stack location is a pointer to the object.

Let's break that down with an example. Assume we're creating a variable named numbers and giving it an initial value of a list with 3 numbers:

Copy Code
```
numbers = [1, 2, 3]
```
Since numbers is a new variable, Python adds a new entry to the stack. For simplicity, we'll assume the variable's stack address is at memory location 10240.

Next, Python allocates enough memory on the heap to hold the list and its elements (actually, the list object may contain pointers to the elements, but we'll ignore that for now). Let's say that Python allocates 32 bytes at address 4344278784 for the list and its elements. It then constructs an appropriate list object with its elements at address 4344278784.

Finally, Python copies the address 4344278784 into the variable's stack address. Suppose we now want to print the object assigned to numbers:

Copy Code
```
print(numbers)
```
To do this, Python looks up the name numbers in its list of variables and sees it is on the stack at address 10240. On the stack, Python can see that the

object associated with numbers is at address 4344278784. Finally, it passes that address to print, which formats and prints the object.

All this sounds incredibly complex for something as conceptually simple as giving a name to some data. Maybe it is. However, in the end, the process is lightning-fast and memory-efficient. There would be almost no benefit to simplifying the process in Python; in fact, it would likely eliminate some crucial benefits.

Variables and Shared Objects
Let's assign a new variable to the same object that is referenced by numbers:

Copy Code
```
numbers2 = numbers
```
Since numbers2 is new, Python must create a new variable on the stack, which it does. We'll assume it's at address 10256. It then determines the memory address of the object assigned to numbers and stores that address in numbers2's stack item. That means both numbers and numbers2 now point to the same object. You can verify this by using the id function or the is operator:

Copy Code
```
print(id(numbers) == id(numbers2))      # True
print(numbers is numbers2)              # True
```
Note also that both the numbers and numbers2 ids reference address 4344278784 (or whatever address your Python used for the list):

Copy Code
```
print(id(numbers))            # 4344278784
print(id(numbers2))           # 4344278784
```

Pointers have a curious effect when you assign a variable that references an existing object to another variable. Instead of copying the object referenced by the variable on the right to the variable on the left, Python only copies the pointer. Thus, when we initialize numbers2 with numbers, we make both numbers and numbers2 point to the same list object: [1, 2, 3]. It's not just the same value but the same list at the same address. The two variables are independent, but since they point to the same list, that list depends on what you do with both numbers and numbers2.

Equality and Identity
Two objects, obj1 and obj2, are said to be equal when obj1 == obj2 returns True. The objects are the same object when obj1 is obj2 returns True. We can also say that obj1 and obj2 have the same identity since id(obj1) and id(obj2) return the same value when obj1 is obj2 returns True.

Consider the following code:

Example 1Copy Code
```
numbers = [1, 2, 3]
numbers2 = numbers

print(numbers)                # [1, 2, 3]
print(numbers == numbers2)    # True
print(numbers is numbers2)    # True
```
Example 2Copy Code
```
numbers = [1, 2, 3]
numbers2 = [1, 2, 3]

print(numbers)                # [1, 2, 3]
print(numbers == numbers2)    # True
print(numbers is numbers2)    # False
```
On line 5 of both examples, numbers and numbers2 are equal since numbers == numbers2 returns True. This makes sense. Both variables point to a list whose values are the same: 1, 2, and 3. As we've already seen, the lists on line 6 of

Example 1 are the same objects.

However, are the lists on line 6 of Example 2 the same lists? No, they are not. They are entirely different objects even though they have the same values. As a result, line 6 prints False. Line 2 created a new object, which we assigned to numbers2

Shallow vs. Deep Copies
When you copy objects in any language, you must understand the difference between shallow and deep copies. Most copies created by Python programs are shallow copies. That might seem strange early on, but it works out for the best. Deep copies just aren't needed all that often. The copy.copy and copy.deepcopy functions from the built-in copy module are Python's primary ways to create shallow and deep copies, respectively.

Before we explore the differences, we must first understand that the memory picture is more complex than we described earlier. Let's take this object as an example:

Copy Code
[[1, 2], 3, 4]
As we discussed earlier, this object gets created in the heap. However, we assumed that Python would put all 3 elements in the heap memory allocated for the list object. We now have a nested list, though. That list, [1, 2], can't be stored directly in the [[1, 2], 3, 4] list. Instead, Python allocates additional memory on the heap for the inner list ([1, 2]). Instead of storing [1, 2] in the memory allocated for [[1, 2], 3, 4], it stores a pointer to the [1, 2] object

s you can see, we have two lists stored at different places in the heap. The outer list on the left contains 3 elements: the integers, 3 and 4, and a pointer to the second list. The inner list on the right includes integer elements, 1 and 2.

The actual memory picture is more complicated than this. The integers from both lists are also stored as separate objects. However, we're not interested in them right now, so we won't overcomplicate the diagram.

One last note: the various built-in constructors create shallow copies when passed an element of the same type:

Copy Code
```
my_list = [[1, 2], 3, 4]
shallow = list(my_list)
print(shallow[0] is my_list[0])         # True

my_dict = {'abc': [1, 2, 3]}
shallow = dict(my_dict)
print(shallow['abc'] is my_dict['abc']) # True
```
The main takeaway is that we have two different objects. Now, let's see what happens when we try to duplicate the first list.

Shallow Copies
A shallow copy of an object is a duplicate of the original object's outermost (topmost) level. Any nested objects within the copied object aren't duplicated; they still reference the nested objects from the original object. Thus, if you mutate the nested objects in the original, those mutations will be visible in the duplicate.

For instance, consider this code:

Copy Code
```
import copy

orig = [[1, 2], 3, 4]
```

```
dup = copy.copy(orig)

print(orig is dup)          # False
print(orig == dup)          # True
orig[2] = 44
print(dup)                  # [[1, 2], 3, 4]

print(orig[0] is dup[0])    # True
orig[0][1] = 22
print(dup[0])               # [1, 22]
```
In this example, line 3 creates the list we talked about above. Remember, the list is actually stored as separate objects. Note that [1, 2] is referenced by a pointer stored in the original list. We then create a shallow copy of the orig list on line 4 and call the duplicate dup.

On line 6, we confirm that orig and dup reference distinct objects. However, line 7 tells us they have the same values. We then mutate orig on line 8, then, on line 9, confirm that dup hasn't changed. Remember: orig and dup are separate and distinct objects.

Line 11 shows that both orig[0] and dup[0] reference the same objects. On line 12, we use orig[0] to mutate the [1, 2] list. When we print dup[0] on line 13, we can see that it reflects the change made to orig[0].

For completeness, here's what memory looks like now. We've reduced the diagram to the bare essentials. We use [inner] to represent a pointer to the inner list.

Deep Copies
A deep copy of an object is an exact duplicate of the original object at the outermost (or topmost) level and every nested object, no matter how deeply nested. There's basically nothing you can do to the original object that can be seen in the duplicate or vice versa.

Let's look at what happens to our previous code example when we introduce deep copies. We've highlighted the differences:

Copy Code
```
import copy

orig = [[1, 2], 3, 4]
dup = copy.deepcopy(orig)

print(orig is dup)          # False
print(orig == dup)          # True
orig[2] = 44
print(dup)                  # [[1, 2], 3, 4]

print(orig[0] is dup[0])    # False
orig[0][1] = 22
print(dup[0])               # [1, 2]
```
There are no code differences besides using copy.deepcopy on line 4. Furthermore, there are no behavioral differences until we reach lines 11-13. Line 11 shows that orig[0] and dup[0] are no longer the same objects. Thus, when we mutate the [1, 2] list referenced by orig[0], we don't see any changes in dup[0].

Note that copy.deepcopy doesn't duplicate everything; in most cases, it only duplicates mutable objects. Since immutable objects don't change, there's no need to copy them - references are enough to ensure a deep copy that works.

Should I Make Deep or Shallow Copies?
The answer to this question depends on your data structure, whether your data structure has mutable content, and whether it matters to your application.

In practice, shallow copies are frequently okay. They work best when:

working with objects that are not collections, e.g., integers and booleans.
working with immutable objects with no mutable components, e.g., strings.
working with collections that have no mutable elements, e.g., tuples that don't contain mutable elements.
needed for performance reasons. Shallow copies are always faster.
you don't care if the mutable components of an object are shared.
You should use deep copies when shallow copies are not okay. In particular, they work best when working with collections that have mutable elements, e.g., nested lists.

More Stuff

Let's focus on several useful topics that don't fit elsewhere. Most of these topics aren't crucial for a beginner. However, you will encounter them when you read Python code, documentation, and articles. Be ready!

Some of these topics have enough depth for a separate chapter, if not an entire book! We'll stick to the basics and give you a quick introduction to each.

Function Composition

A common Python technique is composing function calls, also known as composition. Composition occurs when a function call is used as an argument to another function call, which may, in turn, be passed to another function call. In each case, the return value of the inner function call is used as the argument to the outer function. We've seen many examples like this throughout this book but haven't really remarked on it. For instance, in the Introduction to Collections chapter, we saw this code:

Copy Code
```
print(list(range(3, 17, 4)))
```
When Python sees code like this, it first evaluates what it can without function calls. In this case, it determines that 3, 17, and 4 are integers. With those values, the only operation that can be performed directly is to evaluate range(3, 17, 4), so that's what it does. Once it has the range object, it can use the object as the argument to the list constructor. The constructor returns the list [3, 7, 11, 15], which Python passes to print for printing.

Let's look at some other examples of composing functions. First, let's define add and subtract functions and call them:

Copy Code
```
def add(a, b):
  return a + b

def subtract(a, b):
  return a - b

sum = add(20, 45)
print(sum)                              # 65

difference = subtract(80, 10)
print(difference)                       # 70
```
Both add and subtract take two arguments and return the result of performing an arithmetic operation on what we assume to be numeric values.

This works fine. However, we can use function composition to use a function call as an argument to another function. Stated differently, we're saying that we can write add(20, 45) and subtract(80, 10) as arguments for another function:

Copy Code
```
print(add(20, 45))                      # 65
print(subtract(80, 10))                 # 70
```
Passing the function call's return value to print is a canonical Python example

of function composition. It's functional but uninteresting. Things get more intriguing when you pass function call results to a function that does something more complicated:

Copy Code
```
def add(a, b):
  return a + b

def subtract(a, b):
  return a - b

def times(num1, num2):
  return num1 * num2

print(times(add(20, 45), subtract(80, 10))) # 4550
# 4550 == ((20 + 45) * (80 - 10))
```
Here, we pass the return values of add(20, 45) and subtract(80, 10) to the times function, and we pass the return value of times to print! It produces the same result as the following verbose code:

Copy Code
```
total = add(20, 45)
difference = subtract(80, 10)
result = times(total, difference)
print(result)
```
Composition works best when each of the inner functions returns an object other than None. The outermost function can return anything, including None. For instance, in print(times(add(20, 45), subtract(80, 10))), the inner functions (add, subtract, and times) all return integers, but the outermost function, print, returns None.

Most built-in functions return an object that another function can use. Thus, composition in Python is relatively commonplace. If you use composition, however, keep things simple. A deeply nested chain of function calls can be challenging to understand.

Method Chaining
Well-designed methods tend to do one thing and one thing only. For instance, the str.upper method returns a string in uppercase, and the str.split method returns the result of splitting a string into words. That means you can do something like this:

Copy Code
```
tv_show = "Monty Python's Flying Circus"
tv_show = tv_show.upper()
# "MONTY PYTHON'S FLYING CIRCUS"

tv_show = tv_show.split()
# ['MONTY', "PYTHON'S", 'FLYING', 'CIRCUS']
```
Method chaining is similar to function composition but applies to methods specifically rather than ordinary functions. It lets us rewrite the above code more concisely.

Copy Code
```
tv_show = "Monty Python's Flying Circus"
tv_show = tv_show.upper().split()
# ['MONTY', "PYTHON'S", 'FLYING', 'CIRCUS']
```
To use method chaining, you start by calling a method on an appropriate object. If that method returns another object, you can use it to call another method. We use the tv_show string in the above code to call the str.upper method. That method returns a new string in uppercase. We then use the uppercase string to call str.split to create a list of words.

You can chain as many method calls as necessary, though it can get messy if you

chain more than 2 or 3 methods on a single line. Fortunately, you can format your code to make it more understandable:

Copy Code
```
letters = 'abcdefghijklmnoqrstuvwxyz'

# Note that the parentheses surrounding this
# multi-line chain are required.
consonants = (letters.replace('a', '').
                      replace('e', '').
                      replace('i', '').
                      replace('o', '').
                      replace('u', ''))
print(consonants)    # bcdfghjklmnqrstvwxyz
```
Chaining only works when each method in the chain except the last returns an object with at least one useful method. (It doesn't matter what the last method returns.) This is rare in Python compared to most languages; many methods simply return None. Thus, chaining in Python isn't very common. However, you will encounter it. If you use chaining, keep things simple. A long chain of different kinds of operations can be challenging to understand.

Modules
You can use code from other Python files in your programs. No, we don't mean you should copy and paste the code. Instead, this is code you can make available to your program by importing it.

These files are called modules. Once you load a module, you can use its functionality in your program. This is the chief way Python programmers reuse code and keep their programs organized.

Python provides several hundred modules with the main Python distribution. You can read about these modules here. As of summer 2023, you can find nearly a half-million additional modules at PyPI.

The modules distributed with Python are always available. You don't need to download anything; you import them and use the code. For the modules at PyPI, you must download the modules before you can import them. You can do that with the pip command, which we describe in our PY101 course.

Finally, you can write your own modules; every Python file is a module. We'll show you how to use this feature in the Core Curriculum.

With all these modules, you don't need to reinvent the wheel whenever you write code. Reinventing the wheel can be fun but wastes time on real projects. You can look for modules to use when working on real projects. However, this can be a significant challenge. On large projects, you can have people devoted entirely to choosing and managing which modules the team uses.

That said, you're learning now. With standard and non-standard modules, you can solve, partially or entirely, many of the exercises and projects we'll ask you to work on. However, please don't do that. There's little benefit to going on a module hunt when you should be learning to write your own Python code. We'll tell you where you can use modules.

The import and from Statements
The import statement is used in Python to load code from Python modules into your code. The most basic way to load a module is to write import module_name, where module_name is the module's name. The import statement(s) are conventionally coded at the very top of the program file. For example:

Copy Code
```
import math

print(math.sqrt(math.pi))   # 1.7724538509055159
```

This code first imports the math module and then prints the square root of the pi constant. Note that both sqrt and pi are fully qualified names: both names are prefixed with the module name and a period (.). These fully qualified names mean you don't have to worry about naming conflicts with your code or other modules. (However, the name math might cause a conflict if you aren't careful.)

If you don't want to write math. every time you use a function or constant from the module, you can import just the names you want by using the from statement:

Copy Code
```
from math import pi, sqrt

print(sqrt(pi))             # 1.7724538509055159
```
Thisfrom statement imports the math module's pi and sqrt identifiers into your program. You can then use those names without qualification. Be aware, though, that from circumvents the naming conflict benefits of import.

The math Module
The information in this section may be helpful at Launch School. Still, it is not crucial to your development as a Python programmer. Take away what you can from the section, but don't struggle.

Python is heavily used in the science and mathematics realms. The amount of support available for Python in these realms is breathtaking. For instance, libraries like NumPy, SciPy, and Pandas provide advanced mathematical functions and data structures. Matplotlib, Seaborn, and Plotly offer potent data visualization tools. Even if you work in a different field, you may still need mathematical and visualization tools.

Even without such sophisticated libraries, most programs must perform some arithmetic or mathematical operations. That doesn't mean you need much math as a programmer; most programs need little more than basic arithmetic like a + 1.

However, sometimes you need a bit more: you may need to calculate the square root of a number, determine the value of Ï (pi), round floating point numbers, or even do some trigonometric calculations. You can use well-known algorithms to make these computations, but you don't have to. The Python math module provides a collection of functions and constants you can use without a complete understanding of how they work.

Let's say you want to calculate the square root of a number. It's possible to design and implement an algorithm that calculates the root. Why bother? You can use math.sqrt without first designing, writing, and testing some code:

Copy Code
```
import math
print(math.sqrt(36))        # 6.0
print(math.sqrt(2))         # 1.4142135623730951
```
Perhaps you need to use the number Ï for something. Again, you can calculate Ï with some precision with one of the many algorithms. However, the math.pi constant gives you immediate access to a reasonably precise approximation of its value:

```
print(math.pi)              # 3.141592653589793
```
The datetime Module
The information in this section may be helpful at Launch School. Still, it is not crucial to your development as a Python programmer. Take away what you can from the section, but don't struggle.

Suppose you want to determine the day of the week on which July 16th occurred in 2022. How would you go about that? You may be able to develop an appropriate algorithm on your own, but working with dates and times is a messy process. It's often much more complex than you think.

You don't have to work that hard, however. Python's datetime module provides classes for creating and manipulating objects representing a time and date. Compared to doing it yourself, it's not hard to determine the day of the week corresponding to a date. It's not easy, however; it involves two methods with particularly opaque formatting arguments:

Copy Code
```
from datetime import datetime as dt

date = dt.strptime("July 16, 2022", "%B %d, %Y")
weekday_name = date.strftime('%A')
print(weekday_name)                    # Saturday
```
We first import the datetime class from the datetime module (otherwise known as datetime.datetime), and then give the class an alias of dt.

Next, we use the strptime method to create an instance of the datetime.datetime class that represents "July 16, 2022". The second argument passed to strptime describes the how strptime should treat the first argument:

%B tells strptime to look for the spelled-out month name (July).
%d tells strptime to look for the day of the month (16).
%Y tells strptime to look for the 4 digit year (2022).
Note that the remaining characters in the second argument must match the corresponding characters in the first string:
At least one space must follow the month name.
A comma and at least one space must follow the day of month.
There can be no other characters in the string.
Finally, we call date.strftime('%A') to obtain the spelled-out weekday name (as given by %A).

Writing the format arguments for dt.strptime and dt.strftime can be demanding. This is a great place to give artificial intelligence a whirl. The author gave ChatGPT-4 two prompts to get the proper formatting codes:

How can I use strptime to convert "October 16, 2022" to a datetime object in Python?
How can I use strftime to determine the weekday name from a datetime object in Python?
As always, be wary of using AIs. The technology is still in its infancy, and answers are often incorrect. No matter how confident ChatGPT is about its answers, that doesn't mean they are correct. Fortunately, the answers the author was given matched their original solutions to this problem.

Messing with dates and times is incredibly difficult and error-prone. Even with the powerful tools provided by Python, it can be tough to figure out what you need to do.

The real benefit of datetime and some other date/time-related modules is that they abstract away most of the nasty details of working with dates and times. For a good time, search the web or use an AI to find information on some of these things:

How are leap years determined?
How many hours elapsed in the US between 1 a.m. and 5 p.m. on October 28, 1956?
How many hours elapsed during that same period in Indiana?
How many days were there in September 1752 in the US?
How many days were there in February 1918 in Russia?
When does Daylight Savings Time begin and end? How about in Australia?
Are all time zones an hour apart?
What time is it now in Iran? What about Nepal? Compare with your current time.
How do Greeks spell the name of the second month of the year?
How do Germans spell the name of the weekday the English-speaking world knows as Wednesday?
On what day of the week does the week begin?

Your head is probably swimming by now. Rest easy, though. The datetime module takes care of all this for you.

Function Definition Order
Let's look at the function definition order. Take a look at this code:

Copy Code
```
def top():
    bottom()

def bottom():
    print('Reached the bottom')

top()
```
Note that top is trying to call the bottom function, but bottom isn't yet defined on line 2. Will this code work? Think about it briefly and try to figure out what will happen and why. Once you've got your answer, go ahead and run the code.

What happened? The code ran just fine! It printed Reached the bottom. How does that work?

The answer lies in how Python executes def statements. When Python encounters a def statement, it merely reads the function definition into memory. It saves it away as an object in the heap. The function's body isn't executed until it's called explicitly. In this code, this read-and-save-but-don't-execute process occurs when Python encounters both functions. When we eventually invoke top on line 7, Python knows what and where top and bottom are.

What's more, Python also knows what code those functions contain. Thus, when top tries to invoke bottom, Python only has to find and call the bottom function object. That means the code runs correctly even though bottom was defined after top was.

What will happen if we call top before defining the function?

Copy Code
```
top()

def top():
    bottom()

def bottom():
    print('Reached the bottom')
```
Oops. That doesn't work. It fails with a NameError: name 'top' is not defined error. It doesn't work because we're trying to invoke top on line 1 before it gets defined. It has no idea what top is, so it gives up.

These issues arise when you start writing slightly longer programs with multiple functions. Many new developers stress over the order in which they define their functions, worried that Python won't be able to find them when the program runs. In practice, you don't have to worry about it. The only rule of thumb is that you should define all your functions before you try to invoke the first one. This is why Pythonistas almost always put the main program code at the bottom of the program after defining all functions.

Nested Functions
You can create functions anywhere, even nested inside another function:

Copy Code
```
def foo():
    def bar():
        print('BAR')
```

```
    bar() # BAR

foo()
bar() # NameError: name 'bar' is not defined
```
Here, the bar function is nested within the foo function. Such nested functions get created and destroyed every time the outer function runs. (This usually has a negligible effect on performance.) They are also private functions since we can't access a nested function from outside the function where it is defined.

Nested functions play a valuable role in Python's power and flexibility. You will meet them again if you continue on the Python learning path.

The global and nonlocal Statements

Functions add an additional level of complexity to the concept of scope. What happens when a function assigns a variable to a new value while a variable with the same name already exists in the outer scope? Enter the global and nonlocal statements.

By default, any variable that is defined in the outer scope of a function is also available inside that function:

Copy Code
```
greeting = 'Salutations'

def well_howdy(who):
    print(f'{greeting}, {who}')

well_howdy('Angie')
print(greeting)
```
Python assumes that all variables that are assigned a value inside a function are local variables. Even if a variable by the same name exists in an outer scope, Python will create a new local variable if the variable's name appears on the left side of an assignment. (This is the concept of shadowing that we met earlier.)

Copy Code
```
greeting = 'Salutations'

def well_howdy(who):
    greeting = 'Howdy'
    print(f'{greeting}, {who}')

well_howdy('Angie')
print(greeting)
```
In the above code, we have a variable named greeting defined in the global (top-most) scope. Its value is 'Salutations'. However, in the well_howdy function, we have an assignment of 'Howdy' to a variable named greeting. In this case, the greeting variable inside the function is completely independent from the greeting in the top-level code; it shadows the top-level greeting. Thus, the output from the above code is:

Copy Code
```
Howdy, Angie
Salutations
```
Python's global and nonlocal statements let the programmer override this behavior. They tell Python to use a variable that is defined elsewhere. Python will not create a new local variable when global or nonlocal is used to override this behavior.

In the case of global, Python is told to look to the outermost scope (the global scope) for the variable to be used. It works in any function. The nonlocal statement, however, only works in nested functions: functions that are defined inside an outer function. When Python processes a nonlocal statement, it looks for the associated variable in one of the outer functions.

In the well_howdy code shown earlier, suppose we want well_howdy to use the global greeting variable. To do that, all we have to do is include a global greeting statement in the function:

Copy Code
```
greeting = 'Salutations'

def well_howdy(who):
    global greeting
    greeting = 'Howdy'
    print(f'{greeting}, {who}')

well_howdy('Angie')
print(greeting)
```
When we run this program now, we'll see this output:

Copy Code
```
Howdy, Angie
Howdy
```
Instead of creating a new local variable in the well_howdy function, the presence of global greeting caused Python to use the global greeting variable instead.

An interesting variation occurs when the variable named by the global statement has not yet been defined:

Copy Code
```
def set_pi():
    global pi
    pi = 3.1415

set_pi()
print(pi)
```
In this case, the assignment ended up creating a new global variable named pi.

Suppose instead that we had some nested functions, each with its own local variable, foo:

Copy Code
```
def outer():
    def inner1():
        def inner2():
            foo = 3
            print(f"inner2 -> {foo} with id {id(foo)}")

        foo = 2
        inner2()
        print(f"inner1 -> {foo} with id {id(foo)}")

    foo = 1
    inner1()
    print(f"outer -> {foo} with id {id(foo)}")

outer()
```
Here, we have an outer function that we invoke on line 15. It creates an inner1 function and then invokes it. inner1, in turn, creates an inner2 function and then executes it. Each function -- outer, inner1, and inner2 -- assigns a local variable named foo to a value, calls the nested function, and then prints the value of its version of foo along with the id of foo. As you might expect, we get the following output:

Copy Code
```
inner2 -> 3 with id 4339312328
```

```
inner1 -> 2 with id 4339312296
outer -> 1 with id 4339312264
```
Notice that all 3 foos have different values and ids.

As with the earlier situations, we may want some of these functions to use one of the variables defined in the surrounding scope. For instance, assume we want foo in inner2 to reference the foo variable in inner1. Maybe we can use global:

Copy Code
```
def outer():
    def inner1():
        def inner2():
            global foo
            foo = 3
            print(f"inner2 -> {foo} with id {id(foo)}")

        foo = 2
        inner2()
        print(f"inner1 -> {foo} with id {id(foo)}")

    foo = 1
    inner1()
    print(f"outer -> {foo} with id {id(foo)}")

outer()
print(f"global -> {foo} with id {id(foo)}")
```
outputCopy Code
```
inner2 -> 3 with id 4339312328
inner1 -> 2 with id 4339312296
outer -> 1 with id 4339312264
global -> 3 with id 4339312328     # Note: same as `inner2`
```
Hmm. That's not what we wanted. Instead of updating the foo inside inner1, it created a new global variable and assigned that. No matter how deeply nested we are, the global statement stipulates that the listed identifiers are to be interpreted as global variables; that is, as identifiers in the global namespace.

The answer to this problem is to use the nonlocal statement:

Copy Code
```
def outer():
    def inner1():
        def inner2():
            nonlocal foo
            foo = 3
            print(f"inner2 -> {foo} with id {id(foo)}")

        foo = 2
        inner2()
        print(f"inner1 -> {foo} with id {id(foo)}")

    foo = 1
    inner1()
    print(f"outer -> {foo} with id {id(foo)}")

outer()
# print(f"global -> {foo} with id {id(foo)}")  # statement removed
```
outputCopy Code
```
inner2 -> 3 with id 4339312328
inner1 -> 3 with id 4339312328       # Same as inner2
outer -> 1 with id 4339312264
```
Ah. This time, we managed to change foo from inner1. However, it did not change foo in outer. We can address that readily by adding a nonlocal statement in inner1:

Copy Code
```
def outer():
    def inner1():
        def inner2():
            nonlocal foo
            foo = 3
            print(f"inner2 -> {foo} with id {id(foo)}")

        nonlocal foo
        foo = 2
        inner2()
        print(f"inner1 -> {foo} with id {id(foo)}")

    foo = 1
    inner1()
    print(f"outer -> {foo} with id {id(foo)}")

outer()
```
outputCopy Code
```
inner2 -> 3 with id 4339312328
inner1 -> 3 with id 4339312328
outer -> 3 with id 4339312328     # All 3 are the same
```
This time, we got 3's across the board. Effectively, the assignment in inner2 changed the value of foo in inner2, inner1, and outer.

One final note: unlike with the global statement, the nonlocal statement requires the named variable to already exist. You can't create the variable in the function that uses nonlocal.

Don't use global and nonlocal haphazardly. It's not usually good practice to reassign variables outside the local scope. Doing so almost always makes the program harder to read, understand, and maintain.

In general, if you think you need global or nonlocal, think carefully on whether there is a better solution. The global and nonlocal statements often reflect poor design choices that can be implemented more clearly. There are some situations where nonlocal is necessary, but global rarely is.